

**Expanding Robot Manipulation Capability Through
Dynamics-Constrained Motion Generation**

by

Anuj Pasricha

M.S., University of Colorado Boulder, 2019

B.S., University of Illinois at Urbana-Champaign, 2015

A thesis submitted to the
Faculty of the Graduate School of the
University of Colorado in partial fulfillment
of the requirements for the degree of
Doctor of Philosophy
Department of Computer Science
2026

Committee Members:

Alessandro Roncone, Chair

Bradley Hayes

Nikolaus Correll

Jeffrey Ichnowski (Carnegie Mellon University)

Mehmet Dogar (University of Leeds)

Pasricha, Anuj (Ph.D., Computer Science)

Expanding Robot Manipulation Capability Through Dynamics-Constrained Motion Generation

Thesis directed by Prof. Alessandro Roncone

Robots operating in the physical world must generate motions that are not only directed toward a task objective, but also admissible under the kinematic and dynamic constraints of the robot-world system. In manipulation, these constraints—for instance, contact geometry, friction, actuation limits, and object inertia—determine not only which states are reachable, but also which transitions are physically realizable and which regions of the motion space are meaningful to search. This thesis develops traditional and learning-based planning methods for manipulation problems in which both kinematic and dynamic constraints shape the set of achievable behaviors.

In developing these methods, my work expands the scope of robotic manipulation beyond geometric pick-and-place by incorporating dynamic non-prehensile skills and motions governed by object inertial properties, impulsive contact, and motion-induced forces. First, it introduces dynamics-aware planning methods that evaluate admissibility online through kinodynamic primitives, physics simulation, and control-aware search, enabling manipulation behaviors involving impulsive contact, coupled robot-object dynamics, and liquid transport. Second, it develops reusable kinodynamic planning representations that amortize repeated feasibility computation while preserving explicit dynamically feasible motion structure across related manipulation queries. Third, it transfers admissibility knowledge into conditional generative models that rapidly sample trajectories conditioned on task, embodiment, and physical parameters such as payloads and actuation limits. Fourth, it reframes constraint identification in terms of induced behavior, using anchor probes to compare constraints by behavioral equivalence rather than by perceptual or parameter similarity alone. Together, these contributions expand the range of manipulation behaviors that robotic systems can plan for and execute, and provide methods for doing so efficiently as task conditions, embodiments, and physical parameters of the robot-world system change.

Dedication

To family.

Acknowledgements

Thank you to my close friends for the conversations and laughter that kept the experience sane; to my colleagues in the Human Interaction & Robotics Group, for their collaboration and for all the work that made this body of research possible; to my committee, for their curiosity and guidance throughout this process; and, of course, to Alessandro, for giving me this opportunity in the first place, and for the continued support in the many years that followed.

Contents

Chapter

1	Introduction	1
1.1	Motivation	1
1.2	Contributions	4
1.2.1	Themes	4
1.2.2	Roadmap	5
2	Prior Work in Manipulation Planning with Dynamics	7
2.1	Planning as Search	7
2.2	Planning as Optimization	9
2.3	Planning as Inference	10
2.4	Hybrid Approaches	10
3	Online Evaluation of Kinodynamic Feasibility	11
3.1	Introduction: Poking as a Complementary Non-Prehensile Primitive	11
3.2	Background and Related Work	14
3.3	Methods	15
3.3.1	Formalization of the Poking Motion Primitive	16
3.3.2	Simulation Model for Poking	16
3.3.3	Motion Planning for Poking	18
3.4	Evaluation	23

3.4.1	Experimental Setup	24
3.4.2	Simulation Experiments	26
3.4.3	Real-World Experiments	29
3.5	Conclusion and Discussion	31
3.6	Case Study: Liquid Container Manipulation with Obstacle Avoidance	33
3.6.1	Task-Biased Geometric Planning	33
3.6.2	Analytical Static Models for Fill Level	34
3.6.3	Learned Dynamics Model for Trajectory Verification	35
3.6.4	Where Analytical and Learned Components Meet	35
4	Reusable Constraint Structure for Dynamic Feasibility	37
4.1	Introduction: Multi-Query Planning with Edge Bundles	37
4.2	Background and Related Work	40
4.3	Methods	42
4.3.1	Problem Setup	42
4.3.2	Edge Bundle Construction with the Robot Dynamics Model	43
4.3.3	Edge Perturbation for Lazy Propagation and Collision-Checking	44
4.3.4	Kinodynamic Motion Planner	47
4.4	Evaluation	50
4.4.1	Test Setup	50
4.4.2	Quantitative Analysis	52
4.4.3	Significance Testing	52
4.5	Conclusion and Discussion	55
4.6	Case Study: Whole-Body Non-Prehensile Manipulation Under Actuation Failures . .	56
4.6.1	Long Deployments and Locked-Joint Failures	56
4.6.2	Effects on Workspace, Manipulability, and Feasibility	56
4.6.3	Whole-Body Non-Prehensile Manipulation as Recovery	57

5	From Sampling to Generation: Planning as Conditional Sampling	60
5.1	Reframing Planning	60
5.2	Background on Diffusion	61
5.3	Levers for Tightening Constraint Compliance	64
5.4	What the Literature Emphasizes and What It Underexplores	65
6	Compressed Representation for Kinodynamic Feasibility	66
6.1	Introduction: Diffusion for Super-Nominal Payload Manipulation	66
6.2	Related Work	69
6.3	Diffusion Models for Trajectory Generation	71
6.4	Payload-Conditioned Trajectory Denoising	72
6.4.1	Training Data	72
6.4.2	Payload-Conditional Trajectory Generation	74
6.5	Evaluation	77
6.5.1	Comparison of Payload Encodings	77
6.5.2	Comparison across Planner Types	79
6.6	Conclusion of this Work	82
6.7	Limitations	84
6.8	Case Study: Diffusion Under Failure-Induced Embodiments	86
6.8.1	Motivation	86
6.8.2	Embodiment-Aware Conditioning	87
6.8.3	Results	91
6.9	Looking Forward	94
7	Behavioral Anchor Conditioning for Constrained Trajectory Diffusion	95
7.1	Introduction	95
7.2	Related Work	96
7.3	Methods	99

7.3.1	Problem Setup	100
7.3.2	Conditional Trajectory Diffusion	100
7.3.3	Diagnostic Anchor Signatures	101
7.3.4	Entropy-Based Anchor Selection	102
7.3.5	Behavioral Equivalence and Encoding New Environments	103
7.4	Evaluation	104
7.4.1	Experimental Setup	104
7.4.2	Metrics	105
7.4.3	Collision-Avoidance Results	105
7.4.4	Discussion of Results	109
7.5	Discussion	110
7.5.1	Extension to Other Constraint Oracles	110
7.5.2	Limitations	110
8	Discussion and Future Directions	112
8.1	Summary of Contributions and Key Takeaways	112
8.1.1	Poking as a Complementary Dynamic Manipulation Primitive	112
8.1.2	Multi-Query Planning via Lazy Edge Bundles	113
8.1.3	Conditional Trajectory Diffusion for Dynamic Constraint Satisfaction	114
8.1.4	Behavioral Anchor Conditioning for Task-Agnostic Trajectory Seeding	115
8.2	Lessons and Open Questions	116
8.2.1	Constraints and System Design	116
8.2.2	Multimodal Capabilities Beyond Geometric Skills	118
8.2.3	Scope and Limits of Statistical Learning	119
8.3	Final Synthesis	122
	Bibliography	123

Tables

Table

3.1	Simulation task times and success rates across planners.	24
3.2	Number of executed actions per planner in simulation.	27
3.3	Real-world task performance across planners.	29
4.1	Workspace reachability under joint failure with whole-body NPM recovery.	59
6.1	Success rate of the Baseline methods and the DEFT method.	92
6.2	Real-world task results over 10 runs.	94

Figures

Figure

1.1	Overview of thesis work from kinodynamic planning to diffusion-based generation.	4
2.1	Motion planning approaches across search, optimization, and inference.	8
3.1	Poking as a skill and failure recovery tactic for planar non-prehensile manipulation.	13
3.2	PokeRRT action sampling and graph expansion pipeline.	17
3.3	PokeRRT evaluation across six obstacle scenarios.	22
3.4	Two-robot collaborative poking task in non-overlapping workspace (S6).	30
3.5	Real-world task time breakdown by planning and execution phase.	32
3.6	Tilt angle comparison between SFRRT* and SpillNot baseline.	36
4.1	Dynamic constraint examples motivating kinodynamic planning.	39
4.2	Lazy propagation probability estimation via state perturbation.	45
4.3	LazyBoE lazy candidate path evaluation and tree extension.	48
4.4	Time to first solution across planners.	51
4.5	Final solution cost comparison across planners.	53
4.6	Success rate comparison across planners.	53
4.7	Number of solutions explored per planner.	54
4.8	Two locked multi-joint failure configurations on the Franka Panda.	57
5.1	Diffusion model components for admissible trajectory generation.	62

6.1	Diffusion model for super-nominal payload trajectory generation.	67
6.2	Plan-and-filter process to create training data.	73
6.3	1D UNet architecture conditioned on payload embeddings.	75
6.4	Aggregate success rate by payload and conditioning method.	78
6.5	Planning time and success rate across payload conditions.	80
6.6	Joint torque profiles during super-nominal payload transport.	83
6.7	DEFT embodiment-aware trajectory synthesis under joint failure.	88
6.8	DEFT architecture: embodiment and task conditioning.	89
6.9	Real-world DEFT evaluation: erasing and drawer tasks.	93
7.1	Diagnostic anchor selection via binary feasibility oracle.	97
7.2	Collision-avoidance evaluation comparing anchor conditioning to baselines.	106

Chapter 1

Introduction

1.1 Motivation

Robotic manipulation has advanced substantially along two complementary domains: deployed industrial systems operating in structured settings, and learned robot policies that aim to generalize across broader classes of objects, scenes, and tasks. In industrial settings, manipulation systems now support tasks such as bin picking, depalletizing, truck unloading, machine tending, assembly, and warehouse automation. These systems are typically built as engineered pipelines: perception modules localize objects and estimate scene structure, planners compute feasible motions, and controllers execute those motions on physical hardware. Their success relies on a combination of constrained task design, calibrated sensing, compute-efficient motion generation, and accumulated engineering effort over repeated deployment cycles, enabling reliable manipulation in settings where the objects, workspaces, and task objectives are sufficiently well-specified.

In parallel, the field has seen growing interest in general-purpose learned policies. Rather than designing a new perception-planning-control stack for each task, these methods aim to acquire manipulation behaviors from large and diverse datasets, simulation, teleoperation, reinforcement learning, or combinations of these sources [18, 157, 22, 3, 160]. The promise of this direction is that robots may eventually generalize across objects, scenes, and task variations without requiring extensive task-specific engineering. Together, engineered pipelines and learned policies represent two ways that modern manipulation systems are being scaled: one through modular systems that can be debugged and adapted to real deployments, the other through data-driven generalization.

Despite their methodological differences, many successful manipulation systems across both settings assume the task can be treated as a primarily geometric or quasi-static problem. In the prehensile (or pick-and-place) case, once an object is securely grasped, its motion is largely determined by the motion of the robot, and planning can focus on grasp validity, reachability, collision avoidance, joint limits, and other configuration space constraints. Even when manipulation is not strictly prehensile, many tasks are executed slowly enough, or under sufficiently controlled contact conditions, that inertial effects and higher-order dynamic constraints do not play a role.

This abstraction has motivated decades of work in grasp synthesis, motion planning, and robot control, and it continues to underlie many modern manipulation systems [12, 14, 85]. Recent advances in geometric planning infrastructure have further strengthened this paradigm, including GPU- and SIMD-parallelized collision checking and forward kinematics [138, 140], large-scale robot motion datasets [102], and high-fidelity simulation environments [25]. However, many useful manipulation behaviors are not captured by geometry alone. Humans routinely exploit physical effects that are difficult to express as purely kinematic constraints: pushing an object without grasping it [89, 136], striking or throwing to accumulate velocity [155], using momentum to move heavy objects, adapting motion to changing payloads, or continuing a task after a mechanical failure [17]. In the context of robotic manipulation, a pushing action cannot be evaluated only by the collision-free path of the end-effector, because the object trajectory depends on surface friction and object inertia [89]. Similarly, a robot transporting a liquid-filled container cannot reason only about the swept volume of the cup, because accelerations may induce sloshing or spilling [1].

This thesis studies manipulation through the lens of constraints on motion generation. I use the term *kinematic constraints* to refer to constraints on the geometry of motion, including joint position limits, self-collision avoidance, environmental obstacle clearance, reachability, singularity avoidance, and valid grasp sampling. These constraints define where the robot can be placed and how it can move through configuration space without violating geometric feasibility. I use the term *dynamic constraints* to refer to constraints that depend on the physics of motion. Dynamic constraints arise from two broad sources: i) *higher-order robot constraints*: limits on velocity, accelera-

tion, jerk, torque, traversal time, and related quantities imposed by the robot’s kinematic structure, actuators, and control limits [112, 11]; and, ii) *task-induced dynamic constraints*: constraints imposed by robot-object interaction, including contact mechanics, friction, inertial coupling, payload mass, deformable object behavior, fluid motion, and motion-induced phenomena such as sloshing. Unlike purely geometric constraints, these constraints depend not only on the configuration of the system but also on the trajectory by which that configuration is reached.

The central motivation of this thesis is that dynamic constraints are not secondary details to be handled only after a geometric plan has been found—they shape the feasible motion space itself. Accounting for them during motion generation can expand the class of behaviors available to robots, especially in tasks where useful behavior depends on contact, force, inertia, momentum, or the limits of the robot’s own embodiment. Expanding manipulation capability therefore requires moving beyond kinematic behaviors to plan under dynamic constraints, enabling robots to reason about both their available embodiment and the complex physical interactions induced by the task.

1.2 Contributions

My Contributions

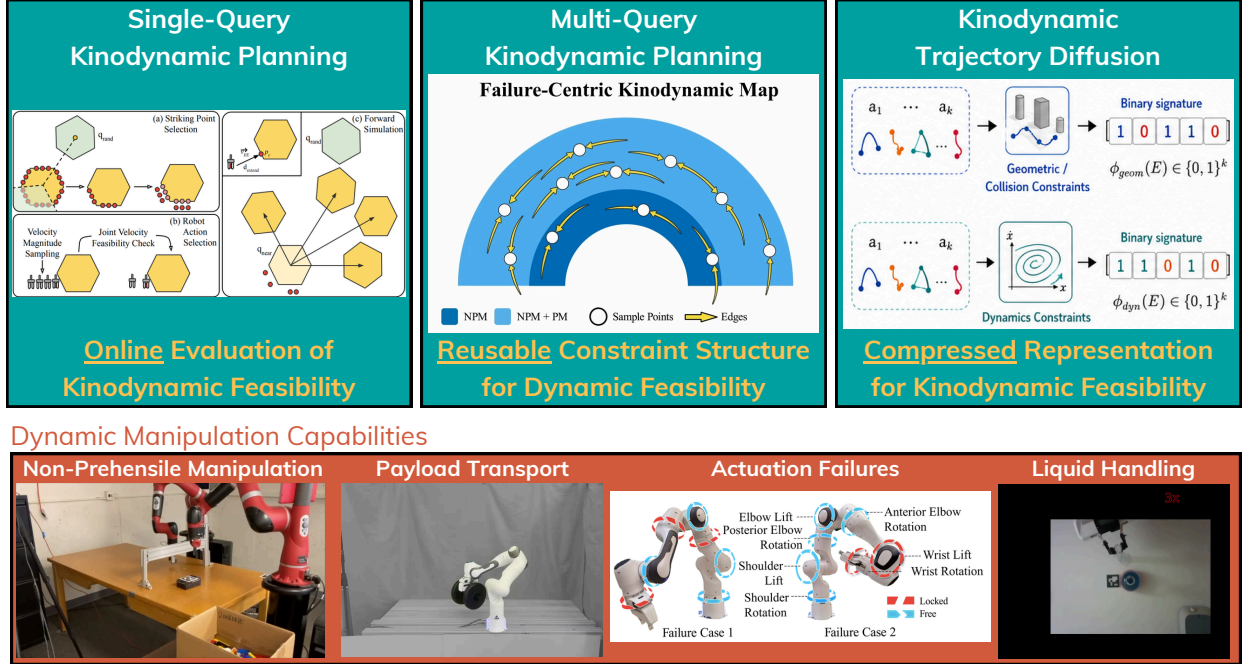


Figure 1.1: My thesis work progresses from sampling-based kinodynamic planning to diffusion-based trajectory generation, leveraging various modeling approaches applied to a broad class of manipulation problems that are subject to both kinematic and dynamic constraints.

1.2.1 Themes

This dissertation develops planning and learning methods for dynamic manipulation that expand robot capability beyond predominantly geometric pick-and-place to tasks governed by object inertial properties, impulsive contact, and motion-induced forces. Three families of methods are developed, each incorporating dynamic feasibility into the motion generation process differently:

Part I: Online Evaluation of Kinodynamic Feasibility (Chapter 3). The first part develops dynamics-aware planning methods for non-prehensile manipulation involving impulsive contact, coupled robot-object dynamics, and liquid transport, exploiting physical constraints di-

rectly through kinodynamic primitives and simulation-based forward reasoning. In this setting, the planner maintains an explicit connection between sampled actions and dynamically feasible outcomes, but pays the cost of physics-based reasoning during each planning query.

Part II: Reusable Constraint Structure for Dynamic Feasibility (Chapter 4). The second part develops lazy reusable kinodynamic planning representations for multi-query manipulation, reducing repeated planning cost while preserving explicit dynamically feasible motion structure across related problems. Kinodynamic admissibility is not fully recomputed from scratch; instead, prior feasibility computation is stored in a reusable structure and selectively validated when needed.

Part III: Compressed Representation for Kinodynamic Feasibility (Chapter 5–Chapter 7). The third part develops conditional generative models for dynamic manipulation that rapidly sample trajectories conditioned on physical parameters such as payload mass and embodiment configuration. The thesis further considers scalability to multiple tasks and high-dimensional problem spaces by framing the problem of constraint identification in terms of induced behavior rather than perceptual or parameter similarity.

1.2.2 Roadmap

Chapter 3 develops methods in which dynamic feasibility is evaluated online: constraint knowledge resides in a forward model, and each planning query incurs the full cost of feasibility checking. The chapter introduces poking as a dynamic non-prehensile manipulation primitive, using impulsive contact within a sampling-based planner to achieve object rearrangement without explicit contact modeling. Across six scenarios, poking achieves higher success rates and lower task completion times than pushing and grasping baselines. A case study within the chapter applies the same online-evaluation principle to spill-free liquid transport in cluttered environments, where coupled robot-object dynamics constrain feasible motion; the resulting planner expands feasible spill-free solution spaces by up to $3\times$ over prior approaches.

Chapter 4 introduces the first amortized representation: constraint knowledge is moved upstream into precomputed control-parameterized edge bundles, reducing per-query cost while

preserving kinodynamic feasibility through lazy propagation and collision checking. A case study within the chapter carries this precomputation principle into the task space, constructing reusable motion structure for whole-body manipulation under locked multi-joint failures and maintaining up to 92% of the nominal reachable workspace under failures.

Chapters 5 and 6 develop a more scalable form of amortization: constraint knowledge is absorbed into the parameters of a conditional trajectory distribution during training, eliminating runtime feasibility checking entirely. Chapter 5 reframes motion generation as conditional sampling, establishing the conceptual bridge between kinodynamic planning and trajectory diffusion models. Chapter 6 develops a diffusion-based trajectory generator conditioned on task-level dynamic properties such as payload mass, enabling super-nominal payload manipulation without runtime constraint checking. The method achieves super-nominal payload manipulation—67.6% workspace accessibility at $3\times$ rated payload capacity—in approximately 10 ms without runtime constraint checking, and generalizes to fail-active manipulation under actuation failures. A case study within the chapter extends the same conditioning principle to fail-active manipulation under actuation failures, demonstrating adaptation to changes in robot embodiment.

Chapter 7 presents a more broadly applicable form of conditioning for the kinodynamic trajectory diffusion model. A task-agnostic conditioning mechanism represents dynamic constraints as robot state samples to generate near-feasible trajectory seeds that improve optimization convergence and feasibility across diverse manipulation tasks without task-specific engineering.

Finally, **Chapter 8** synthesizes the thesis contributions, discusses their implications for dynamics-aware robot manipulation, and outlines directions for future research.

Chapter 2

Prior Work in Manipulation Planning with Dynamics

Higher-order constraints such as joint torques and end-effector accelerations can be addressed through several approaches that range from “plan-and-filter” methods which post-process geometric paths [1, 67] to kinodynamic planners that directly incorporate dynamics during planning [79, 99]. Broadly there are three classes of methods (Figure 2.1) spanning search, optimization, and inference—the literature is vast so we focus our approaches on manipulation tasks which involve the consideration of environment or object dynamics in addition to robot dynamics.

2.1 Planning as Search

At its very basic, this class of methods involves finding a configuration-space path is generated to satisfy geometric constraints including collision avoidance and joint limits [56, 72, 103]. Subsequently, time parameterization is applied—through bang-bang control or optimization techniques—to minimize traversal time while respecting velocity and acceleration constraints [113, 112, 83, 11]. The resulting trajectory is then evaluated against environmental dynamic constraints [31]. This sequential process iterates until identifying a trajectory that successfully enables the desired dynamic robot-object interaction. Although modular in nature which allows for the tuning of each component, this sequential framework may yield sub-optimal solutions and be computationally inefficient due to iterative refinement cycles when trajectories fail to satisfy all constraints, particularly when environmental models are inaccurate or constraints are too restrictive.

Kinodynamic planning simultaneously considers robot and object dynamics through inter-

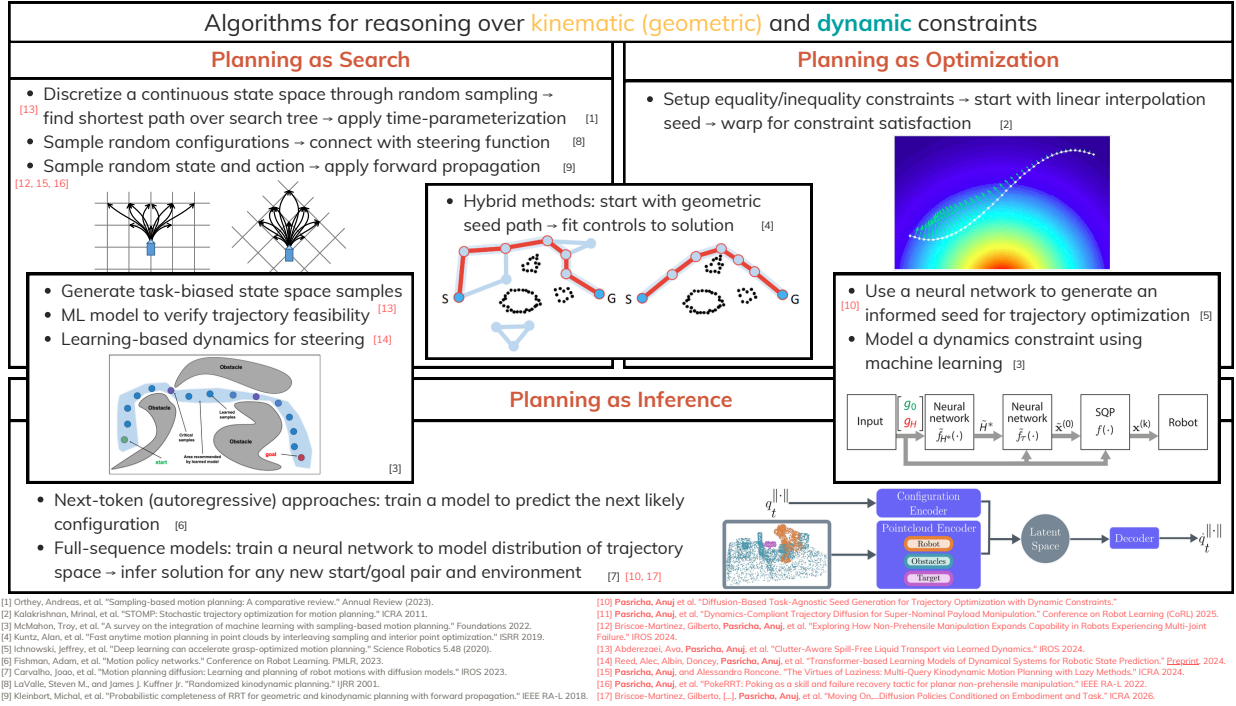


Figure 2.1: Motion planning approaches for robotic manipulation with kinematic and dynamic constraints span three broad categories: search, optimization, and inference, with hybrid methods existing at each of their intersections.

leaved state space exploration and control propagation. Many approaches employ steering functions (inverse models) to connect states. Examples include DIMIT-RRT’s quadratic steering function that obeys joint acceleration limits and non-zero initial and goal velocities [68], AVP’s computation of reachable velocities along a geometric path [114, 51], and LQR-RRT*’s linearized dynamics with quadratic cost functions [110, 20]. Alternatively, forward control propagation methods use random states and actions to construct dynamically feasible trees [108]. SyCLoP and KPIECE guide exploration via state space discretization [116, 137], while SST optimizes paths through pruning and heuristic biasing [79]. GBRRT and GABRRT employ multiple propagation methods with bidirectional search [99]. Other techniques pre-map valid motions and search discretized state and control spaces [130, 16]. Despite theoretical completeness guarantees under certain conditions [63], these methods face practical limitations in high-dimensional spaces and exhibit high variance in planning times, particularly when dealing with uncertainty and planning near operational limits [62].

2.2 Planning as Optimization

Optimization-based methods naturally integrate dynamics constraints by framing motion planning as the minimization of objective functions (e.g., smoothness, time optimality) subject to equality and inequality constraints that enforce collision avoidance, robot limitations, and dynamic feasibility [119, 52, 127, 141, 105, 45]. Typically formulated as quadratic programs, these approaches guarantee constraint satisfaction but demonstrate high sensitivity to initialization quality. This dependency has prompted the development of hybrid methodologies that bootstrap the optimization process with sampling or learning-based initializations [44, 67, 151]. These integrated approaches leverage the strengths of both paradigms: the exploration capabilities of sampling/learning methods for identifying promising trajectory seeds, and the precision of optimization for refining solutions to satisfy strict dynamic constraints. Optimal control formulations for dynamics-aware manipulation also exist [4, 60]. Despite their theoretical rigor, they face practical limitations in handling complex constraints and achieving reliable convergence for real-world manipulation problems.

2.3 Planning as Inference

Finally, learning-based approaches offer promising directions for both generalizability and efficient trajectory generation while respecting system dynamics. Autoregressive models implicitly encode dynamics through dataset design [30, 118], while recent diffusion-based approaches provide non-autoregressive alternatives with improved sampling diversity [49, 21, 123, 128, 41]. Task-level diffusion policies, conditioned on visual embeddings [22] or abstract specifications such as returns, constraints, and skills [2], demonstrate potential for generating diverse behaviors from high-level task specifications. While these approaches yield impressive real-world behaviors, provide continual improvement over deployment lifespan, and avoid explicit state space exploration for each new query, they typically focus on position-controlled tasks and grasping-based manipulation and incur significant training and data acquisition costs.

2.4 Hybrid Approaches

Hybrid approaches can also exist at each intersection of the aforementioned methods and leverage the benefits of each to create a more robust system. At the intersection of search-based and optimization-based methods, hybrid techniques typically begin with a seed path generated through sampling-based algorithms and subsequently fit controls to this solution [67]. This approach combines the exploratory advantages of search-based methods with the refinement capabilities of trajectory optimization. In search-based planning, neural networks generate task-biased state space samples [90], while in optimization contexts, they provide informed seeds for trajectory optimization [44]. This integration addresses the initialization sensitivity of pure optimization approaches while maintaining their convergence properties.

Despite progress across methods in search, optimization, and inference, significant challenges remain in developing unified approaches that can handle complex robot-object dynamics while maintaining computational efficiency and generalizability across multiple tasks—a gap my work directly addresses to enable a broad range of new capabilities that rely on kinodynamic reasoning.

Chapter 3

Online Evaluation of Kinodynamic Feasibility

Chapter 2 reviewed motion-generation methods for manipulation under dynamic constraints, including kinodynamic search, optimization-based formulations, learning-based models, and hybrid approaches. This chapter focuses on kinodynamic search with forward propagation: candidate actions are sampled by the planner and feasibility is evaluated entirely *online*, by rolling out a physics simulation as the forward model. This enables poking to act as a complementary non-prehensile primitive that extends manipulation capability beyond pick-and-place by using brief impulsive interactions to move objects where pick-and-place fails. A case study at the end of the chapter applies the same online-evaluation principle to spill-free liquid transport in cluttered environments, where coupled robot-object dynamics constrain feasible motion.

3.1 Introduction: Poking as a Complementary Non-Prehensile Primitive

Humans engage naturally in multiple forms of dexterous manipulation that involve grasping, pushing, poking, rolling, and tossing objects [19]. Consequently, the development of similar functionality in autonomous machines is an essential milestone for robotics and an area of active research with fundamental work needed ahead [12]. However, the human manipulation skill that has attracted the most attention from roboticists is prehensile manipulation, or *grasping*. Manipulation by grasping is attractive primarily because, once an object is grasped, it generally does not need to be tracked over time and uncertainty on its state is reduced. However, grasping is limited in capability by i) reachability of the robot arm, ii) mechanical design limitations of the end-effector,

iii) physical properties of the object being manipulated, and iv) accuracy of the perception system.

Non-prehensile manipulation (i.e., any kind of manipulation not involving grasping, hereinafter referred to as NPM) offers a complementary solution to prehensile manipulation by significantly expanding the size (intended as the set of reachable configurations) and dimensionality (intended as the number of degrees of freedom) of the operational space of even the simplest robot manipulator [82]. In other words, NPM can be used to manipulate objects when conventional grasping-based manipulation is infeasible or unnecessary. Realistic robot applications might expect the robot to operate in dense clutter, in the presence of occlusions, or in ungraspable configurations—for example, the target object is in a pose that is not directly reachable by the end-effector or the target object is too large or too heavy. These applications may result in failure modes for robot operation through traditional grasping. Consequently, in such situations it is beneficial to complement the robot’s skillset with NPM primitives. Indeed, NPM can be used both as a skill and a failure recovery mechanism, which points to the versatility of this paradigm.

In this work, we demonstrate the utility of NPM through poking, a skill that allows fast object manipulation and expands the size of a manipulator’s reachable workspace. The basic idea behind this work is detailed in Figure 3.1. *Poking* is a NPM primitive wherein a robot end-effector applies an instantaneous force to an object of interest to set the object in planar translational and rotational motion (*impact* phase). The object eventually slows down and comes to rest due to Coulomb friction (*free-sliding* phase). Poking has a multitude of desirable properties that makes it complementary to grasping and serves as a generalized form of pushing where applied impulse forces are low [42, 107]. These characteristics make poking especially suitable for industrial settings where robots need to operate in their delineated workspaces alongside other robots or humans while still being able to pass objects. Additionally, in logistics or e-commerce settings, poking can be used for either stowing objects in boxes or in synergy with pick-and-place to optimize operations by increasing speed and coverage area.

We design and implement two closed-loop, kinodynamic, sampling-based planners called *PokeRRT* and *PokeRRT** which decouple skill modeling and path planning and specifically focus on

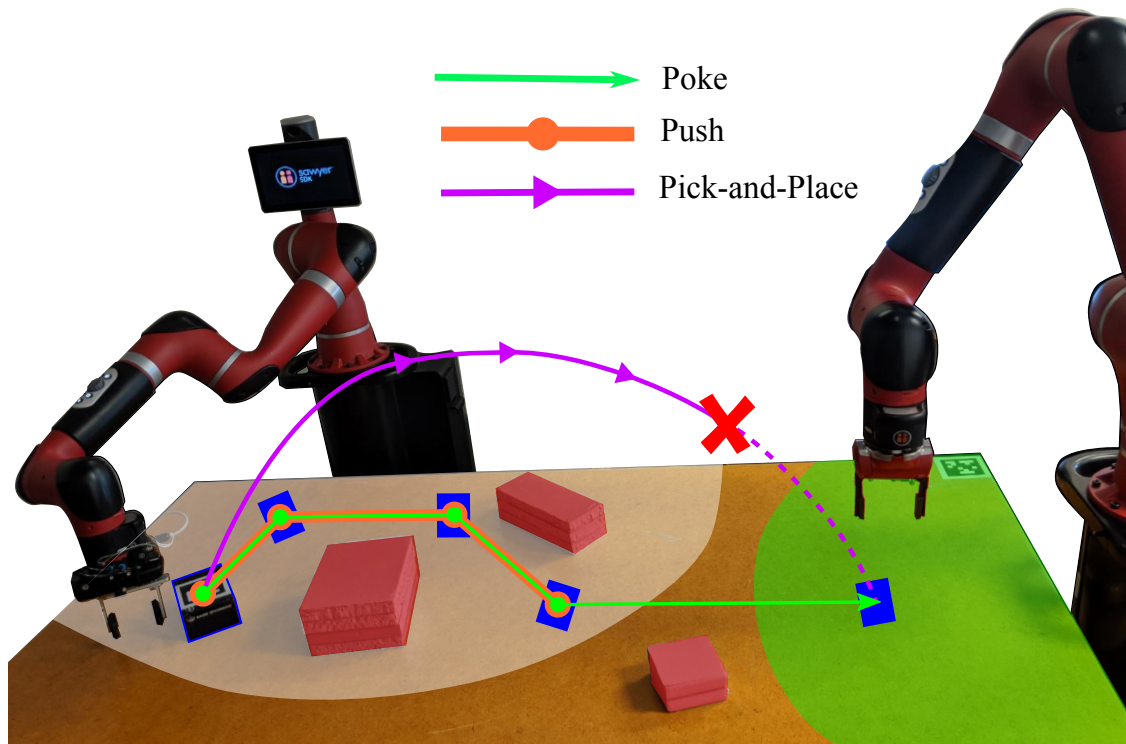


Figure 3.1: This work demonstrates poking as a skill and a failure recovery tactic to increase the portfolio of capabilities at the robot’s disposal. Here, an object (blue) is located in an obstacle-rich (red) workspace with non-overlapping reachable regions for each robot defined by beige and green shading. The first robot manipulates the object into the green region successfully via poking (path shown in green), but fails to do so via pushing or grasping (paths shown in orange and purple, respectively).

leveraging the following advantages of poking over pushing and grasping: i) it does not require constant contact between the manipulator and the object, therefore greatly expanding the size of the manipulator’s workspace, ii) it does not impose restrictions on the shape or size of objects that can be manipulated, and iii) it is inherently faster and therefore capable of covering large distances in short periods of time.

3.2 Background and Related Work

Research in non-prehensile manipulation dates back to the 90s [82, 122, 142], with the vast majority of prior work leveraging heuristics or analytical models based on simplifying assumptions. Related work has focused on skills such as throwing [155], poking [42], and pushing [146, 132, 88]. More recently, pushing manipulation has received increased attention due to availability of large-scale datasets and the inherent controllability of the skill [9]. Pushing operates under the quasistatic assumption (i.e., the inertial effects of robot–object and object–environment interaction are ignored) to reduce modeling complexity, thereby limiting robot velocities and accelerations. On the contrary, poking must plan around the non-negligible effects of inertial forces: the object continues moving after robot–object contact is broken, thus allowing for faster planar manipulation of objects. Past work in pushing manipulation incorporates simulation and analytical models in a motion planning loop to get the next feasible state [161, 158]. Additional contributions in push modeling involve combining object state estimation with affordance prediction from image data to determine contact points for achieving the optimal push [64] and creating a deep recurrent neural network model to model push outcomes for a variety of objects [76]. However, both approaches use a greedy planner operating in obstacle-free environments. In this work, we propose a sampling-based kinodynamic framework that is capable of planning dynamic collision-free paths in the object configuration space.

Past work on sampling-based planning techniques for non-prehensile manipulation involve kinodynamic approaches to rearrangement planning and whole-arm manipulation. This work is constrained by the quasistatic assumption, thereby limiting object manipulation speed through the use of non-dynamic primitives [61]. However, when operating in a dynamic regime, lack of kinematic

modeling of the robot and lack of contact modeling between the robot arm and environmental objects may result in unusable actions explored during the planning phase [37]. These approaches also employ an open-loop paradigm and do not consider robot, sensing, and model uncertainty. Moreover, open-loop kinodynamic planners that consider uncertainty in the planning process may generate conservative plans that do not exploit robot and object dynamics to their full extent [95]. Our approach uses a simulation engine that considers robot kinematics and contact while planning and operates in a closed-loop manner, replanning if the resultant pose is outside a certain threshold.

In all, evidence from prior work suggests that *poking*—sometimes referred to as “releasing” or “impulsive manipulation”—is a relatively unexplored primitive. Analytical models for poking dynamics are restricted to rotationally symmetric objects or situations where pusher–object contact can be geometrically modeled [42, 159]. The need for specialized impulse-delivery apparatus to achieve poking is explored in [42]. This makes planar manipulation of objects cumbersome due to the manual relocation of the apparatus required. In this work, we use a standard open-chain robot arm equipped with an electric parallel gripper to deliver impulses using joint space velocity control.

In order to generate impulse-based action paths that obey the robot’s kinematic constraints and produce feasible object motion, we use a PyBullet [25] simulation environment as the forward physics model in the planning loop. While simulation may not accurately capture real-world dynamics [24], the closed-loop nature of our planning framework compensates for this approach. This effective use of simulation captures essential characteristics of robot and object dynamics at faster than realtime speeds and in a safe manner without inducing significant wear-and-tear caused by dynamic trajectory execution on a real robot.

3.3 Methods

Here, we present an overview of the proposed approach for poking (Section 3.3.1), its characterization in a simulated environment (Section 3.3.2), and our motion planning algorithms, *PokeRRT* and *PokeRRT**, that leverages the simulated environment to plan a collision-free path for the target object (Section 3.3.3). We also introduce empirically-driven heuristics to *PokeRRT* that

leverage the large and quick displacement properties intrinsic to poking manipulation.

3.3.1 Formalization of the Poking Motion Primitive

Poking manipulation is modeled as a process composed of two phases: i) *impact*, where the robot end-effector makes instantaneous contact with the object; and ii) *free-sliding*, where the object slides on a planar surface and comes to a stop due to Coulomb friction. As detailed in Figure 3.2c, two parameters are required to describe the first phase of poking: the point of contact p_c (i.e., where on the contour of the object to strike), and the magnitude of the impact velocity $\|\vec{v}_{EE}\|$. Slippage at the contact point between the end-effector and the object may lead to non-linearities; therefore, we fix the direction of $\vec{v}_{EE}$ as being normal to the object’s contour. To apply an instantaneous force, the robot must come to a complete halt upon contact with the object. Therefore, collision between the end-effector and the object is treated as an elastic collision. The motor torques applied to stop the end-effector upon contact prevent the impulsive interaction from being truly elastic; however, this can be ignored by stopping the end-effector slightly past the contact point.

Given a uniform-density object of mass m with a coefficient of friction μ whose center of mass starts in an initial planar pose $q_i = (x_i, y_i, \theta_i)$ and the control parameters (see Figure 3.2c) that determine the contour of the object to strike (p_c) and the velocity magnitude ($\|\vec{v}_{EE}\|$), we can solve the second phase of poking and determine the final pose of the object $q_f = (x_f, y_f, \theta_f)$ using a physics simulation engine.

3.3.2 Simulation Model for Poking

In order to understand the effects of impulsive forces on objects, we use the PyBullet physics simulation engine [25]. PyBullet models rigid body dynamics by performing numerical integration over time with equations of motion to solve for object position and velocity. Joint constraints, contact forces, and external forces, are taken into account in the forward dynamics solver of the engine and modeled as constraints in a Linear Complementarity Problem (LCP).

The main point of interest when modeling impulsive interactions is contact force. In the

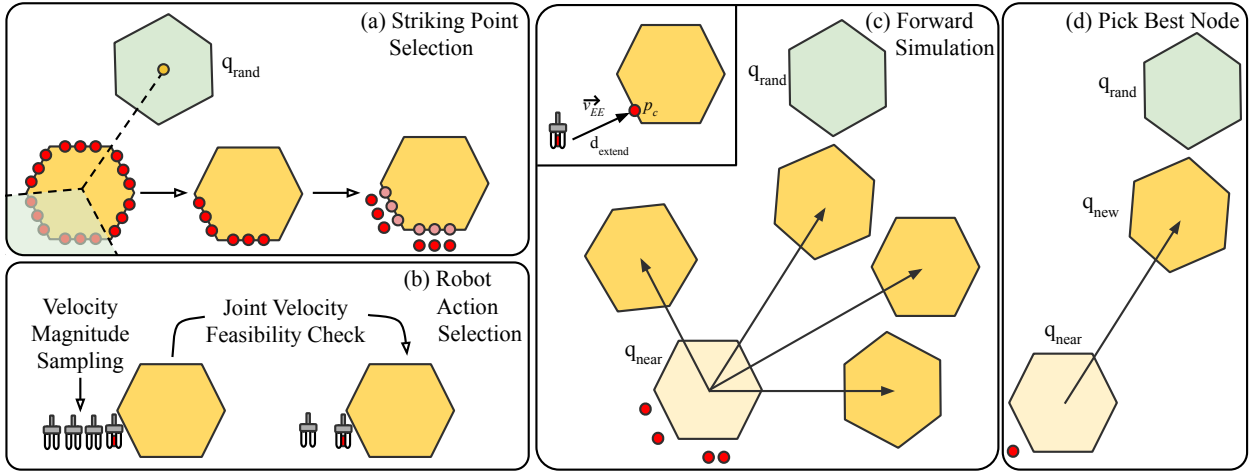


Figure 3.2: Path planning for poking consists of 2 steps: action sampling (a, b) and graph expansion (c, d). (a) Points are sampled uniformly on the object contour (red) and filtered through a conical region originating from the target position (green). Striking points are generated by extending away from contour points in the normal direction. (b) End-effector velocity magnitudes are sampled for each striking point and filtered out if joint velocities are infeasible due to mechanical limitations of the robot. (c) Feasible actions are applied in simulation to get resultant poses. (d) The resultant pose closest to the target position is added to the planning graph.

context of poking, contact is primarily dominated by frictional interactions between i) the robot end-effector and the object of interest, and ii) the object and the environment (i.e., the object’s planar support surface and surrounding obstacles). Analytical models for contact make simplifying assumptions that do not fully represent the complexity of real-world dynamics, including the non-linear nature of friction and actuator degradation and latency. Additionally, they do not generalize well to a diverse set of objects. Learning-based approaches, while capable of achieving generalization and modeling uncertainty, are data-inefficient. Collecting abundant and task-representative data in real-world robotics is expensive with regard to the time taken and the wear-and-tear caused on a real robot through repetitive, and potentially, high-acceleration trajectory executions such as those characteristic to poking manipulation. Executing such trajectories and modeling collisions along the action path with the object and the environment is therefore far safer in simulation.

While simulation does not perfectly capture the inherent complexity and stochasticity of real-world contact dynamics due to the simplistic nature of the underlying analytical models used, it nonetheless provides a good balance between pure learning and analytical models by ensuring interaction modeling is both cheap and safe while encapsulating the essential characteristics of robot–object and object–environment interactions. The closed-loop nature of our proposed motion planner also compensates for any inaccuracies in simulation modeling while executing poke plans in the real-world. That is, if a resultant pose violates a predefined object pose threshold, we compute a new poke plan from the current object pose to the goal region. This crucial feature allows our planner to plan feasible poke paths in simulation and execute them in the real-world.

3.3.3 Motion Planning for Poking

3.3.3.1 Object Configuration Space

Our planner operates in $\mathcal{C}$, the continuous (x, y, θ) configuration space of the object. x and y are confined by the limits of the planar workspace, whereas θ is defined in the range $[-180^\circ, 180^\circ)$.

3.3.3.2 Action Sampling

The proposed planner decouples skill modeling and object path planning to allow for the evaluation of multiple skill models which directly improve planning outcomes. We achieve this through an action-oriented approach by employing filtering steps to pick a set of valid actions $\mathbf{a}_{valid} = \langle \mathbf{p}_c, \|\vec{\mathbf{v}}_{EE}\| \rangle$ to apply at a given object configuration q . A sampling approach that yields actions which can be simulated in a physics engine is desirable since inverse modeling of frictional contact and object motion may either be intractable or not guarantee feasible robot motion.

To achieve the correct direction of motion, candidates for object contour points that are ideal for impact must lie on the side nonadjacent to the target object position. Given an object configuration q , contour points $\mathbf{p}_{contour}$ are chosen as acceptable candidates if they lie within a cone originating from a target pose q_{rand} (Figure 3.2a). The target pose is sampled during path planning as described in Section 3.3.3. For each candidate contour point $p_c \in \mathbf{p}_{contour}$, a striking point p_s is computed at a fixed distance, d_{extend} , from p_c in the normal direction away from the object (Figure 3.2c). The robot engages in joint space velocity control to apply a poke in its operational space by moving from p_s to p_c at velocity $\vec{\mathbf{v}}_{EE}$.

Collision-free robot joint configurations, $\boldsymbol{\theta}_s$ and $\boldsymbol{\theta}_c$, for both p_s and p_c are computed using *TRAC-IK* [10]. Since impactful contact is a core requirement for successful manipulation, we do not check for collisions between the robot end-effector and the object. A final filter is applied to see if $\dot{\boldsymbol{\theta}}_c = J^{-1}(\boldsymbol{\theta}_c)\vec{\mathbf{v}}_{EE}$, i.e., the joint velocities at $\boldsymbol{\theta}_c$ given the given operational space velocity $\vec{\mathbf{v}}_{EE}$, satisfies the joint velocity thresholds for the robot, $\langle \dot{\boldsymbol{\theta}}_{min}, \dot{\boldsymbol{\theta}}_{max} \rangle$ (Figure 3.2b).

The kinematic feasibility check along the entire path from $\boldsymbol{\theta}_s$ to $\boldsymbol{\theta}_c$ is provided by the simulation engine, i.e., the success or failure of executing the sampled action in simulation will determine whether our planner adds this action to the planning graph. Given this process for generating feasible actions that move the object from its current position towards the target position, we build a global path planner to generate an action path through object configuration space from the object's current pose to task goal region.

3.3.3.3 Path Planning

In this section, we introduce a novel motion planner named *PokeRRT*; it generates a graph of feasible robot actions in the object configuration space. With the aforementioned $\mathbf{a}_{valid}$ as a set of valid control parameters, exploration properties of the rapidly-exploring random tree (RRT) algorithm [72] are leveraged to generate a planning graph. The integration of poking-specific control inputs to RRT ensures that all nodes in the planning graph are achievable configurations, while the RRT algorithm ensures exploration bias to the largest Voronoi regions of the configuration space. This bias is especially central to poking manipulation due to its inherent ability to cover large distances in the operational space.

A new node q_{rand} is set as the goal node with probability p_{bias} , otherwise q_{rand} is randomly sampled in the object configuration space. Actions from $\mathbf{a}_{valid}$ are applied from the nearest graph node q_{near} towards q_{rand} (Figure 3.2c) and the resultant node q_{new} that minimizes the distance to q_{rand} is added to the graph (Figure 3.2d; GET_BEST_RESULT() in Algorithm 1). Resultant poses are added to the planning tree until the task goal region $\mathcal{Q}_{goal}$ is reached, at which point the path from q_{start} to $\mathcal{Q}_{goal}$ in the planning tree is extracted by backtracking (Algorithm 1, Line 13). *PokeRRT* does not find the absolute least-cost path since RRT is not asymptotically optimal by nature [54]. The current implementation operates in a closed-loop paradigm—we replan on the fly if the resultant pose after a poking action is outside a specified threshold to compensate for inaccuracies of the simulation poking model and collisions between the object and environmental obstacles. The replanning threshold for an object is decided based on a collection of pokes executed in simulation and the real-world, i.e. the average object resultant pose error caused by the *sim2real* gap serves as an empirical estimate of the threshold. Therefore, this measure encompasses any uncertainty resulting from sensor, actuation, and skill model noise. The replanning strategy is particularly important for poking since frictional interactions between the object and the support surface are stochastic in nature and motor slippage and nonlinearities in joint modeling in the real-world can induce noise in robot–object interaction. Additionally, unlike pushing and grasping,

poking lacks full controllability which induces further uncertainty on the final object pose. Using this planner, poking can be used both as a skill and as a failure recovery tactic and can be shown to operate faster and in a more diverse set of scenarios than pushing or grasping. Since poking is inherently capable of covering larger distances in short periods of time due to high impact interactions, we can leverage this insight and add extensions to *PokeRRT* to plan sparse paths in object configuration space.

Algorithm 1: *PokeRRT**

Input: Start Node q_{start} , Goal Region $\mathcal{Q}_{goal}$, Neighborhood Radius r , Object Configuration Space $\mathcal{C}$, Poke Dataset $\mathcal{D}$
Output: Poke Path $\mathcal{P}$

```

1  $\mathcal{G}.\text{add\_vertex}(q_{start});$ 
2 while not GOAL_REACHED( $\mathcal{G}, \mathcal{Q}_{goal}$ ) do
3    $q_{rand} = \text{RAND}(\mathcal{C}); q_{near} = \text{NEAREST}(q_{rand}, \mathcal{G});$ 
4    $q'_{new} = q_{near} + \text{SAMPLE\_DISP}(\mathcal{D}, q_{rand});$ 
5    $q_{min} = \arg \min_{N\_POKES} \text{RADIAL\_NN}(q'_{new}, \mathcal{G}, r);$ 
6    $q_{new} = \text{GET\_BEST\_RESULT}(q_{min}, q'_{new}, \mathcal{G});$ 
7    $\mathcal{G}.\text{add\_vertex}(q_{new}); \mathcal{G}.\text{add\_edge}(q_{min}, q_{new});$ 
8   foreach  $q \in \text{RADIAL\_NN}(q_{new}, \mathcal{G}, r)$  do
9     if  $N\_POKES(q_{new}) + 1 < N\_POKES(q)$  and IS_LEAF_NODE( $q$ ) then
10        $\mathcal{G}.\text{remove\_edge}(\text{PARENT}(q), q);$ 
11        $q_{result} = \text{GET\_BEST\_RESULT}(q_{new}, q, \mathcal{G});$ 
12        $\mathcal{G}.\text{add\_edge}(q_{new}, q_{result});$ 
13  $\mathcal{P} = \text{BACKTRACK}(\mathcal{G}, q_{start}, \mathcal{Q}_{goal});$ 
14 return  $\mathcal{P}$ 
```

3.3.3.4 Online Path Smoothing

To minimize the number of pokes and enable poke planning to cover large distances, we combine empirically-derived heuristics with insights from RRT* [54] to develop *PokeRRT** (Algorithm 1). RRT* allows for the discovery of new lower-cost paths until the goal region is reached by introducing two additional steps to RRT—choose-parent and rewiring. In the context of *PokeRRT**, choosing the best parent node q_{min} replaces the edge from q_{near} to q_{new} with the edge from q_{min} to q_{new} . Planning time for *PokeRRT** can be reduced by using a data-driven heuristic to generate q'_{new} —instead of applying actions from $\mathbf{a}_{valid}$ to get q'_{new} , we sample Δq from a range of displacements from a dataset of random pokes $\mathcal{D}$. q'_{new} is then computed as $q_{near} + \Delta q$ in the

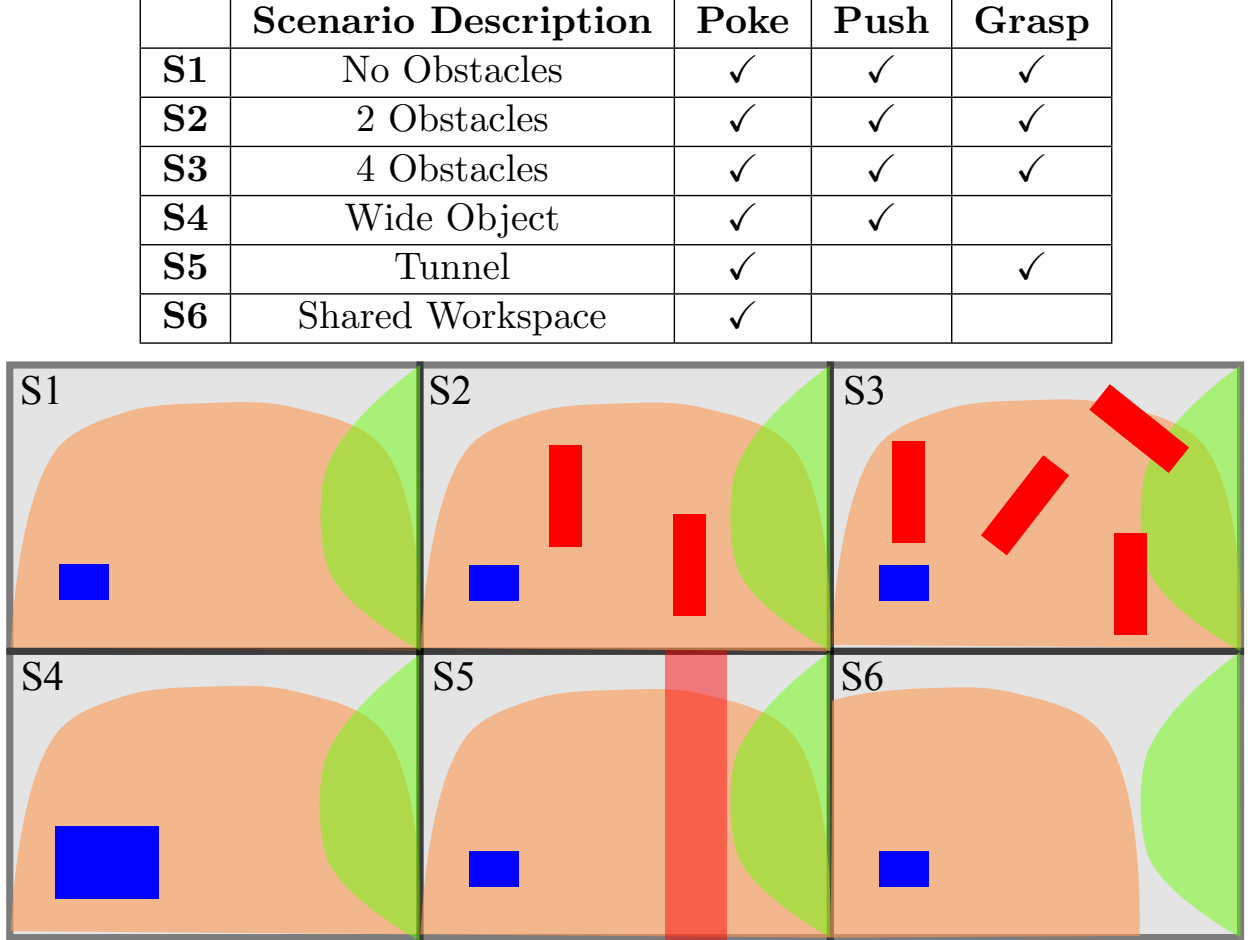


Figure 3.3: The robot successfully pokes the object (blue) from its reachable workspace (orange) to the goal region (green) in all scenarios while avoiding obstacles (red). *PokeRRT*, *PokeRRT**, and baseline algorithms are evaluated in 6 scenarios—no obstacles (S1), 2 obstacles (S2), 4 obstacles (S3), wide object (S4), tunnel (S5), and non-overlapping shared workspace (S6). The robot is unable to i) push or pick-and-place in S6 due to limited robot reach, ii) push in S5 due to workspace obstruction in the action path, and iii) pick-and-place in S4 due to object being wider than gripper width.

direction of q_{rand} (Algorithm 1, Line 4).

Then q_{min} is chosen as the neighbor within radius r of q'_{new} that minimizes the number of pokes from q_{start} to q'_{new} (Algorithm 1, Line 5). Radius r is selected empirically as the average displacement of the object in the dataset $\mathcal{P}$. The best action from q_{min} to q'_{new} is recomputed by sampling actions from $\mathbf{a}_{valid}$ to make sure each edge in the graph is a valid action. Resultant pose q_{new} is chosen as one that minimizes the distance to q'_{new} . q_{new} and the edge from q_{min} to q_{new} are added to the graph. In the rewiring step, the neighborhood of q_{new} is rewired to minimize the number of pokes along the path from q_{start} through q_{new} to a neighboring leaf node (Algorithm 1, Lines 8–12). Note that *PokeRRT** does not use the extension to RRT* that exploits the anytime nature of RRT* to continuously improve path quality by allocating planning time [55]—choose-parent and rewiring are the primary modes for online path sparsification. As a result, *PokeRRT** leads to fewer pokes in the planned path than *PokeRRT*, thereby taking advantage of poking’s core competency of allowing larger displacements.

3.4 Evaluation

In this section, we qualitatively and quantitatively evaluate *PokeRRT* and *PokeRRT** in simulation and the real-world under various environment setups. We measure success rates, task times (in seconds), and number of executed actions in the final planned path for both motion planners. Success rates are averaged across all scenarios for a given planner, whereas task times and number of executed actions are presented separately for each scenario. Task time is defined as the sum of planning, execution, and replanning times. Object start pose is kept fixed across various trials to aid reproducibility of results. Simulation and real-world results are averaged over 250 and 10 trials, respectively. Collectively, our results demonstrate that *PokeRRT* and *PokeRRT** indeed enable fast and successful manipulation of objects in conditions where push planning and pick-and-place fail.

Planner	Task Time [seconds]						Success Rate
	S1	S2	S3	S4	S5	S6	S1 - S6
PokeRRT*	46.43 (21.67)	212.74 (100.01)	232.42 (118.63)	70.47 (38.82)	132.62 (98.27)	130.01 (84.05)	0.87 (0.31)
PokeRRT	49.69 (21.11)	196.54 (124.61)	167.55 (138.63)	64.14 (52.45)	116.35 (89.72)	171.77 (103.21)	0.88 (0.31)
LI-PokeRRT*	169.87 (71.28)	378.90 (122.21)	346.70 (111.71)	165.91 (44.87)	N/A	N/A	0.53 (0.28)
LI-PokeRRT	123.71 (36.72)	328.66 (107.18)	322.28 (138.05)	135.80 (32.91)	N/A	N/A	0.63 (0.18)
Push Planner	122.68 (47.07)	284.78 (104.40)	249.32 (68.30)	117.58 (67.05)	N/A	N/A	0.44 (0.26)
Pick-and-Place	16.29 (3.76)	17.71 (3.81)	16.14 (3.25)	N/A	19.15 (4.99)	N/A	0.67 (0.00)

Table 3.1: Task times [*mean (stddev)*] for various planning algorithms in simulation are presented. Success rates are aggregated across all 6 scenarios. Overall, poke planning is faster than push planning and leads to higher success rate than pushing or grasping. Low-impulse (LI) poke planners and the push planner are unsuccessful in S5 and S6 due to action path collisions (S5) or goal region being outside the robot’s reachable workspace (S6). The robot is unable to pick or place object in S4 and S6 due to limitations in robot mechanics.

3.4.1 Experimental Setup

3.4.1.1 Scenarios

We test multiple algorithms for poking, pushing, and grasping manipulation in six scenarios (Figure 3.3). Our motivating application is the concurrent operation of two robots with potentially non-overlapping reachable regions located in adjacent workcells on a factory floor. Therefore, the objects being manipulated must be accessible by both robots to enable successful collaboration. Given this motivation, scenarios are designed to test the flexibility of the planner in generating plans for manipulating the object in a planar workspace from a fixed start pose to a goal region, which represents the workspace of a second robot. To ensure that object–obstacle collisions are handled through the replanning strategy, we enforce two properties in our scenarios: i) the objects used in each scenario are rigid and therefore retain their shape across multiple trials and ii) obstacles are fixed to the table so that collisions do not change the planning configuration space.

Scenarios 1–3 (S1–S3 in Figure 3.3) are designed to test baseline manipulation capability through uncluttered (S1) and cluttered (S2, S3) environments. S2 and S3 contain 2 and 4 obstacles, respectively, at fixed poses in the shared robot workspace. The object being manipulated has size $[9, 14, 5]$ cm and mass $m = 87$ g. Poke planning, push planning, and pick-and-place will all work in these scenarios since the goal region overlaps with the first robot’s reachable workspace. Scenario 4 (S4 in Figure 3.3) contains a bigger cuboid object of size $[11, 18, 11]$ cm and mass $m = 112$ g in

an obstacle-free workspace. Poke planning and push planning will work in this scenario, however pick-and-place will not since gripper width (10 cm) is smaller than the minimum dimension of the cuboid. This scenario represents situations where object properties are incompatible with robot kinematics, i.e., if the object of interest is too heavy or too wide to be grasped or if the end-effector is malfunctional, and the robot needs to formulate an alternate manipulation plan.

Scenario 5 (S5 in Figure 3.3) presents two work cells with a tunnel in the center of the table. The first robot cannot push the object to the second robot because the end-effector will collide with the divider along its action path. However, poke planning will succeed due to the short duration of robot-object contact. The first robot is able to grasp the object over the divider in our setup but for larger dividers such as a screen, pick-and-place will fail. Scenario 6 (S6 in Figure 3.3) contains an obstacle-free workspace with non-overlapping reachable regions for each robot. The first robot is able to poke the target object to the second robot’s reachable workspace, but cannot push or pick-and-place due to limited reachability. Since pushing operates under the quasistatic assumption, it requires constant contact between the robot end-effector and the object being manipulated and therefore, manipulation is limited by robot kinematics and reachability. This scenario represents cases where task success is limited by the robot’s kinematic characteristics, i.e., if the goal pose is outside the robot’s reachable workspace.

3.4.1.2 Parameters

The action vector $\mathbf{a}_{valid} = \langle \mathbf{p}_c, \|\vec{\mathbf{v}}_{EE}\| \rangle$ for our proposed motion planners is generated at runtime. Contour points p_c close to object corners are ignored for clean pokes. The velocity magnitudes $\|\vec{\mathbf{v}}_{EE}\|$ are sampled in the $[0.3, 1.0]$ m/s range to increase likelihood of robot joints achieving the commanded operational space velocities. Goal region is defined as the workspace of the second robot, determined as the frequency of inverse kinematics poses achievable on discretized xy-locations on the planar support surface of the object. The empirically-determined replanning threshold is 5cm and 10° .

3.4.1.3 Algorithms

PokeRRT and *PokeRRT** are compared against several baseline approaches. To evaluate pushing, we use the *Two-Level Push Planner* presented in [161] since it also uses simulation in the planning loop to get the next feasible environment state. This work integrates simulation-based forward modeling with sampling-based motion planning to explore the space of feasible pushing actions required to get an object from start to goal. Our baseline approaches, *Low-Impulse (LI) PokeRRT* and *Low-Impulse (LI) PokeRRT**, are designed to show that poking is a more fundamental manipulation skill and encompasses pushing if the applied impulse magnitudes are kept small. They operate similarly to *PokeRRT* and *PokeRRT** but with $\|\vec{v}_{EE}\| = 0.2$ m/s to simulate pushing. Any replanning for a given planner is done using that same planner. Lastly, *Pick-and-Place* is performed in an open-loop manner with predefined grasps for known objects. An experimental trial fails if the planner does not find a valid plan to the goal region in 240 seconds or if the object falls off the table during execution.

3.4.2 Simulation Experiments

Table 3.1 shows the task times and success rates for *Pick-and-Place* and five non-prehensile manipulation planners—*PokeRRT**, *Low-Impulse PokeRRT**, *PokeRRT*, *Low-Impulse PokeRRT*, and *Two-Level Push Planner*. Results are averaged over 250 trials. *PokeRRT* and *PokeRRT** are successful in all scenarios while *Two-Level Push Planner* fails in S5 and S6 and grasping fails in S4 and S6. *Two-Level Push Planner* has a low overall success rate (44%), as expected based on the reasons presented in Section 3.4.1.1.

PokeRRT and *PokeRRT** task times are lower than *Two-Level Push Planner* task times across all scenarios. Standard deviations are high due to the sampling-based nature of our planners. The task time for S4 is greater than for S1 since the object used in S4 is not only bigger in size but also larger in mass than the object used in S1, therefore poke displacements are lower given the same contact force. *Pick-and-Place* has the lowest task time because it does not involve planning in

Planner	Number of Actions in Execution Path					
	S1	S2	S3	S4	S5	S6
PokeRRT*	3.93 (0.98)	4.62 (1.04)	4.49 (1.02)	4.76 (1.20)	3.71 (1.02)	3.65 (1.08)
PokeRRT	6.10 (1.96)	8.21 (2.95)	7.94 (2.08)	6.77 (2.35)	4.56 (1.27)	8.17 (2.46)
LI-PokeRRT*	16.43 (2.28)	22.27 (2.26)	19.00 (1.54)	18.54 (1.44)	N/A	N/A
LI-PokeRRT	19.08 (1.51)	19.62 (1.64)	20.74 (2.13)	19.38 (1.59)	N/A	N/A
Push Planner	8.72 (1.42)	8.61 (1.71)	7.68 (1.19)	8.53 (1.51)	N/A	N/A
Pick-and-Place	1.00 (0.00)	1.00 (0.00)	1.00 (0.00)	N/A	1.00 (0.00)	N/A

Table 3.2: Number of executed actions in the planned path for various algorithms in simulation are presented here. Both *PokeRRT** and *Low-Impulse PokeRRT** lead to fewer actions than their RRT-based counterparts, thus exploiting the large displacement property inherent to poking manipulation.

the object configuration space—the robot moves to object pose, grasps, and moves to goal pose, so only a single action is executed. Low-impulse poke planners have large search trees due to shorter displacements of the object. This results in longer planning times and also lower success rates. Task times are higher for obstacle scenarios because the robot end-effector is more likely to run into obstacles and collision checking is a computationally expensive procedure.

Pick-and-Place succeeds in simulation for S1, S2, S3, and S5 because there is no uncertainty in object pose so manipulation consists of moving to a predefined grasp configuration and moving to the goal pose. We intentionally set up the start and goal configurations for *Pick-and-Place* to create an upper bound for comparison with our planners. For the reasons presented in Section 3.4.1.1, it fails in S4 and S6. Low-impulse poke planners have higher success rates than the push planner because the shorter robot control trajectories corresponding to poking result in fewer obstacle collisions. Low-impulse poke planners fail in S5 and S6 because the pokes are not strong enough to pass the workspace divider or cross into the second robot’s reachable workspace.

Table 3.2 presents the number of executed actions in the final planned path for multiple planning algorithms across several scenarios. It indicates that the number of executed actions is lower for *PokeRRT* and *PokeRRT** compared to *Two-Level Push Planner*, showing that poke planners can displace the object further with fewer actions in less time. Notably, the number of actions is lowest for *PokeRRT**, thereby supporting our claim that minimizing the number of poking actions exploits the large displacement property of poking manipulation. However, the task time is similar to that of *PokeRRT* since *PokeRRT** introduces the choose parent and rewiring steps to *PokeRRT*, which increases the number of actions sampled while planning. In general, *PokeRRT* is preferable in scenarios with obstacles—even though the number of pokes for *PokeRRT** is lower (i.e., faster overall execution), a greater percentage of time is spent on resampling actions in *PokeRRT**, most of which lead to object–obstacle collisions. Task times for low-impulse poke planners are comparable to *Two-Level Push Planner*, supporting our claim that pushing is a limiting case of poking where applied impulses are low in magnitude.

Planner		PokeRRT*	PokeRRT	Two-Level Push Planner	Pick-and-Place
Task Time [seconds]	S4	69.58 (26.15)	83.55 (29.59)	120.21 (47.09)	N/A
	S5	127.45 (57.80)	148.55 (86.23)	N/A	20.31 (3.21)
	S6	182.03 (122.77)	157.03 (94.41)	N/A	N/A
Number of Actions	S4	5.27 (2.00)	6.20 (1.60)	6.00 (1.26)	N/A
	S5	5.12 (1.05)	6.67 (1.56)	N/A	1.00 (0.00)
	S6	6.11 (1.91)	6.88 (2.15)	N/A	N/A
Number of Replans	S4	3.45 (1.67)	4.2 (1.6)	2.80 (1.54)	N/A
	S5	3.38 (1.32)	4.11 (1.37)	N/A	N/A
	S6	4.22 (1.81)	4.38 (1.49)	N/A	N/A
Success Rate	S4 - S6	0.9 (0.3)	0.9 (0.3)	0.33 (0.47)	0.27 (0.44)

Table 3.3: Real-world task times, number of actions in executed path, and success rates are presented in this table. Results are averaged across 10 trials. Pushing has the highest success rate and fastest task times. PokeRRT* generates plans with the fewest number of actions.

3.4.3 Real-World Experiments

We evaluate *PokeRRT**, *PokeRRT*, *Two-Level Push Planner*, and *Pick-and-Place* for a subset of the scenarios in the real-world (Table 3.3). The task times and the number of actions executed in the real world are slightly higher than in simulation. This is expected because the plan is generated in simulation and simulation does not fully capture the complexities of the real-world environment. Therefore, replanning is required for real-world plan execution which leads to an increase in task times. As shown in Figure 3.5, execution times for poke planners are lower than for the push planner indicating that poke planning leads to faster object manipulation than push planning. Additionally, the replanning time for push planning is higher than for poke planning since simulating push actions to generate the planning graph takes more computational cycles than simulating instantaneous contact for poke actions. However, the number of replans for push planning is lower than for poke planning thereby pointing to the inherently uncontrollable nature of poking, which may not be desirable in certain situations (Table 3.3). Collectively, real-world results align with the results from simulation.

Figure 3.4 depicts a failure recovery case for S6 where failure to grasp or push to the second robot’s reachable workspace does not result in task failure—the first robot pokes the object to the second robot’s reachable workspace, allowing the second robot to successfully manipulate the object. Additionally, while grasping is a faster form of manipulation with fewer actions than

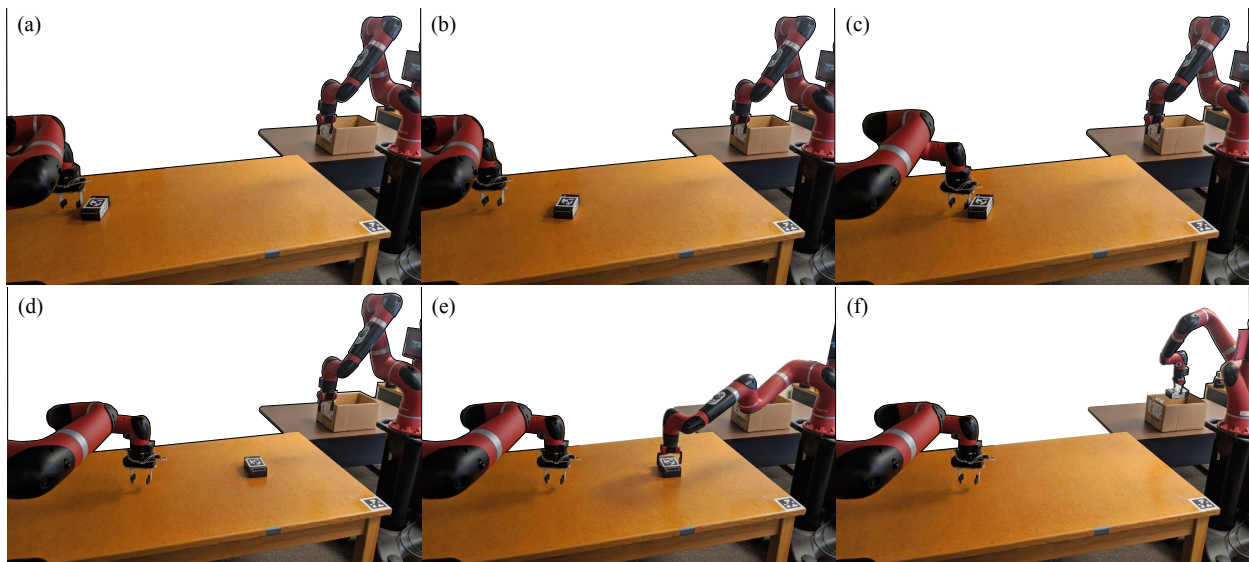


Figure 3.4: Two robots with non-overlapping reachable regions are shown (S6). Robot A (left) applies 2 pokes to manipulate the object to Robot B's (right) workspace (a-d). Robot B then grasps the object and places it in a bin that is not reachable by Robot A (e-f).

poking or pushing, it has a lower overall success rate (27%) than pushing (33%) due to perception inaccuracies. This discrepancy between simulation and real-world results points to a fundamental difference between grasping and poking or pushing—sensing uncertainty leads to total failure in grasping, whereas for poking it leads to just partial failure as our presented planners operate in a closed-loop manner.

3.5 Conclusion and Discussion

In this work, we demonstrate poking manipulation as a fundamental motion primitive that complements grasping and encompasses pushing in terms of capability. Our work is the first to show qualitative and quantitative results for multiple test conditions to demonstrate the flexibility and robustness of poking as a skill through *PokeRRT*. We present the task times, number of executed actions, and success rates of our proposed motion planners and four baseline algorithms across six different scenarios. The results demonstrate the strengths of the poking motion primitive: poking is not as limited by robot reachability, robot end-effector design, object properties, and inaccuracies due to perception as grasping or pushing. Success rates are higher for poking-based planners than for the push planner indicating that poking expands the size of reachable workspace by its ability to execute longer object displacements using shorter robot end-effector trajectories. Task times for computed plans are significantly lower for poking than for pushing, indicating that poking allows fast object manipulation because it does not face the same constant-contact restriction as pushing. *PokeRRT* and *PokeRRT** also demonstrated the qualitative behaviors visualized in Figure 3.3.

Errors in dynamic modeling caused by the gap between simulation and the real-world are covered by the replanning strategy in our work. However, future work in identifying sources of uncertainty will allow for the quantification of object–obstacle collision margins and incorporating them directly into the planning process will lower the chances of collision in the real-world. In order to make plan execution in the real-world more robust, our future work will also formalize analytical and learned models for poking and quantify object constraints which will lead to additional insights about the capabilities and limitations of poking. For instance, poking may not be an effective mode

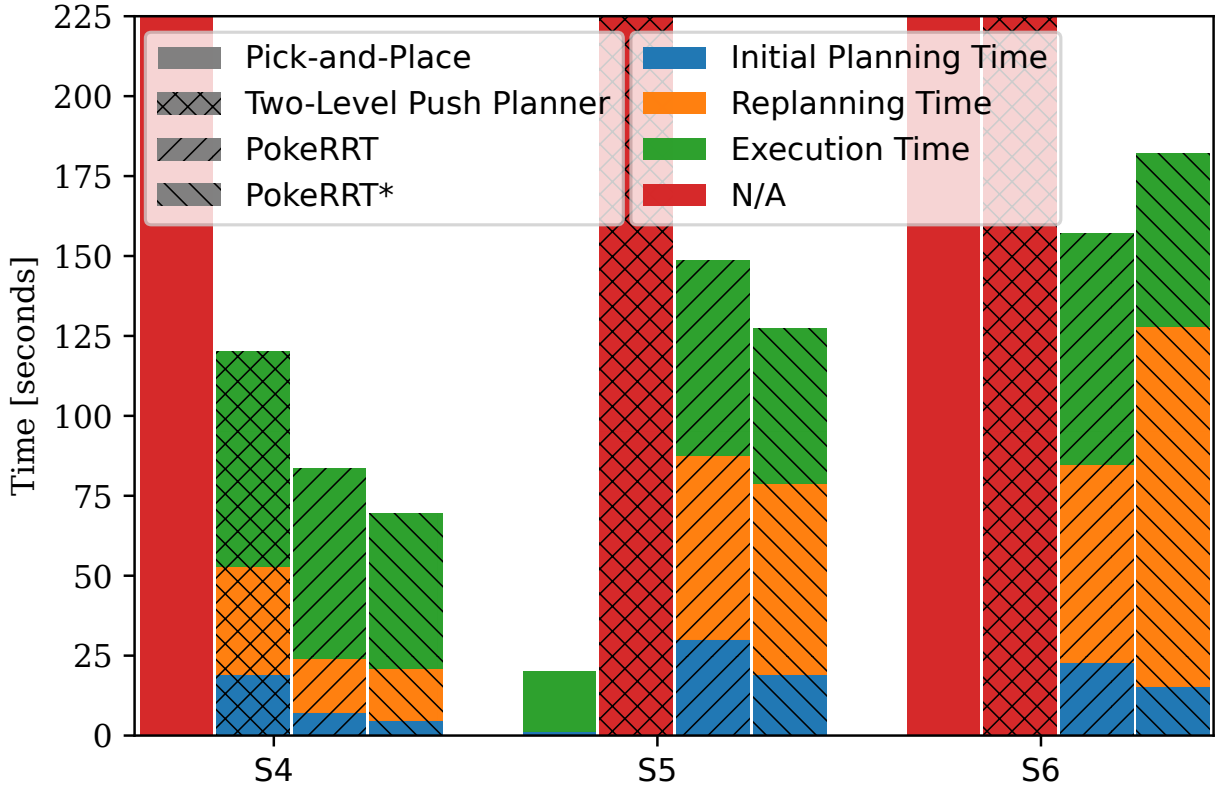


Figure 3.5: A breakdown of task time as the sum of initial planning, replanning, and execution times is presented for various planners in the real-world. A single bar indicates total task time. Long red bars indicate cases where tasks cannot be solved. Overall, poke planning demonstrates lower execution and replanning times than push planning.

of manipulation for high friction interactions and should also be used sparingly since it may cause significant wear-and-tear on a real robot through repeated execution of high-speed trajectories. Consequently, by discovering the feasibility conditions of multiple skills (e.g. poking, pushing, and grasping) through interaction and planning around the key strengths of each motion primitive, robots will be able to more efficiently manipulate objects.

3.6 Case Study: Liquid Container Manipulation with Obstacle Avoidance

This case study applies the online-evaluation paradigm of this chapter to a setting where the grasped object imposes dynamic limits on feasible trajectories; this section summarizes the problem structure, method, and key results.

3.6.1 Task-Biased Geometric Planning

Among the different categories of objects robots encounter, liquids present a unique challenge due to their complex behavior. Despite the importance of robotic fluid manipulation in applications ranging from handling hazardous materials in laboratories to facilitating assistive feeding technologies designed for differently-abled individuals, research in robotic fluid manipulation remains under-explored and challenging. Fluid manipulation is challenging due to the intricate and nonlinear nature of fluid dynamics. Accurately modeling fluid dynamics is heavily task-dependent and requires substantial computational resources, hindering real-time control [71, 32]. Existing approaches tackle these challenges by simplifying the complex fluid dynamics into more computationally efficient linearized pendulum or mass-spring-damper models. However, these methods struggle in cluttered scenes: they restrict the robot’s motions to pendulum-like or mass-spring-damper-like behavior [35, 92, 94, 26, 27, 120], impose restrictions on the container’s orientation by setting fixed or limited Euler angle ranges throughout the trajectory [94, 27], and rely on constant monitoring of the liquid surface to execute trajectories, which can be obstructed by opaque fumes produced by the liquid or by obstacles present in cluttered scenes [84, 120, 91]. Consequently, the action space is over-constrained, and the robot can only perform limited motions, inhibiting its

ability to maneuver around obstacles in cluttered environments.

To address these limitations, we propose **Spill-Free RRT*** (*SFRRT**), for spill-free manipulation of open-top liquid-filled containers in cluttered environments. SFRRT* first uses the container shape to inform the generation of a spill-free path via RRT*: by considering the container geometry, the algorithm calculates the maximum tilt angle $\theta_{\max}$ beyond which the container will spill and constrains the path search accordingly. An informed sampler samples robot configurations where the container orientation is less than the spill threshold $\theta_{\max}$, and is incorporated into an RRT* path planner to generate spill-free paths. RRT* is chosen due to its asymptotic optimality properties; by preferring transitions with minimal cost, it generates smooth motion profiles that maintain orientation stability [54].

3.6.2 Analytical Static Models for Fill Level

A challenge in spill-free trajectory generation is that the volume of the robot configuration space containing spill-free trajectories is small and therefore inefficient to explore with random sampling alone [62]. This dictates the need for an informed approach to explore dynamically-valid robot states. However, developing a model for combined robot and object dynamics is computationally expensive, task-specific, and impractical for real-world control. To address this, we mathematically formalize how container properties inform the construction of the *spill-free space*.

We model containers as frustums of cones, which accurately represents most common container shapes and provides an upper bound on the maximum tilt angle of containers with other shapes, such as round bottom flasks or wine glasses. Under quasi-static conditions, we consider tilting a container to the angle $\theta_{\max}$ such that any $\theta_t \leq \theta_{\max}$ is spill-free. Given the container’s bottom radius r_b , top radius r_u , container height h_c , and water height h_w , two spillage cases arise: one where the tilted liquid forms a trapezoid and a triangle, and another where it forms only a triangle. We leverage the key insight that the total amount of liquid remains constant before and after tilting, yielding closed-form expressions for $\theta_{\max}$:

$$\theta_{\max} = \tan^{-1} \left[\frac{h_c(r_u - r_{w'})}{(r_u - r_b)(r_u + r_{w'})} \right] \quad (\text{Case I}) \quad (3.1)$$

where $r_{w'} = \frac{h_c r_u r_b - h_c r_2 r_b + h_w r_2 r_w + h_w r_2 r_b}{h_c r_2 + h_c r_b}$, and

$$\theta_{\max} = \tan^{-1} \left[\frac{h_c}{r_u - r_b + \frac{2h_w(r_w + r_b)}{h_c}} \right] \quad (\text{Case II}) \quad (3.2)$$

where $r_w = r_u - \left[\frac{r_u - r_b}{h_c} (h_c - h_w) \right]$.

3.6.3 Learned Dynamics Model for Trajectory Verification

While RRT* generates spill-free paths, to ensure the generation of a spill-free *trajectory*, we must carefully parameterize the path such that the velocity, acceleration, and jerk do not cause spillage. To capture this relationship, we employ a transformer-based machine learning model trained to classify whether a container’s trajectory is spill-free—the *Spill-Free Classifier* (SFC).

The input to SFC is the container trajectory (position and orientation over time) and container properties (top and bottom radius, container height, and liquid fill level). The SFC model employs the transformer architecture because its input is sequence data and transformers have become the leading architecture for handling sequential data due to their efficient parallel training capabilities and long context window [145]. To train the SFC model, we utilized a motion capture system to collect container trajectories performed by individuals across different container types and liquid volumes. We collected a total of 700 trajectories, including 655 human-performed trajectories and 45 trajectories from the Panda robot; the SFC achieves 94% validation accuracy.

3.6.4 Where Analytical and Learned Components Meet

Having modeled the relationship between container trajectory and spillage risk, we leverage SFC to generate spill-free trajectories via the *Spill-Free Time Parameterization* (SFTP) algorithm. The key idea is to find the maximum jerk that does not cause spillage: SFTP iteratively reduces the maximum jerk of the trajectory and queries SFC after each iteration until SFC predicts the

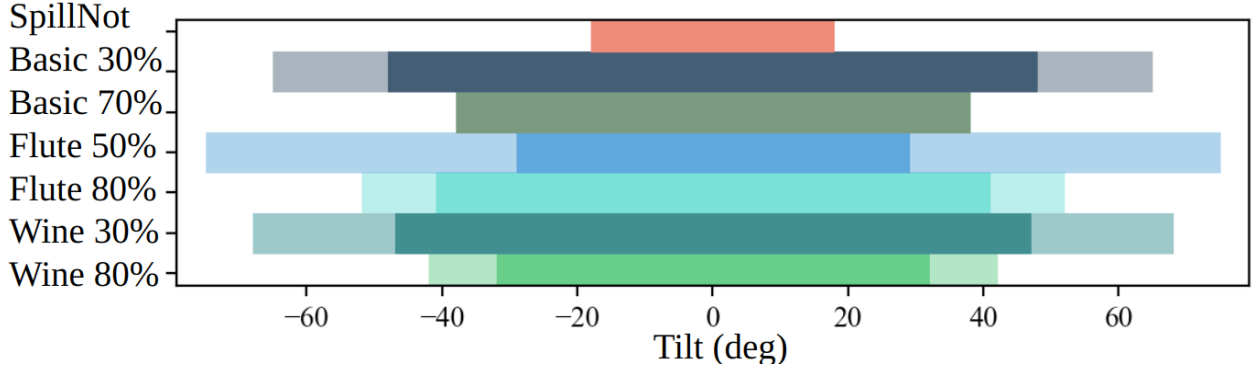


Figure 3.6: Maximum tilt angles executed across all 30 evaluation trials by SFRRT* (solid lines, within the shaded allowable range per container) compared to the SpillNot baseline [94] (fixed 15° limit). In every experiment, SFRRT* utilizes a larger tilt range than SpillNot.

resulting trajectory is spill-free. Jerk constraints are especially critical in preventing sudden changes in acceleration that could cause spillage.

We evaluate SFRRT* on the 7-DoF Franka Emika Panda across two scenarios (transporting a drink while avoiding table obstacles; transporting a chemistry vessel into a fume hood). Overall, we achieved a 100% success rate for all experiments except experiment 5, where spill-free trajectories were accomplished in four out of five trials. As shown in Figure 3.6, in every experiment SFRRT* utilizes a larger tilt range than SpillNot [94]; SpillNot’s tilt angles are always limited to a maximum of 15° to maintain low linearization error. SFRRT* achieves a maximum tilt angle of 45° when transporting the basic container with a 30% fill level, surpassing SpillNot’s 15° —a $3\times$ increase in orientation range. For the flute glass with a 70% fill level the maximum tilt angle is 72° , a $4.8\times$ increase. Furthermore, SFRRT* can generate trajectories where all three axes of orientation are utilized, with ranges for roll, pitch, and yaw of $[-49, 41]$, $[-30, 30]$, and $[-16, 9]$ degrees respectively. In contrast, the SpillNot method imposes a fixed yaw angle throughout the trajectory [94].

However, the formulations in this chapter remain single-query: sampled controls are evaluated online, accepted nodes are tied to the current object and workspace, and the planners do not retain a reusable representation of the feasibility computations they perform. Chapter 4 then asks whether the repeated cost of dynamic feasibility can instead be *amortized*.

Chapter 4

Reusable Constraint Structure for Dynamic Feasibility

The planners of Chapter 3 evaluate dynamic feasibility online, rebuilding their search structure from scratch for every new start and goal and paying the full cost of simulation and collision checking on each query. This chapter introduces the first *amortized* representation in this thesis: the dynamic feasibility of a robot’s motions is precomputed once into a reusable bundle of edges and shared across many related planning queries. By deferring—and largely reusing—propagation and collision checking through lazy evaluation, this shifts constraint knowledge upstream of any single query, reducing per-query planning time while preserving the kinodynamic feasibility guarantees that an online planner provides. A case study at the end of the chapter carries this precomputation principle into the task space, constructing reusable motion structure for whole-body manipulation under locked multi-joint failures.

4.1 Introduction: Multi-Query Planning with Edge Bundles

Robotic manipulation in complex operational environments necessitates the integration of constraints at various levels of abstraction (i.e., kinematics, statics, quasi-statics, and dynamics) in order to effectively govern the interaction between the robot and its surrounding environment [122, 89]. Importantly, dynamics extends the foundational principles of kinematics, statics, and quasi-statics by incorporating the analysis of forces and torques in robot and object motion in real-world tasks such as liquid transport [43], deformable object manipulation [124], and nonprehensile maneuvers [108, 50] (Figure 4.1).

The complexities (and opportunities) introduced by dynamic-based analysis of motion planning set the stage for kinodynamic planning, a class of methods that incorporate these dynamic constraints into planning, consequently extending classical kinematic or geometric planning methods beyond the robot’s state space to its control space [126, 74]. Within this scope, kinodynamic planning frameworks can be broadly classified into two paradigms: *sequential* and *interleaved*. Sequential approaches engage first in geometric path planning and then in time parameterization, a sequence that may result in dynamically-infeasible trajectories and long planning times [113, 67]. In contrast, interleaved methods present a more integrated solution by enabling simultaneous exploration of state and control spaces, thereby ensuring dynamically-valid trajectories, if they exist [80, 79]. It is worth noting that a majority of methods in the interleaved domain are *single-query* methods. In a single-query context, each new planning problem necessitates the reconstruction of the planning tree, thereby limiting computational efficiency. This issue becomes of particular importance in kinodynamic planning due to the higher dimensional space involved. To this end, our work introduces a multi-query approach in the interleaved planning domain, aimed at reducing computation times across multiple planning queries. This contribution obviates the need to re-explore the state and control spaces and their corresponding search trees for each new task, offering a more efficient solution.

More specifically, our planner is conceptually aligned with the bundle of edges (BoE) framework, which adapts the probabilistic roadmap method from a kinematic setting to the kinodynamic domain [56]. The BoE planner employs a forward search through disconnected, randomized rollouts of a dynamics model [130]. This process is computationally demanding due to a large branching factor and is contingent upon both accurate simulation models and reliable collision checkers—assumptions that may not hold in complex, real-world conditions. Crucially, it is these very processes—forward propagation and collision checking—that contribute significantly to the computational burden of the planner. This underscores the need for ‘*lazy*’ strategies that postpone these operations until absolutely required, thus achieving a trade-off between efficiency and accuracy. Toward this goal, in this work we introduce the concepts of lazy forward propagation

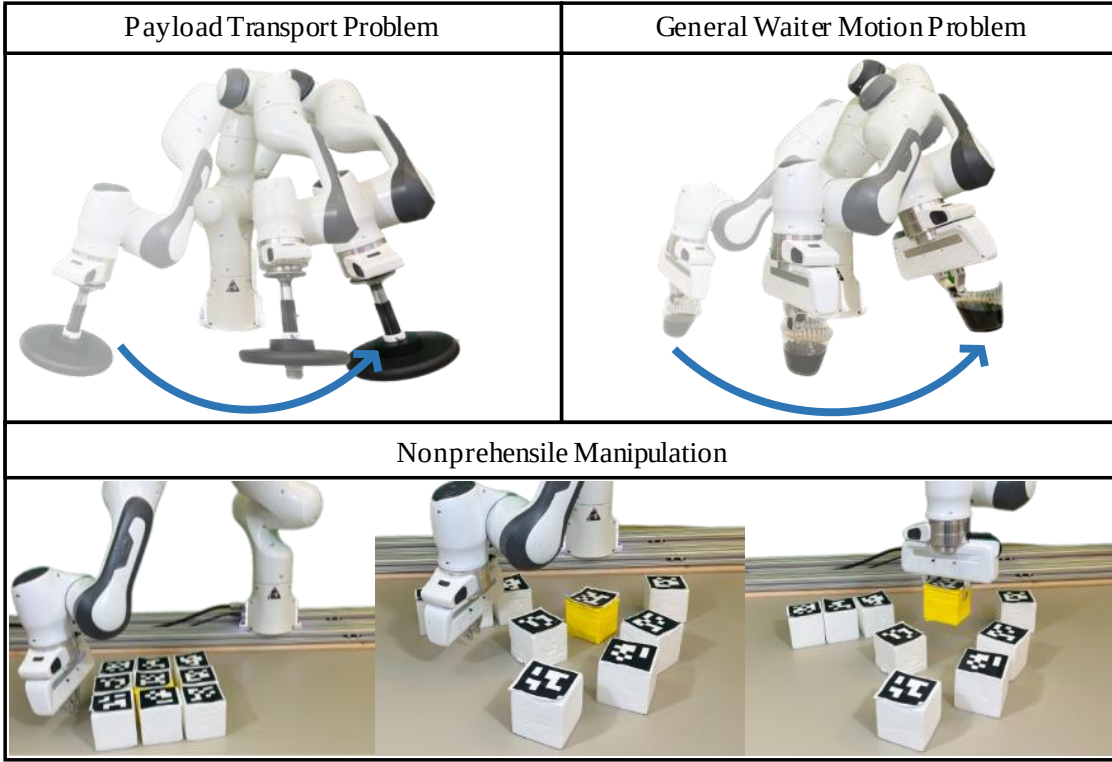


Figure 4.1: Several applications motivate the need for considering dynamic constraints in motion planning. The payload transport problem requires the robot to account for the added mass at its end-effector and its effects on the inertia, Coriolis, and gravity matrices in the robot dynamics model [125]. Similarly, liquid transport in the General Waiter Motion Problem imposes acceleration constraints on the end-effector to ensure spill-free trajectories [43]. Nonprehensile actions like poking can be used to singulate target objects and require an understanding of combined robot and object dynamics [108]. In this paper, we present a method for planning robot paths while considering its dynamic constraints.

and lazy collision checking in the context of kinodynamic motion planning for BoE, which we refer to as *LazyBoE*. By applying rollouts from the edge bundle to the planning tree directly without resimulating them, these techniques not only significantly reduce planning time but also enable the application of this approach to higher-dimensional systems commonly encountered in real-world applications without compromising the asymptotic optimality guarantees previously established in [130].

In this work, we delineate three key contributions: i) a novel method for the efficient generation of state and control space rollouts from the forward dynamics model of a robot using varying control sequences instead of traditional piecewise-constant controls, aimed at reducing planning times across multiple queries; ii) the introduction of ‘lazy’ forward propagation and ‘lazy’ collision checking strategies that employ bounded end-state perturbations and prioritize forward simulations by their likelihood of collision, thereby mitigating the computational load associated with physics-based simulations; iii) the *LazyBoE* kinodynamic motion planner, a robust framework that capitalizes on the aforementioned strategies to rapidly and reliably compute dynamically valid trajectories for complex tasks. We validate our approach on a 7DoF robot arm, and demonstrate how it enables a broad class of real-world tasks.

4.2 Background and Related Work

Kinodynamic planning involves navigating the complex interplay between the robot’s *state space*—comprising position, velocity, and higher-order derivatives—and its *control space*, defined by the set of feasible actions for a given task. The existing literature mainly bifurcates this analysis into two approaches: *sequential* and *interleaved*. Sequential methods, such as TrajOpt [127], CHOMP [119], STOMP [52], KOMO [141], and ITOMP [105], construct a geometric path in the robot’s state space first and then assign valid controls to each waypoint. While these methods are advantageous for their minimal design complexity and capability to handle non-linear constraints, they have a number of limitations. They are highly susceptible to the quality of the initial seed trajectories, often confining trajectories to a narrow set of homotopic classes [127, 44]. Moreover, they generally

lack theoretical guarantees regarding the completeness or optimality of solutions [67]. Within this line of work, there exist hybrid methods that attempt to reconcile these limitations by incorporating sampling-based planning, which in turn brings about theoretical properties such as probabilistic completeness and asymptotic optimality [67] and permits the discovery of multiple solution classes [53]. Time parameterization techniques can also be applied in place of optimization to assign timesteps to each waypoint in the geometric path [113, 11, 83].

In contrast to sequential approaches, interleaved methods cast motion planning as a search problem, alternating between state space sampling and control propagation to find a valid trajectory. These methods often employ steering functions, or inverse models, to delineate a set of controls connecting two arbitrary points in the state space. For instance, DIMT-RRT utilizes a quadratic steering function to account for joint acceleration limits and non-zero initial and goal velocities [68]. AVP computes the range of reachable velocities at the end of a path, given reachable velocities at the start of it [114, 51]. LQR-RRT* uses a quadratic cost function and linearized dynamics in conjunction with a linear-quadratic regulator (LQR) to connect consecutive states in an RRT* tree [110, 20]. The optimization techniques discussed in sequential methods can also be used as steering functions in interleaved approaches. In all, these techniques enable precise goal state attainment, allow to perform backward searches, and facilitate exact tree connections in bidirectional searches, thus expediting the planning process [73]. However, they come with a set of challenges. Steering functions may not exist or could be difficult to design for a given dynamical system. Furthermore, exact state connections may not be reliable in real-world settings due to uncertainties and the use of optimization as steering may cause planning to get stuck in local minima [80].

The second variety of interleaved methods involves navigating the state and control spaces using forward control propagation. Using a random state and action, these methods determine the resultant state, constructing a tree of dynamically feasible edges [108]. SyCLoP and KPIECE leverage state space discretizations and guide exploration in low-coverage areas [116, 137]. SST optimizes path quality via pruning and heuristic biasing [79]. GBRRT and GABRRT combine multiple propagation methods and backward tree costs as heuristic values for bidirectional kinody-

namic search [99]. Other techniques involve pre-mapping the space of valid motions and conducting a search over this state and control space discretization [130, 16]. Existing approaches that exploit lazy strategies for collision checking involve learning-based methods [152] and evaluating paths on an as-needed basis [15, 86, 36]. We take an empirical approach by computing a collision probability for each kinodynamic action in our state and control space discretization.

Overall, interleaved approaches are easy to design since they leverage general-purpose physics simulations, explore both state and control spaces uniformly, and easily adapt to environmental interactions and multi-object systems. However, they may face slow convergence owing to costly simulation and collision checking methods. Next, we outline how our contribution mitigates these issues, paving the way for fast computation of asymptotically optimal solutions.

4.3 Methods

Here, we present *LazyBoE*, our multi-query kinodynamic motion planner. The planner relies on two key contributions: 1) a method for generating a bundle of edges in order to provide a discrete approximation to the robot’s state and control spaces (Section 4.3.2), and 2) a quantification of lazy propagation and collision probabilities to reduce planning time (Section 4.3.3). These pieces enable us to design and implement a planner that defers computation in the form of simulation and collision-checking until there is a viable path that minimizes a heuristic value (Section 4.3.4).

4.3.1 Problem Setup

We formalize the problem of kinodynamic motion planning using edge bundles similarly to [130]. The robot state space $\mathcal{Q} \subset \mathbb{R}^m$ is defined where $q \in \mathcal{Q}$ represents the joint angles of an m -dimensional robot. The control space $\mathcal{U} \subset \mathbb{R}^m$ contains elements $\dot{q} \in \mathcal{U}$ that represent joint velocities. An *edge bundle* is a set of disconnected edges generated by applying randomly sampled control parameters to random robot states for random time durations, thereby connecting random start states to resultant end states:

$$\mathcal{E} = \{(q_0, \dot{q}, \Delta t, q_f) \mid q_0, q_f \in \mathcal{Q}, \dot{q} \in \mathcal{U}\}$$

such that $f(q_0, \dot{\mathbf{q}}) = q_f$, where $f(\cdot)$ defines the forward dynamics model of the robot and $\dot{\mathbf{q}} = (\dot{q}_0, \dots, \dot{q}_{f-1})$ represents a sequence of joint velocities. The i -th edge is denoted as e_i and defined as the tuple $(q_0^i, \dot{\mathbf{q}}^i, \Delta t^i, q_f^i)$. The objective of *LazyBoE* is to find a continuous, collision-free trajectory

$$\pi(t) : [0, 1] \rightarrow \mathcal{Q}_{\text{free}} \subseteq \mathcal{Q}, \quad \pi(0) = q_0, \quad \pi(1) = q_f$$

through the state and control space discretization enabled by $\mathcal{E}$, while respecting the dynamics constraints $\mathcal{D}$ of the robot.

4.3.2 Edge Bundle Construction with the Robot Dynamics Model

The backbone of our planner lies in constructing a discrete approximation of the robot’s state and control spaces, as a result capturing the nuances of real-world dynamics including nonlinearities like joint friction. Defined as a bundle of edges $\mathcal{E}$, this approximation is a collection of valid, disconnected, and randomly generated forward rollouts of the robot dynamics model over a variable duration Δt . In order to enable uniform and rapid exploration of the control space, we propagate a sequence of randomly-sampled joint velocity actions, $\dot{\mathbf{q}} = (\dot{q}_0, \dots, \dot{q}_f) \in \mathcal{U}$, from a randomly sampled state, $q \in \mathcal{Q}$. In addition to providing a broader local reachability range (and as a result expanding the explored volume in the state space) from a given state, varying joint velocities over a single propagation leads to longer-duration edges as it prevents the robot from becoming locally “locked” along fixed controls. Moreover, it is necessary that these Monte Carlo propagations are collision-free and obey the Euler-Lagrange robot dynamics model $\mathcal{D}$ to ensure validity in real-world execution. Our approach to bundle generation can be extended to other robot embodiments as long as a valid dynamics model is provided.

As an example, we focus on one particular embodiment and use an analytically-derived and empirically-tuned dynamics model for the 7 degrees-of-freedom (DoF) Franka Emika Panda arm [31]. This analytical model is defined as $\tau(q, \dot{q}, \ddot{q}) = M(q)\ddot{q} + C(q, \dot{q})\dot{q} + g(q) + f(\dot{q})$, where q , $\dot{q}$, $\ddot{q}$, and τ are 7×1 vectors representing the robot’s joint angles, velocities, accelerations, and torques, respectively. M is a symmetric, positive-definite 7×7 joint inertia matrix, C is a 7×1 matrix that

captures the Coriolis and centrifugal effects caused by robot motion, g is a 7×1 vector encapsulating the torque required by each motor to counteract gravitational forces, and f is a 7×1 vector that quantifies the torques necessary to counteract friction at the joints. The model is used to verify whether the propagated joint velocity sequence $\dot{\mathbf{q}}$ obeys the robot torque limits.

Furthermore, validating an edge involves three classes of checks: collision checks, robot state limit checks, and continuity checks (Algorithm 2, Lines 20-22). Acceptable ranges for the state and control variables and additionally, dynamics quantities are defined as $q_{min} < q < q_{max}$, $\dot{q}_{min} < \dot{q} < \dot{q}_{max}$, $\ddot{q}_{min} < \ddot{q} < \ddot{q}_{max}$, and $\tau_{min} < \tau < \tau_{max}$. In addition, the dynamics simulations must obey continuity constraints due to the stringent control requirements of this robot. For a given static environment, edges are validated to be free of self-collision and environmental collisions via geometric collision-checking algorithms. This incurs a one-time cost since the edges can be used lazily for a significant portion of our motion planning process. We next explain how we can compute useful quantities from this generated edge bundle to enable lazy kinodynamic planning.

This method of edge generation with a sampled duration is known as Monte Carlo propagation and ensures the preservation of asymptotic optimality in kinodynamic planning algorithms [69]. By applying rollouts from the edge bundle to the planning tree directly without resimulating them, these techniques not only significantly reduce planning time but also enable the application of this approach to higher-dimensional systems without compromising the asymptotic optimality guarantees previously established in [130]. By applying rollouts from the edge bundle to the planning tree directly without resimulating them, these techniques not only significantly reduce planning time but also enable the application of this approach to higher-dimensional systems without compromising the asymptotic optimality guarantees previously established in [130].

4.3.3 Edge Perturbation for Lazy Propagation and Collision-Checking

The task of generating an edge bundle that uniformly samples the state and control spaces of a robot is a computationally intensive process, particularly in the case of a high-DoF robot and increased environment complexity. A substantial part of this computational workload is incurred

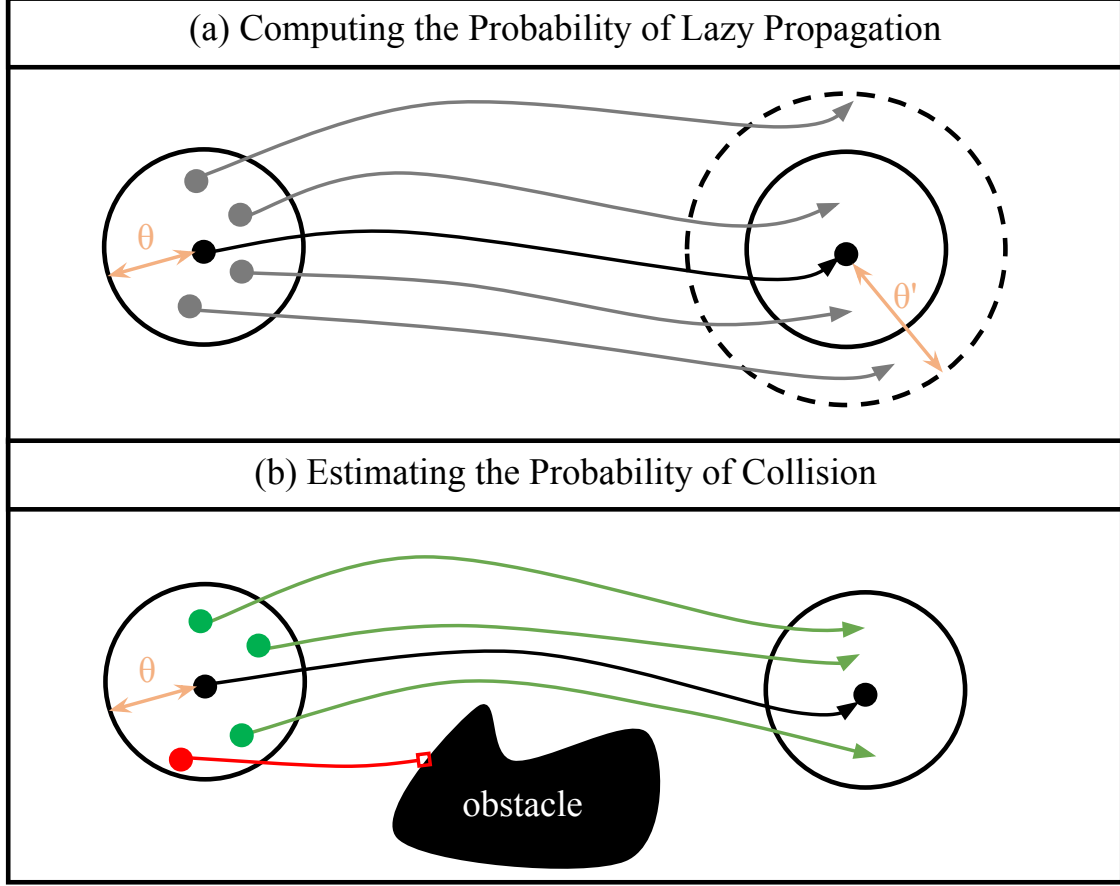


Figure 4.2: Perturbing the start state of an edge $e_i \in \mathcal{E}$ (black arrow) by a maximum of θ and applying $e_i.u$ for duration $e_i.\Delta t$ results in a perturbed end state ($\hat{q}_f$). Applying multiple random perturbations for e_i (grey arrows in (a), green and red arrows in (b)) allows us to estimate $e_i.P_{\text{lazy_prop}} = Pr(\|\hat{q}_f - e_i.q_f\|_2 < \theta) = 2/4$ and $e_i.P_{\text{collision}} = 1/4$ for the provided example. We repeat this process for all edges in $\mathcal{E}$ to integrate lazy approaches to simulation and collision checking in *LazyBoE*.

again during the planning stage, where edges emerging from θ -regions around planning tree nodes undergo simulation and collision checks [130]. This θ is dependent on state-space dimensions and denotes the maximum level of similarity in the state space, thereby bounding the set of valid controls. Additionally, the greedy, heuristic-biased search strategy used by the BoE planner has the potential to get stuck in local minima, thereby necessitating a method for quickly evaluating these greedy paths without simulating them entirely. This motivates the need for lazy approaches to simulation and collision-checking in the kinodynamic setting. More specifically, we need to address the following two questions for each edge $e_i \in \mathcal{E}$:

- (1) Can e_i be lazily propagated instead of undergoing a full simulation?
- (2) What is the probability that a lazily propagated edge will result in a collision?

We address these questions empirically by conducting a perturbation analysis on each edge in the bundle. A disturbance Δq in the range $(0, \theta]$ is added to the start state for each edge. The resultant end state for an e_i is computed as $\hat{q}_f^i = f(q_0^i + \Delta q, \dot{q}^i)$. In the limit that the number of perturbations for each edge (p) approaches infinity, we are able to accurately estimate both the maximum error in the end state, $\Delta q_f^i = \max(\|e.\mathbf{q}_f - \hat{\mathbf{q}}_f\|_2)$, and the potential collision likelihood.

To address the first question, an edge viable for lazy propagation is characterized as one that confines the end-state error within the region radius θ , so that viable edges for the next level of propagations can be picked from this fixed θ -neighborhood. However, this criterion is subjected to a more stringent requirement, as depicted in Figure 4.3, i.e., the end-state error for e_i must be confined to $\theta/2$ if e_i undergoes lazy propagation to ensure the worst-case error between its real simulation and the following connection to the lazy edge is θ . Put differently, the probability that an edge can be lazily propagated $P_{\text{lazy_prop}}^i = Pr(\Delta q_f^i < \theta/2)$ is the ratio of the number of valid (collision-free and dynamically-feasible) perturbations for which the maximum end state error is $\theta/2$ to the total number of valid perturbations, p_{valid} (Figure 4.2a).

Similarly, in response to the second question, we can quantify the probability that an edge will collide when lazily propagated, i.e., $P_{\text{collision}}^i = Pr(e_i \text{ collides})$, as the ratio of the number

of edges that collide to the total number of perturbations p (Figure 4.2b). This is in contrast to prior work which only considers lazy collision checking in the geometric planning domain and takes a deterministic approach, iteratively discarding edges that are not collision-free [15, 36]. The neighborhood-based control propagation in our work allows for a more nuanced and stochastic approach to lazy collision checking by allowing edges to be re-explored, potentially uncovering paths that would be prematurely eliminated by deterministic methods. In this work, we primarily deal with self-collisions and environmental collisions between the robot and its surrounding environment. These two probability values for each edge extend the definition of an edge bundle $\mathcal{E}$ to the following:

$$\mathcal{E} = \{(q_0, \dot{\mathbf{q}}, \Delta t, q_f, P_{\text{lazy_prop}}, P_{\text{collision}}) \mid q_0, q_f \in \mathcal{Q}, \dot{\mathbf{q}} \in \mathcal{U}\}$$

These two metrics, associated with each edge, are used to guide the planner by deciding when to apply lazy methods for faster planning times.

4.3.4 Kinodynamic Motion Planner

In order to reduce the computational costs associated with dynamics simulation and collision checking, we present a lazy approach for exploring the space of edge bundles $\mathcal{E}$, deferring computation until a viable, low cost approximation to a true candidate path has been found. Our method, termed *LazyBoE*, builds upon the design decisions laid out in the BoE planner [130]. In addition, it leverages the computed probabilities $P_{\text{lazy_prop}}$ and $P_{\text{collision}}$ and combines them into the probability of selecting an edge for lazy propagation as $P_{\text{select}} = (1 - P_{\text{collision}}) * P_{\text{lazy_prop}}$ (Algorithm 2, Line 14). This encourages edges with a lower collision probability to be propagated first, while also promoting lazy propagation. Since the edges are evaluated probabilistically, they can be used more than once, adding diversity and breadth to exploration, countering the downsides of the greedy approach as taken by the BoE planner. Stochasticity thereby avoids local minima and oscillation issues in greedy search.

In the ideal case scenario, all edges can be lazily propagated, enabling us to search through the edge bundle until the goal is reached and doing a real propagation on the shortest path from start

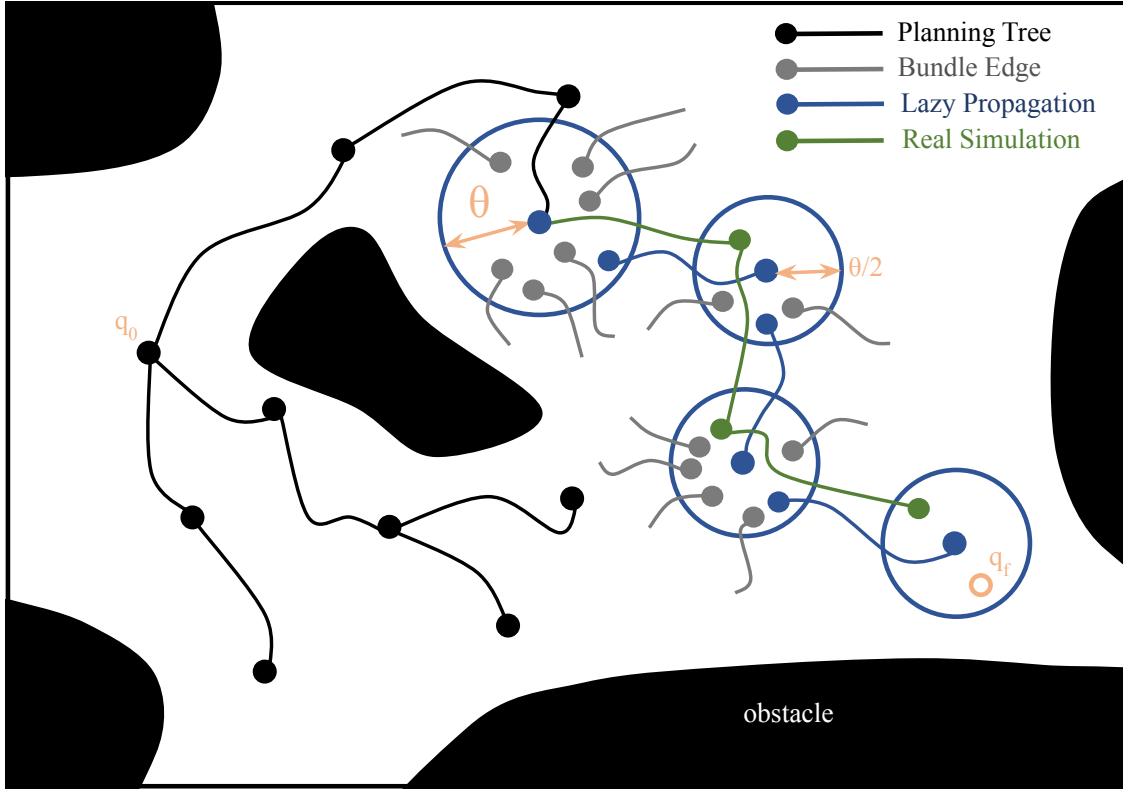


Figure 4.3: *LazyBoE* can lazily propagate a series of edges from $\mathcal{E}$ (shown in grey) in a way that minimizes heuristic cost. After a lazy candidate path is found (*blue*), a full simulation and collision check is performed along this path (*green*) to extend the planning tree (*black*). To ensure the highest rate of success when performing simulation on lazy paths, neighborhood lookup is restricted to $\theta/2$ for lazy search to allow, in the worst case, the maximum error between the end of a simulated edge and the start of the next lazy edge to be θ .

Algorithm 2: LazyBoE

Input: Start State q_0 , Goal Region $\mathcal{Q}_{goal}$, Edge Bundle $\mathcal{E}$, Neighborhood Radius θ
Output: Path π

```

1  $\pi \leftarrow \emptyset$ 
2  $\mathcal{V} \leftarrow \{q_0\}$ 
3 while  $\mathcal{V} \neq \emptyset$  do
    // equivalent to SELECT(.) in [130]
4      $v \leftarrow \text{PICK\_NODE}(\mathcal{V})$ 
    // update  $\pi$  to lower cost path
5     if  $v \in \mathcal{Q}_{goal}$  and !IS\_LAZY(EDGE\_TO( $v$ )) then
6         if cost( $\pi$ ) > cost(PATH\_TO( $v$ )) then
7              $\pi \leftarrow \text{PATH\_TO}(v)$ 
8     if IS\_LAZY(EDGE\_TO( $v$ )) then
9          $\mathcal{E}_{neighbor} \leftarrow \text{RADIAL\_NN}(v, \theta/2, \mathcal{E})$ 
10    else
11         $\mathcal{E}_{neighbor} \leftarrow \text{RADIAL\_NN}(v, \theta, \mathcal{E})$ 
12     $\mathcal{V}_{next} \leftarrow \emptyset$ 
13    foreach  $e \in \mathcal{E}_{neighbor}$  do
14         $P_{select} \leftarrow (1 - e.P_{collision}) * e.P_{lazy\_prop}$ 
15         $\text{apply\_lazy\_prop} \leftarrow \text{rand}() < P_{select}$ 
16        if  $\text{apply\_lazy\_prop}$  then
17             $\mathcal{V}_{next} \leftarrow \mathcal{V}_{next} \cup e.q_f$ 
18        else if !apply\_lazy\_prop or !IS\_LAZY(EDGE\_TO( $v$ )) or (IS\_LAZY(EDGE\_TO( $v$ )) and
            SHOULD\_TERMINATE\_LAZY()) then
19            // simulate
20             $e_{new} \leftarrow f(v, e, u, e, \Delta t)$ 
21            if COLLISION\_FREE( $e_{new}$ ) and WITHIN\_LIMITS( $e_{new}$ ) and
                IS\_CONTINUOUS( $e_{new}$ ) then
22                 $\mathcal{V}_{next} \leftarrow \mathcal{V}_{next} \cup e_{new}.q_f$ 
23     $\mathcal{V} \leftarrow \mathcal{V} \cup \mathcal{V}_{next}$ 
24 return  $\pi$ 
  
```

to goal. However, this does not hold due to real simulations that may lead to end states beyond the neighborhood radius $\theta < \theta'$ (Figure 4.2a), thereby resulting in $P_{\text{lazy_prop}} < 1$ for many edges. Therefore, it becomes important to identify the termination conditions for defining when to stop lazy propagation. These termination conditions (as defined by `SHOULD_TERMINATE_LAZY()` in Algorithm 2, Line 18) are two-fold: i) heuristic value, in our case, the distance to goal state, starts increasing, or; ii) lazy propagation path reaches the goal region $\mathcal{Q}_{\text{goal}}$. As a result, simulation along the lazy path is triggered when these conditions hold, if the preceding edge is not a lazy one, or with probability $1 - P_{\text{select}}$ (Algorithm 2, Line 18). Next, we demonstrate how these lazy approaches lead to lower planning times and higher success rates in comparison to baseline kinodynamic motion planning algorithms.

4.4 Evaluation

4.4.1 Test Setup

We benchmark *LazyBoE* against a number of kinodynamic planners with forward propagation, namely RRT [74], SST [79], and BoE [130]. We test three variants of SST based on different selection-to-pruning radius (0.5x, 1x, and 2x). Each algorithm is evaluated over 25 planning problems, i.e., pairs of collision-free, randomly-sampled start and goal joint angles, on four metrics: i) *time to initial solution* in seconds (Figure 4.4), ii) *final solution cost* as measured by the arclength of the trajectory in joint angle space (Figure 4.5), iii) *success rate* denoted by the fraction of trials that resulted in at least one planned trajectory (Figure 4.6), and iv) *number of solutions* per planning problem (Figure 4.7). This delineation into time to initial solution and final solution cost is necessitated by the fact that SST, BoE, and *LazyBoE* are asymptotically optimal in nature. All algorithms were implemented in Python 3.8 and evaluated on a 6-core AMD Ryzen 5 laptop with 32GB RAM running Ubuntu 20.04. Each algorithm was given a time budget of 60s, except RRT, which terminates on the first solution.

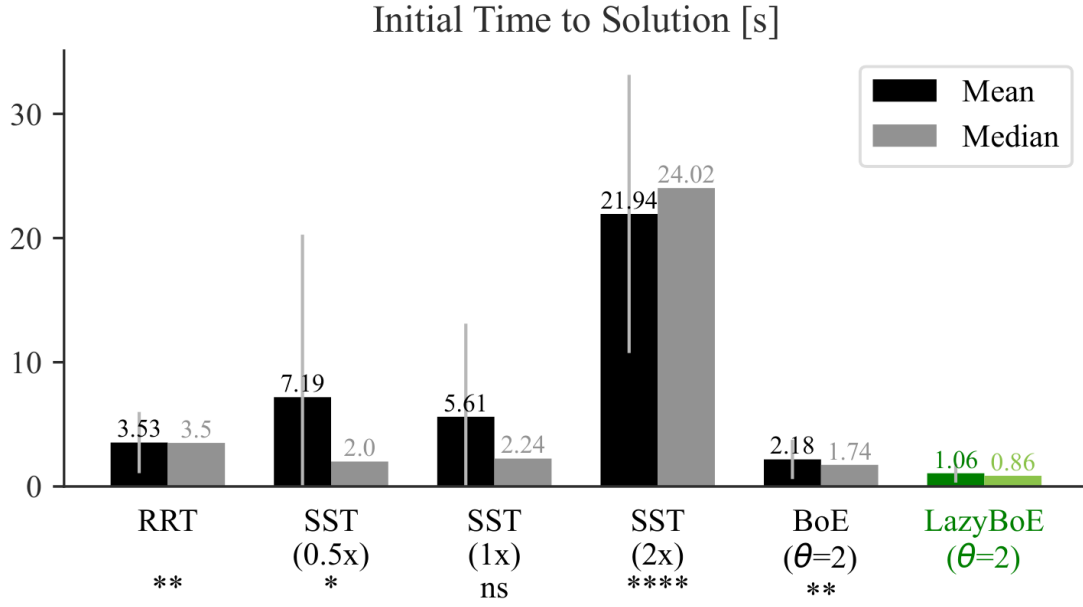


Figure 4.4: Our method is able to find the first solution in less time than baseline approaches, owing to its lazy approach to simulation and collision checking. Asterisks indicate significance level when comparing our planner to the baseline methods. *ns* indicates no significant difference.

4.4.2 Quantitative Analysis

Our *LazyBoE* planner outperformed baseline kinodynamic methods in multiple key metrics. First, it achieved an average solution time of 1.06s, substantially faster than the other methods that were in the range [2.18s, 21.94s] (Figure 4.4). This is due to deferred propagation and collision checking, which enable a more focused search.

Of note, despite this significant performance improvement, *LazyBoE*’s final solution cost was competitive in quality (3.42 cost) to the best performing alternatives, BoE (3.43 cost) and SST (Figure 4.5). In essence, our method provides efficiency without sacrificing optimality.

The planner also found an average of 3.12 solutions, over 1.5x more than the other methods (Figure 4.7). This increased solution diversity arises from stochastic edge selection and reuse compared to greedy approaches. Lazily reusing edges rather than discarding them expands the search frontier faster. More paths are available to evaluate and iterate on, thereby directly translating to higher success rates. With more valid candidate paths approximated and available to the search, the likelihood of discovering a collision-free solution path increases substantially. The results validate this, with *LazyBoE* succeeding 92% of the time compared to [80 – 88]% for other methods (Figure 4.6). In summary, *LazyBoE*’s use of lazy propagation and collisions to minimize wasted computations allows efficient searches that find high quality solutions quickly, while maintaining a broad exploration. This quantitative analysis validates lazy techniques can improve performance in kinodynamic planning.

4.4.3 Significance Testing

Statistical analysis using the Mann-Whitney U test further reinforces the results above. p -values for each test are highlighted in Figure 4.4, Figure 4.5, and Figure 4.7 and marked by ‘ns’ for $p > 0.05$, ‘*’ for $p \leq 0.05$, ‘**’ for $p \leq 0.01$, ‘***’ for $p \leq 0.001$, and ‘****’ for $p \leq 0.0001$.

LazyBoE’s speedup improvement is statistically significant with respect to most baseline algorithms, with notably low p -values in comparison to RRT, SST (0.5x), and SST (2x). However,

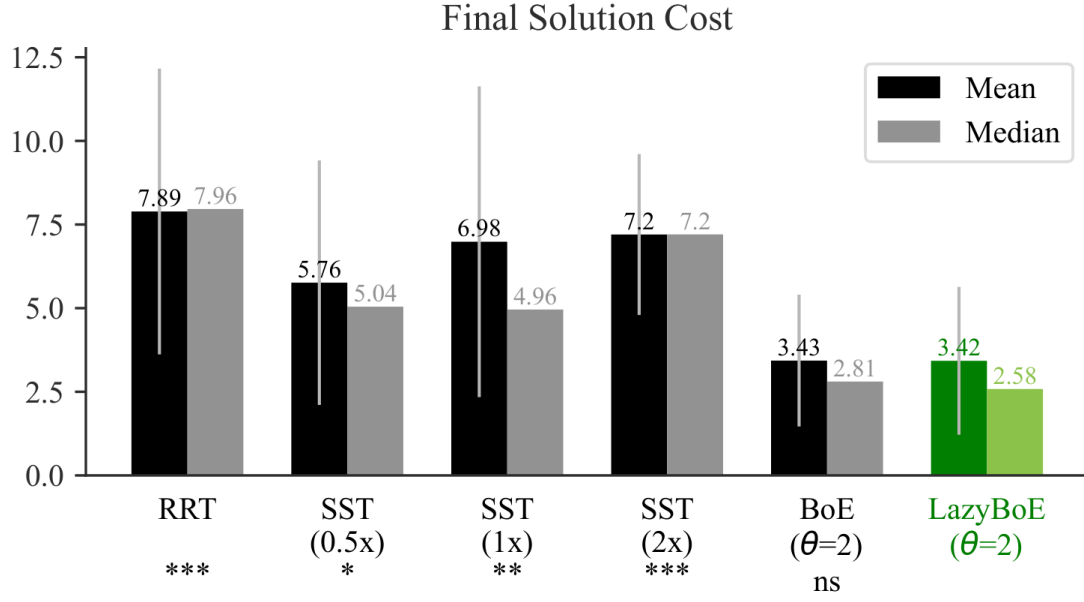


Figure 4.5: The final solution cost, i.e., the solution with the lowest cost, for our planner is comparable to the BoE planner. Asterisks indicate significance levels.

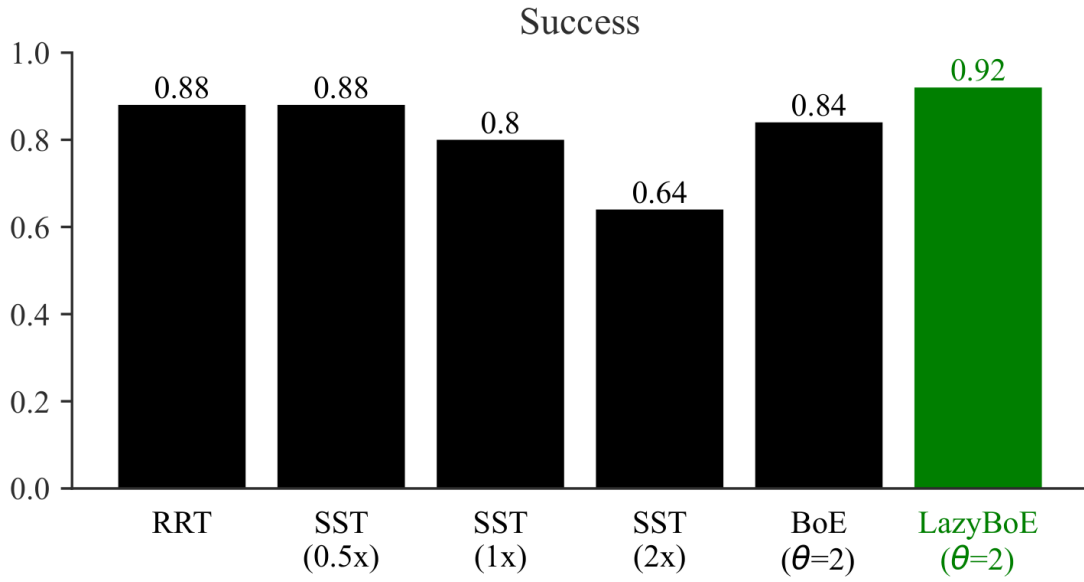


Figure 4.6: Our algorithm has a higher success rate compared to the baselines owing to the exploration of a greater number of potential solutions (Figure 4.7).

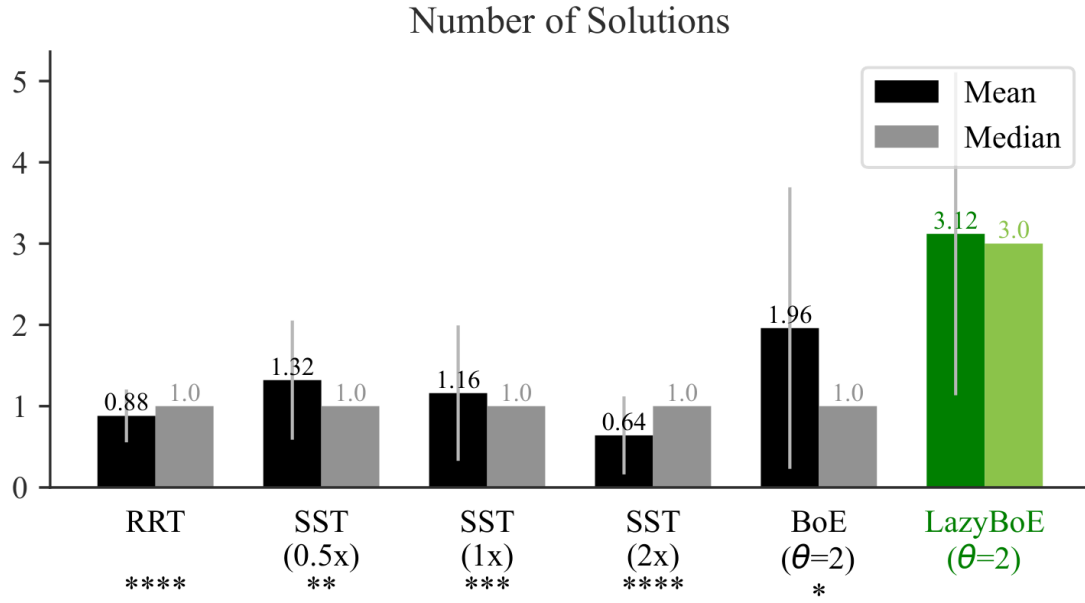


Figure 4.7: Our planner is able to explore a larger number of solutions than the baseline methods. RRT terminates after finding the first solution, but the mean number of solutions is less than 1 because RRT does not succeed in finding a solution for every planning problem Figure 4.6. Asterisks indicate the significance levels.

the comparison with SST (1x) did not reach statistical significance ($p = 0.0529$), indicating that the time difference with this baseline is not conclusive.

For a more robust analysis, we look at the median value, which provides a more accurate representation of the data’s central tendency, particularly in the presence of a skewed distribution. *LazyBoE* has a lower median cost when compared to various iterations of SST (0.5x) and RRT, demonstrating a potential advantage. It is worth noting that the cost comparison with the BoE algorithm did not yield a significant difference ($p = 0.8142$), suggesting comparable cost-efficiency between these two algorithms. Our algorithm also generated a higher median number of solutions compared to all baselines, with statistically significant results in all comparisons. In summary, this significance testing reinforces our claims that *LazyBoE* offers performance improvements in terms of both speed and cost-effectiveness.

4.5 Conclusion and Discussion

In this work, we presented *LazyBoE*, a multi-query approach to kinodynamic motion planning that takes advantage of lazy propagation and collision checking to outperform existing baselines in a number of key metrics. Specifically, our method reduces planning time and demonstrates a high success rate, given its ability to explore a wider range of solutions. However, our method faces certain limitations. The varying control sampling for a given Monte Carlo propagation introduces jitter into the trajectory which has to be smoothed with a low-pass filter before real-world execution. Future work can explore biased sampling techniques that consider the history of control states. Due to memory limitations, *LazyBoE* is confined to a limited subset of the robot’s state and control spaces. Scaling the approach to cover the entire robot workspace is a pressing challenge, stemming from memory constraints, especially when planning dynamically-feasible trajectories that may involve higher-order derivatives of the state space variables. While strategies such as selective loading or using a database could alleviate this, they require significant engineering optimizations.

Beyond the engineering required to scale this method, we also intend to explore application-centric extensions, such as planning for heavy payloads, liquid transport, and nonprehensile manip-

ulation, which can benefit from assigning task-specific weights to edges for more biased, multi-query exploration. While our implementation focuses on kinodynamic planning for robotic manipulation, the foundational principles of our planner are designed with broader applicability in mind. Generalizing the planner to accommodate a wider array of kinodynamic planning problems, beyond the realm of robotic manipulation, represents a promising direction for future work.

4.6 Case Study: Whole-Body Non-Prehensile Manipulation Under Actuation Failures

This case study carries the precomputation principle of this chapter into the task space: reusable kinodynamic motion structure is built for a robot whose embodiment is partially disabled by mechanical failure; this section summarizes the problem structure, methods, and key results.

4.6.1 Long Deployments and Locked-Joint Failures

Robots are increasingly integrated into critical applications, ranging from industrial automation to healthcare and even space exploration [143, 70, 104]. The reliability and continuous operation of these robotic systems are crucial for their acceptance and utilization in such demanding environments. However, as robots perform tasks over extended periods, the likelihood of mechanical failures increases [5, 129, 96]. A particularly critical and crippling case is when multiple joints fail simultaneously. These multi-joint failures can drastically reduce the robot’s task space and challenge standard motion planning algorithms due to the constraints of the robot’s configuration and control spaces [62].

4.6.2 Effects on Workspace, Manipulability, and Feasibility

These constraints in the robot’s kinematics and dynamics can degrade its manipulation capability through reduced abilities. For a robot manipulator that becomes underactuated due to multi-joint failures, several constraints can occur: i) the position and orientation of the end effector become coupled to the robot’s remaining functional joints, ii) the manipulable area will be con-

strained to a subset of its non-failure size, iii) grasping may not be possible due to task constraints, such as an object being in an ungraspable orientation, iv) the robot may lose the ability to manipulate the end-effector entirely due to an adverse configuration, and v) the end effector will be subject to nonholonomic constraints [13].

We consider two failure cases on the Franka Emika Panda arm, shown in Figure 4.8. Failure Case 1 locks three joints—the Posterior Elbow Rotation, Anterior Elbow Rotation, and Wrist Rotation—which limits the graspable area and reduces the range of grasping configurations by coupling the end-effector orientation to the joint configuration. Failure Case 2 locks the Wrist Lift and Wrist Rotation joints, precluding use of the end-effector in completing manipulation tasks entirely.

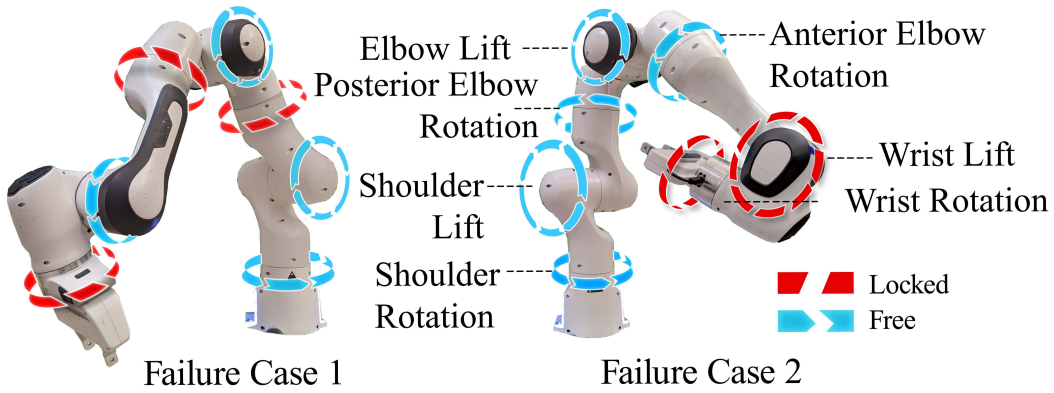


Figure 4.8: Two locked multi-joint (LMJ) failure cases on the Franka Emika Panda arm (failed joints indicated by red rings). **Left (Case 1):** Posterior Elbow Rotation, Anterior Elbow Rotation, and Wrist Rotation joints locked, restricting grasp configurations. **Right (Case 2):** Wrist Lift and Wrist Rotation joints locked, precluding end-effector use entirely.

4.6.3 Whole-Body Non-Prehensile Manipulation as Recovery

To counteract the limits multi-joint failures impose on the robot’s manipulation capability, we must shift the manipulation paradigm to include abilities beyond grasping and interaction beyond the end-effector. We propose using non-prehensile manipulation (NPM), or abilities corresponding to “anything but” grasping, through contact across the whole robot embodiment, to compensate for these constraints.

We leverage the poking primitive from Chapter 3 as a representative NPM ability to enable manipulation in the presence of LMJs, since it significantly expands the set of reachable configurations and the number of degrees of freedom of the robot’s operational space. The challenge of identifying a viable sequence of NPM actions is compounded when dealing with robots experiencing locked joint failures: the search must find actions feasible within the altered kinematic landscape imposed by the failure. This intersection often leads to a zero-measure manifold, which is difficult to sample when following a uniformly random strategy. To mitigate this issue, we propose a pre-computation approach, which maps the reach and capability post-LMJ and then finds a set of dynamic actions the robot can achieve, considering the limitations imposed by LMJs.

We adapt kinodynamic edge bundles to store NPM actions in a failure-centric map [130], a method that captures the effects of nonholonomic constraints on end-effector motion. The set of all possible joint states and valid control inputs form manifolds $\mathcal{C}$ and $\mathcal{U}$; under LMJ failure, these degenerate into constrained manifolds $\mathcal{C}_c$ and $\mathcal{U}_c$ with significantly smaller volume [62]. We approximate $\mathcal{C}_c$ through a collection of Monte Carlo rollouts—kinodynamic edges—of the robot’s dynamical model with random control inputs sampled from $\mathcal{U}_c$. This precomputed *failure-centric kinodynamic map* is multi-query, requiring only a single offline build per failure configuration.

To utilize the kinodynamic map, we developed a sim-in-the-loop task planner. To understand the benefits of utilizing a simulator in the planning loop, we study three Action Selection Mechanisms (ASMs): the **Random** ASM finds all the edges that intersect the object and picks one randomly; the **Lazy** ASM picks the first edge that the simulator predicts will move the target object toward the goal; and the **Greedy** ASM scores a subset of candidate edges in simulation and executes the highest-scored edge on the real robot.

Relying solely on the end effector renders the robot ineffectual in failure cases that severely or entirely limit its use. NPM abilities and the capability to interact with the environment using the whole embodiment allow for increased robot manipulable area. The nominal full-dexterity manipulable area is 0.85 m^2 ; the use of NPM increases this to 108% of nominal (Table 4.1). When using NPM actions, the robot maintains 92% of the reachable area in Failure Case 1 compared to

<i>Failure Case</i>	<i>Ability</i>	<i>Area (m²)</i>	<i>Change in Reachable Area</i>
<i>No Failure</i>	NPM+PM	0.92	Datum
	PM only	0.85	−8%
<i>Case 1</i>	NPM+PM	0.85	−8%
	PM only	0.62	−33%
<i>Case 2</i>	NPM+PM	0.73	−21%
	PM only	0	−100%

Table 4.1: Reachability data. Whole-body NPM recovers most of the nominal workspace even when prehensile-only manipulation fails completely.

67% when using prehensile manipulation only. Since Failure Case 2 precludes use of the end-effector, the prehensile-only reachable area is eliminated; however, using NPM actions, the reachable area is only reduced by 21%.

Across 267 real-world trials over three scenarios and two failure cases, the Greedy ASM achieves a 100% success rate in Failure Case 1 across all scenarios. Overall, our data shows a new and unique ability to conduct manipulation tasks while experiencing LMJs that restrict the robot’s capability to manipulate using traditional pick-and-place approaches. Our sim-in-the-loop planner improved the robot’s ability to do these tasks by looping in a physics model to our NPM actions.

This case study follows the same structure as *LazyBoE*: sampling-based planning augmented with compact precomputed physical structure to make an otherwise low-measure feasible set tractable to explore. Sampling alone is thereby insufficient when physical constraints concentrate valid states on a thin manifold; targeted precomputation is necessary to steer the planner into the feasible region efficiently. Together, both these works take *explicit* search amortization close to its practical limit: they reuse precomputed kinodynamic edge bundles across related queries, but those bundles are still tied to known collision geometry and a fixed dynamics model. Higher path quality therefore comes from denser sampling, which increases storage, lookup, and preprocessing cost, while changes in the constraints require new bundles or revalidation. The next part of the thesis shifts from storing reusable motion structure to learning conditional trajectory distributions that can amortize feasibility across changing task and constraint conditions.

Chapter 5

From Sampling to Generation: Planning as Conditional Sampling

The planners of the preceding chapters either evaluate dynamic feasibility online (Chapter 3) or store it in explicit precomputed structure (Chapter 4). This chapter reframes motion generation as *conditional sampling*: rather than searching for a feasible trajectory, we draw one from a learned distribution whose conditioning encodes the constraints under which it must be feasible. This reframing establishes the conceptual bridge between kinodynamic planning and the trajectory diffusion models developed in Chapters 6 and 7.

5.1 Reframing Planning

Diffusion models have recently emerged as an effective generative modeling approach for producing high-quality, diverse samples from learned approximations to conditional or unconditional *multimodal* distributions, albeit at the cost of longer generation times [131, 39, 121, 101]. This class of generative methods has found application in robotics in two main areas: imitation learning [22, 134] and seeding for trajectory optimization [21, 41], owing to its ability to represent multimodal distributions effectively and produce diverse and high-quality samples from the modeled data space. While existing applications have shown diverse real-world behaviors through end-effector delta actions, they primarily focus on kinematic constraints, require many human demonstrations, and necessitate training a new model for novel behaviors [22, 2, 34, 154]. These limitations direct my research toward the seeding problem for trajectory optimization—specifically, developing diffusion models that produce kinematically and dynamically admissible trajectories which can further

be refined by standard trajectory optimization iterations.

Diffusion models offer a compelling solution to trajectory generation through their inherent properties: they naturally explore diverse solution trajectories through stochastic denoising [22] while modeling complex, multimodal distributions that accommodate rich contextual information. A distinctive advantage of diffusion models is the ability to apply cost guidance *at inference time*, steering the denoising process to generate trajectories that avoid obstacles and respect higher-order constraints—a capability not native to most alternative generative architectures. Diffusion models also afford *parallel sampling*: because denoising is a batched tensor operation, many candidate trajectories can be generated simultaneously on modern hardware for little more than the cost of one, whereas a sampling-based planner must grow its search structure sequentially. The result is a diverse set of candidates from which downstream components can select the best trajectory or seed optimization with multiple initializations.

5.2 Background on Diffusion

Diffusion operates as a two-step process: forward and reverse (also known as inference, sampling, planning, or denoising). In its simplest implementation, the training process consists of adding noise according to a scheduler to a training data sample, then training a neural architecture to predict the magnitude of added noise (epsilon prediction), and minimizing the MSE loss between the added noise and the predicted noise [39]. During inference, random Gaussian noise of the same dimensionality as the target data is generated and iteratively denoised through the trained neural architecture over multiple steps to generate a valid sample.

A joint space trajectory with t waypoints at denoising step k is defined as $\boldsymbol{\pi}^k = [X_0^k, \dots, X_{t-1}^k]^T$ where $X_i = (q_i, \dot{q}_i, \ddot{q}_i) \in \mathbb{R}^n$ represents the complete state (joint angles, velocities, and accelerations) of an n -DoF robot arm at timestep i . The training process for the diffusion model approximates the conditional distribution $p(\boldsymbol{\pi} \mid \mathcal{C})$, where $\mathcal{C}$ is the target conditioning embedding (Figure 5.1). It samples $\boldsymbol{\pi}$ from dataset $\mathcal{D}$, adds noise according to a schedule parameterized by α , γ , and σ , and learns to predict the underlying sample. The noise prediction network ϵ_θ , where θ represents the

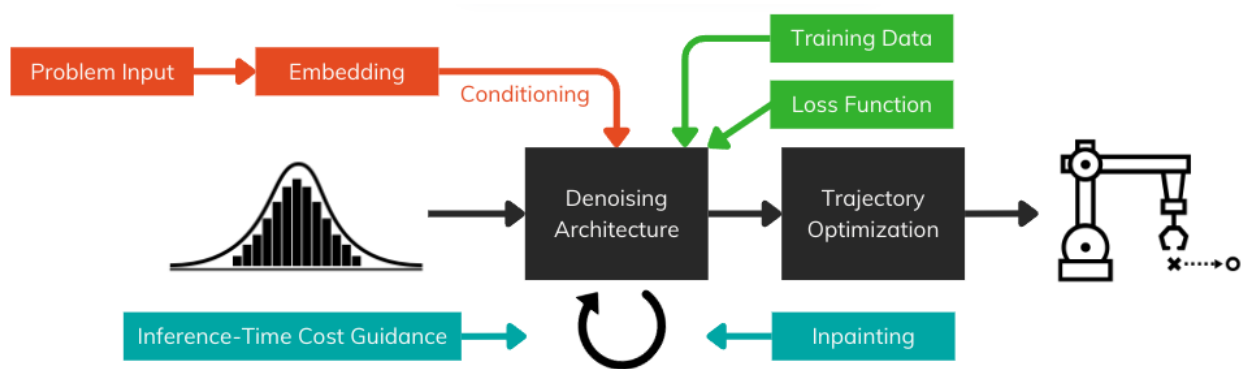


Figure 5.1: When combined, **input conditioning**, **training dynamics**, and **inference-time guidance** enable a strong multimodal approximation of the distribution of (near-)admissible trajectories that can be refined with trajectory optimization.

parameters of the 1D U-Net architecture, is trained to minimize

$$\mathcal{L}(\theta) = \text{MSE}(\epsilon^k, \epsilon_\theta(\mathcal{C}, \boldsymbol{\pi}^0 + \epsilon^k, k)) \quad (5.1)$$

Given a trained ϵ_θ , the K -step iterative denoising process

$$\boldsymbol{\pi}^{k-1} = \alpha \cdot (\boldsymbol{\pi}^k - \gamma \epsilon_\theta(\mathcal{C}, \boldsymbol{\pi}^k, k) + \mathcal{N}(0, \sigma^2 I)) \quad (5.2)$$

starts from $\boldsymbol{\pi}^K \sim \mathcal{N}(0, I)$ and generates a trajectory $\boldsymbol{\pi}^0$ that respects the conditioning information and obeys start, goal, joint limit, collision, and dynamic constraints.

Behavior Cloning and Imitation. In imitation learning, diffusion models learn a policy directly from demonstration data, generating end-effector delta actions conditioned on sensory observations such as RGB images [22, 134], point clouds [154], and tokenized 3D scene features combined with language [148, 57]; the same recipe underlies generalist policies that decode actions through diffusion heads [139]. This role has demonstrated impressive real-world generalization but remains fundamentally tied to kinematic, position-controlled execution.

Trajectory-Level Planning. In trajectory-level planning, the diffusion model generates full trajectories rather than per-step actions: either directly as a planner through guided denoising [49, 21, 123], or as near-feasible joint-space seeds for downstream trajectory optimization [128, 41]. This is the role that the remainder of this thesis follows: rather than executing diffusion outputs directly, we use them as near-admissible initializations that substantially reduce the iteration burden on optimization-based refinement.

In practice, start and goal states are enforced via inpainting [21], i.e., terminal states are ignored from the denoising process and set to problem constraints $\boldsymbol{\pi}_0^k = X_{start}$ and $\boldsymbol{\pi}_{t-1}^k = X_{goal}$ where $\dot{q}_{start} = \dot{q}_{goal} = \ddot{q}_{start} = \ddot{q}_{goal} = \mathbf{0}$. Collisions are avoided via inference-time gradient guidance $\boldsymbol{\pi}^{k-1} = \boldsymbol{\pi}^k - \beta \cdot \nabla J(\boldsymbol{\pi}; E)$ where β is a tunable guidance weight, J defines the collision cost function and E represents the set of all obstacles in the environment [123]. Additionally, joint limits are enforced via clamping during training and inference. While existing approaches have shown diverse real-world behaviors, they primarily focus on kinematic constraints and soft constraint compliance remains the fundamental ceiling of the standard pipeline.

5.3 Levers for Tightening Constraint Compliance

The diffusion-based trajectory generation system comprises four core components (Figure 5.1):

Problem-Specific Conditioning. Real-world tasks invariably operate under both kinematic and dynamic constraints. These constraints are incorporated via the conditioning mechanism, effectively learning a conditional distribution that leverages prior work in sensory feature learning [66] and feature-wise conditioning [111]. This conditioning can take various forms—RGB images [22, 134], point clouds [154], depth information [41], or task-centric representations such as task parameters, returns, and constraint specifications [128, 2]—all aimed at generating near-admissible motions that can be further refined by trajectory optimization.

Training Data Composition. This unconditional approximation can be enhanced to better capture the underlying distribution through high-quality, task-specific training data, typically produced at scale by expert planners [30, 21, 128]. The model can also be guided toward on-manifold learning through task-specific loss function design, in addition to the standard noise prediction loss [22, 123, 21].

Loss Design. Examples of loss functions include trajectory smoothness and collision distance [128], and Cartesian space error and non-linearity-aware weighting [41]. These design choices can be supplemented by the numerous methods available in machine learning literature to support generalization (e.g., regularization, data augmentation, cross-validation).

Inference-Time Cost Guidance. A distinctive advantage of diffusion models is the ability to apply cost guidance *at inference time*, steering the denoising process to generate trajectories that avoid obstacles and respect higher-order constraints [49, 21, 123]. This approach requires diversity in training data to function effectively.

Inpainting. Additionally, inpainting techniques—originally developed for image completion [81]—enforce hard state compliance for problem-specific constraints like start and goal states, while the remainder of the trajectory is generated through the denoising process [49, 123, 21].

5.4 What the Literature Emphasizes and What It Underexplores

While these building blocks provide a foundation for trajectory generation, existing diffusion-based approaches primarily focus on kinematic feasibility and collision avoidance [21, 123, 128, 41], overlooking the critical role of dynamics constraints in real-world robotic tasks. This limitation significantly reduces their effectiveness as initializers for trajectory optimizers like GOMP-FIT [43], which must subsequently reconcile the kinematically-viable but dynamically-infeasible paths. My work addresses the fundamental challenge of incorporating dynamic constraints directly into the diffusion model framework. Rather than treating diffusion models as geometric path planners, I present a work that uses a high-level conditioning mechanism to implicitly encode dynamic constraints for a payload transport task (Chapter 6); and another follow-up work that presents a task-agnostic trajectory generation framework (Chapter 7). Combined, these approaches aim to significantly reduce the computation burden on trajectory optimization with kinematic and dynamic constraints.

Chapter 6

Compressed Representation for Kinodynamic Feasibility

Chapter 4 amortized dynamic feasibility by storing explicit kinodynamic motion structure, while Chapter 5 reframed motion generation as conditional sampling. Because explicit motion structure scales poorly with the dimensionality of the robot state and the range of constraints the system must accommodate, this chapter replaces stored motions with trajectory diffusion models that encode how the distribution of feasible motion varies with task-relevant physical conditions. The primary instantiation is super-nominal payload manipulation, where payload mass conditions the generated joint-space trajectory. A subsequent case study applies the same principle to failure-induced embodiments, where changes in available actuation alter the distribution of feasible motion.

6.1 Introduction: Diffusion for Super-Nominal Payload Manipulation

Manipulator design establishes fundamental performance boundaries through the interplay of mechanical structure, actuation system, and control architecture. At the hardware level, these capabilities manifest as explicit physical constraints: joint angle limits that define the reachable workspace, maximum velocities bounded by motor specifications, and torque limits determined by the drivetrain capacity. However, a crucial distinction exists between these hardware limitations and the operational capacities specified by manufacturers for general-purpose deployment. While hardware limits are absolute physical constraints, nominal specifications—like payload capacity and effective workspace boundaries—are *derived* conservatively by considering worst-case scenarios across possible configurations and trajectories. In practice, system integrators and end-users

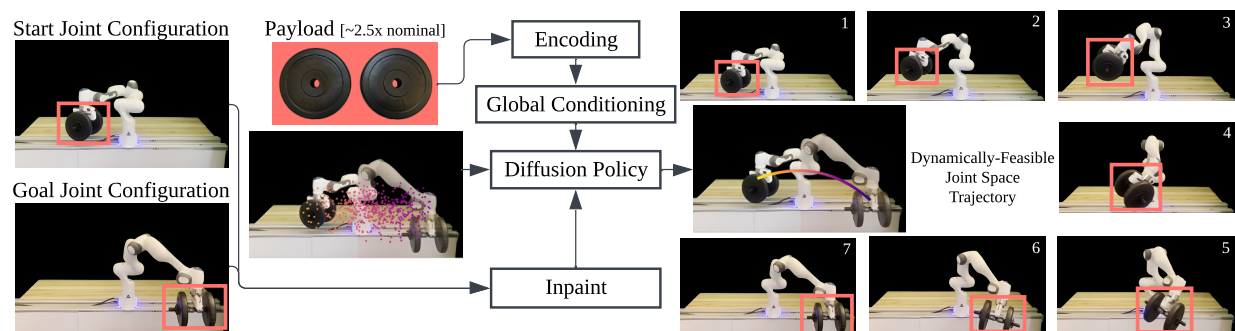


Figure 6.1: The diffusion model presented in this work learns to generate dynamically feasible trajectories directly in joint angle, velocity, and acceleration space, enabling super-nominal payload manipulation.

are locked into treating these derived operational capacities as hard limits rather than contextual capabilities, constrained by warranty requirements to design processes well within these conservative bounds. This well-established approach of *over-provisioning* hardware to ensure ample safety buffers has system-wide cascading effects—from the need to procure oversized actuators and power supplies, to reinforcing mounting structures for added load tolerance, to expanding facility requirements to accommodate larger equipment. For instance, when an application requires handling a 35kg payload, end-users are forced to select a 50kg-rated robot over a 30kg-rated system, despite the latter being capable of safely executing the task within broad regions of its operational space.

It is possible to overcome the issue of hardware over-provisioning through software-based optimizations, which allow to expand the safe operational envelope of a fixed/given robot embodiment and enable it to perform beyond nominal payload specifications. This requires unified handling of both kinematic and dynamic constraints, as payload characteristics interact with joint configurations and higher-order derivatives to determine feasible motions. Even if they have not solved this specific problem, prior approaches in this space have investigated the challenge from multiple perspectives: geometric planners with dynamics post-processing [103, 11], kinodynamic planning [130], optimization-based methods [43], and control-based frameworks [4]. However, these methods face inherent trade-offs between design complexity, constraint handling, and computational efficiency; these become particularly critical when operating near system limits where the feasible solution space becomes highly constrained. These limitations highlight the need for a method that balances constraint satisfaction with generalizability—i.e., the ability to adapt to varying payloads, environmental conditions, and task requirements while maintaining consistent performance.

Of note, diffusion models [22, 134] present a compelling opportunity address the “curse of dimensionality” inherent in trajectory planning with simultaneous kinematic and dynamic constraint satisfaction. *By learning from successful trajectories generated by multiple (imperfect) planning methods*, these models can extract various homotopic solution classes that overcome the limitations of individual approaches. This distills successful solutions from diverse planning approaches into a unified representation that generalizes across the entire feasible space, including regions where

individual planners fail. The inherent stochasticity in the denoising process enables the exploration of diverse trajectories [22], while their capacity to model high-dimensional distributions allows incorporation of rich contextual information such as sensor data and problem constraints. However, existing diffusion-based approaches have been confined to position-controlled systems [21, 123], leaving the challenge of integrating both kinematic and dynamic constraints unaddressed.

In this work, we propose a method that directly tackles this problem by jointly learning valid distributions of positions, velocities, and accelerations that satisfy both constraint types simultaneously—moving beyond prior work that only denoises in position space. Central to our approach is the use of payload mass—a high-level object property—as a conditioning signal that implicitly encodes the full complexity of payload-induced dynamics into the generative process, without requiring explicit constraint formulation at inference time. Through investigation of payload encoding strategies, we demonstrate that diffusion models effectively learn the complex relationships between joint states and payload-dependent dynamics, enabling constant-time generation ($\approx 10\text{ms}$) of feasible trajectories that preserve 67.6% workspace accessibility at $3\times$ nominal payload capacity—without explicit constraint checking at runtime. This demonstrates the importance of dynamics-informed generative models in expanding the operational boundaries of robotic systems.

6.2 Related Work

Manufacturer-specified operational parameters for general deployment of robots are typically far more conservative than hardware-level physical limits and often lack transparency in their derivation. This has led to significant under-utilization of robotic capabilities, as evidenced by geometric motion planning and time parameterization approaches dominating most industrial use [11, 23, 103] and recent advances in foundation models for manipulation in unstructured environments being restricted to exclusively geometric reasoning tasks [102, 139, 160]. A nuanced, configuration-dependent understanding of the robot’s specifications that considers both robot and environment dynamics could enable a broader spectrum of manipulation capabilities within hardware limits.

Higher-order constraints such as joint torques and end-effector accelerations can be addressed through several approaches that range from “plan-and-filter” methods which post-process geometric paths [1, 67] to kinodynamic planners that directly incorporate dynamics during planning [78, 99, 106]. While these approaches offer completeness guarantees, they face practical limitations in high-dimensional manipulation spaces and often exhibit high variance in planning times especially when faced with state or model uncertainty. These challenges are particularly acute when planning near operational limits, where the feasible solution space becomes increasingly constrained [62]. Optimization-based methods accommodate dynamics constraints but rely heavily on good initialization, leading to hybrid approaches that bootstrap the optimization process with learning or sampling-based solutions [44, 67, 151]. Optimal control formulations for dynamics-aware manipulation also exist [4, 60]. Despite their theoretical rigor, they face practical limitations in handling complex constraints and achieving reliable convergence for real-world manipulation problems.

Importantly, there exist a large body of works that demonstrate how a robot’s behavioral repertoire can be significantly expanded when incorporating dynamic constraints—e.g. nonprehensile manipulation [28, 108, 122, 153, 155] or fast inertial transport [4, 43, 94, 6]. These capabilities and considerations are particularly relevant for the specific problem of payload manipulation, where the interaction between payload and induced joint torques significantly influence task success. Interestingly, payload manipulation presents unique challenges that intersect robot design and control. Traditionally, prior work has focused on mechanical modeling for specific low-DoF systems [144], with emphasis on payload capacity optimization [65] and joint torque considerations [100]. Other work in this domain tackles super-nominal payload handling through the use of multi-arm systems [59, 150]. However, the field lacks generalizable approaches for high-DoF systems that can reason about dynamic payload-robot interactions across diverse tasks.

Learning-based approaches offer promising directions for both generalizability and efficient trajectory generation in high-dimensional constrained spaces while respecting system dynamics. Autoregressive models implicitly encode dynamics through dataset design [30, 118], while recent diffusion-based approaches provide alternatives with improved sampling diversity [21, 123]. Task-

level diffusion policies, conditioned on language and visual embeddings [2, 22], demonstrate potential for generating diverse behaviors from high-level task specifications. Our method conditions a denoising diffusion model on the target payload to generate dynamically feasible trajectories.

6.3 Diffusion Models for Trajectory Generation

Diffusion models are a class of generative models that produce high-quality and diverse samples across multiple data modalities [39]. These models operate through a two-phase process: first, gradually corrupting training data with Gaussian noise in a forward process, then learning to reverse this corruption through an iterative denoising process. In the context of robot motion planning, we use diffusion to generate collision-free and dynamically feasible trajectories.

A joint space trajectory with t waypoints at denoising step k is defined as $\boldsymbol{\pi}^k = [X_0^k; \dots; X_{t-1}^k]$ where $X_i = (q_i, \dot{q}_i, \ddot{q}_i) \in \mathbb{R}^{3n}$ represents the complete state (joint angles, velocities, and accelerations) of an n -DoF robot arm at timestep i . The training process for the diffusion model approximates the conditional distribution $p(\boldsymbol{\pi} \mid \mathcal{P})$, where $\mathcal{P}$ is the target payload representation (Section 6.4.2). It does so by sampling $\boldsymbol{\pi}$ from dataset $\mathcal{D}$, adding noise according to a noise schedule parameterized by α , γ , and σ , and learning to predict the underlying sample. The noise prediction network ϵ_θ , where θ represents the parameters of the 1D U-Net architecture, is trained to minimize $\mathcal{L}(\theta) = MSE(\epsilon^k, \epsilon_\theta(\mathcal{P}, \boldsymbol{\pi}^0 + \epsilon^k, k))$.

Given a trained ϵ_θ , the K -step iterative denoising process $\boldsymbol{\pi}^{k-1} = \alpha \cdot (\boldsymbol{\pi}^k - \gamma \epsilon_\theta(\mathcal{P}, \boldsymbol{\pi}^k, k) + \mathcal{N}(0, \sigma^2 I))$ starts from $\boldsymbol{\pi}^K \sim \mathcal{N}(0, I)$ and generates a trajectory $\boldsymbol{\pi}^0$ that can support a target payload and obeys start, goal, joint limit, and collision constraints. In practice, start and goal states are enforced via inpainting [21], i.e., $\boldsymbol{\pi}_0^k = X_{start}$ and $\boldsymbol{\pi}_{t-1}^k = X_{goal}$ where $\dot{q}_{start} = \dot{q}_{goal} = \ddot{q}_{start} = \ddot{q}_{goal} = \mathbf{0}$. Collisions are avoided via inference-time gradient guidance $\boldsymbol{\pi}^{k-1} = \boldsymbol{\pi}^k - \beta \cdot \nabla J(\boldsymbol{\pi}; E)$ where β is a tunable guidance weight, J defines the collision cost function, and E represents the set of all obstacles in the environment [123]. Additionally, joint limits are enforced via clamping during the training and inference processes. Specific to our work, $p(\boldsymbol{\pi} \mid \mathcal{P})$ implicitly models dynamically feasible payload-induced torques through an object property-based global conditioning mechanism

that influences the entire trajectory by appending target payload representation m to the diffusion timestep embedding in the noise prediction network architecture [22].

6.4 Payload-Conditioned Trajectory Denoising

Here, we present our payload-conditioned diffusion model to generate pick-and-place trajectories that transport super-nominal payloads while obeying kinematic and dynamic constraints.

6.4.1 Training Data

Our payload-conditioned diffusion model (Section 6.4.2) is trained on 25,000 time-parameterized, collision-free joint space trajectories (state dimension = DoF $\times$ 3, corresponding to joint angles, velocities, and accelerations) for tabletop pick-and-place motions. These trajectories are generated using a plan-and-filter pipeline that uses cuRobo, a GPU-accelerated trajectory optimization framework [138], followed by a dynamics validation step (Figure 6.2).

First, trajectories are generated for the 7DoF Franka Emika Panda robot arm with an effective payload mass of 0kg. Second, to ensure that payload-induced joint torques remain within hardware limits, each trajectory is verified through an analytically-derived dynamics model parameterized on real-world data [31]. This model captures joint torques induced by inertia, Coriolis effects, gravity, and velocity-dependent joint friction. By using a dynamics model fitted to real robot trajectory data, we ensure that our training dataset reflects the physical constraints observed in hardware.

This dynamics verification step utilizes a complete robot model where q , $\dot{q}$, $\ddot{q}$, and τ are 7×1 vectors representing joint angles, velocities, accelerations, and torques respectively. The dynamics matrices include: $M(q)$ (7×7 positive-definite, symmetric joint inertia matrix), $C(q, \dot{q})$ (7×7 Coriolis/centrifugal matrix), $g(q)$ (7×1 gravity compensation torque vector), and $f(\dot{q})$ (7×1 joint friction torque vector). External task space forces F_{ext} (6×1) map to joint torques through the Jacobian $J(q)$ (7×7). The dynamics model is parameterized as:

$$\tau = M(q)\ddot{q} + C(q, \dot{q})\dot{q} + g(q) + f(\dot{q}) + J^{-1}(q)F_{ext} \quad (6.1)$$

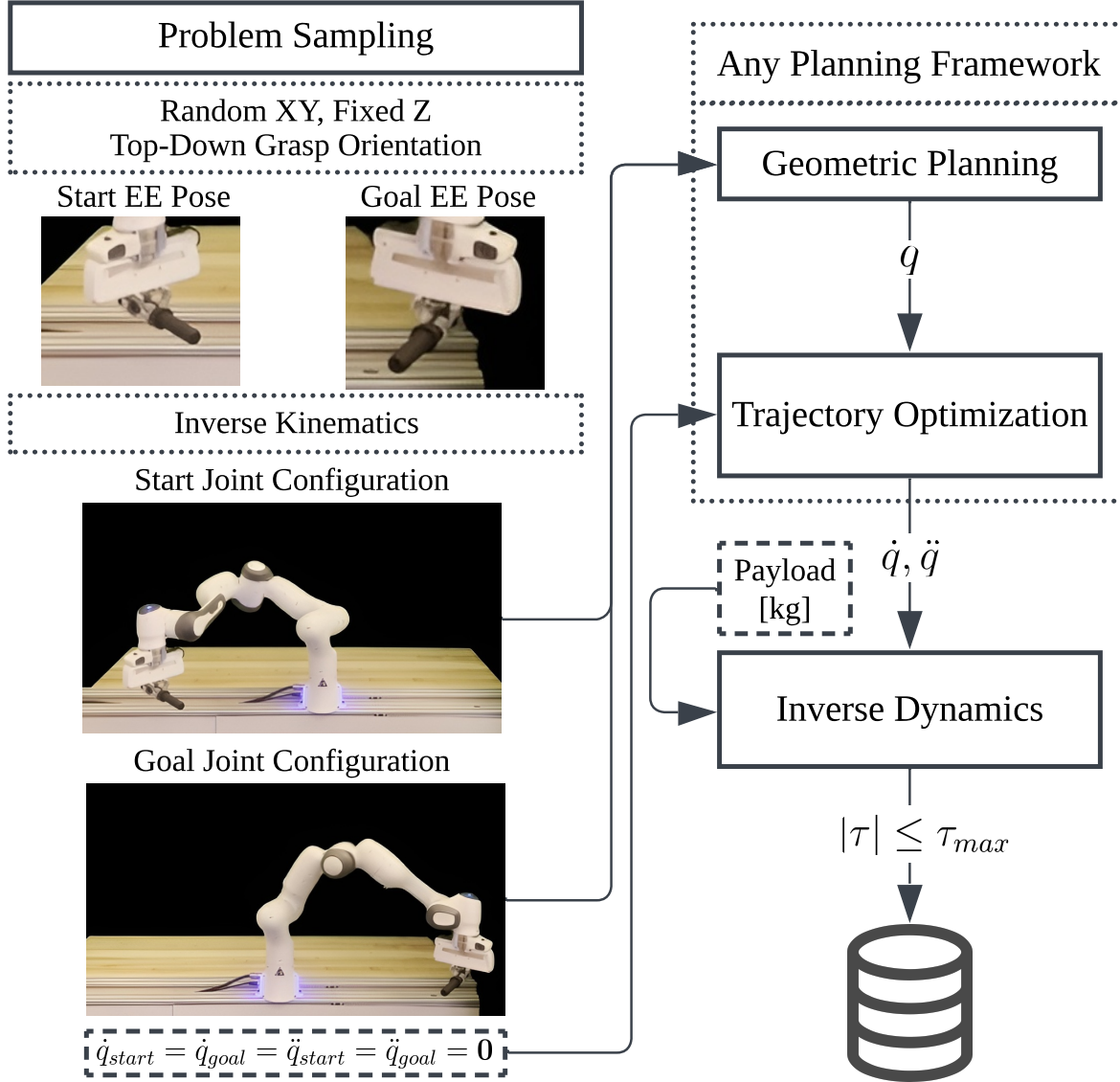


Figure 6.2: Plan-and-filter process to create training data.

Note that this analytically-derived model represents the robot without external payloads which can affect model accuracy when considering varying payload masses in real-world deployment. To account for payload-induced joint torques, we model the gravitational effects as a 6D Cartesian wrench transformed from the world frame to the end-effector frame. This frame transformation is critical for accurately projecting gravitational forces acting on the payload along the robot’s embodiment. We represent the external gravitational wrench $\mathbf{F}_g \in \mathbb{R}^6$ in the world frame with a simplified point mass model: $\mathbf{F}_g = mg \cdot [0, 0, -1, 0, 0, 0]^T$ where $m \in \mathbb{R}^+$ denotes payload mass and $g \approx 9.81 \text{ m/s}^2$ is gravitational acceleration. This representation assumes force application at the object’s center of mass, neglecting potential moment contributions. The wrench’s structure combines linear forces (first three components) and angular forces (last three components), which are mapped to induced joint torques through the manipulator Jacobian, contributing to the total joint torque vector (Equation (6.1)).

The training dataset $\mathcal{D}$ is constructed from these trajectories that satisfy both kinematic and dynamic constraints. Each element $d_i \in \mathcal{D}$ is a tuple of trajectory π_i and maximum supported payload m_i . Each m_i is the maximum supported payload mass for trajectory i subject to $|\tau(\pi_i, m_i)| \leq \tau_{max}, \forall t \in [0, T]$ where τ_t is computed via the dynamics equation (Equation (6.1)).

Having established our dataset $\mathcal{D}$ of dynamically feasible trajectories and their corresponding maximum payload limits, we now consider different ways of incorporating payload conditioning into the diffusion model architecture.

6.4.2 Payload-Conditional Trajectory Generation

We extend the diffusion policy architecture to condition trajectory generation on payload mass through global conditioning [22]. Unlike local conditioning where features influence each waypoint individually, global conditioning applies the mass embedding uniformly across the entire trajectory, preserving dynamic feasibility across the generated motion (Figure 6.3).

For each training trajectory i with corresponding maximum supported payload mass $m_i \in \mathcal{D}$, we investigate four approaches to incorporate a target payload mass p_i into an encoding vector $\mathcal{P}_i$.

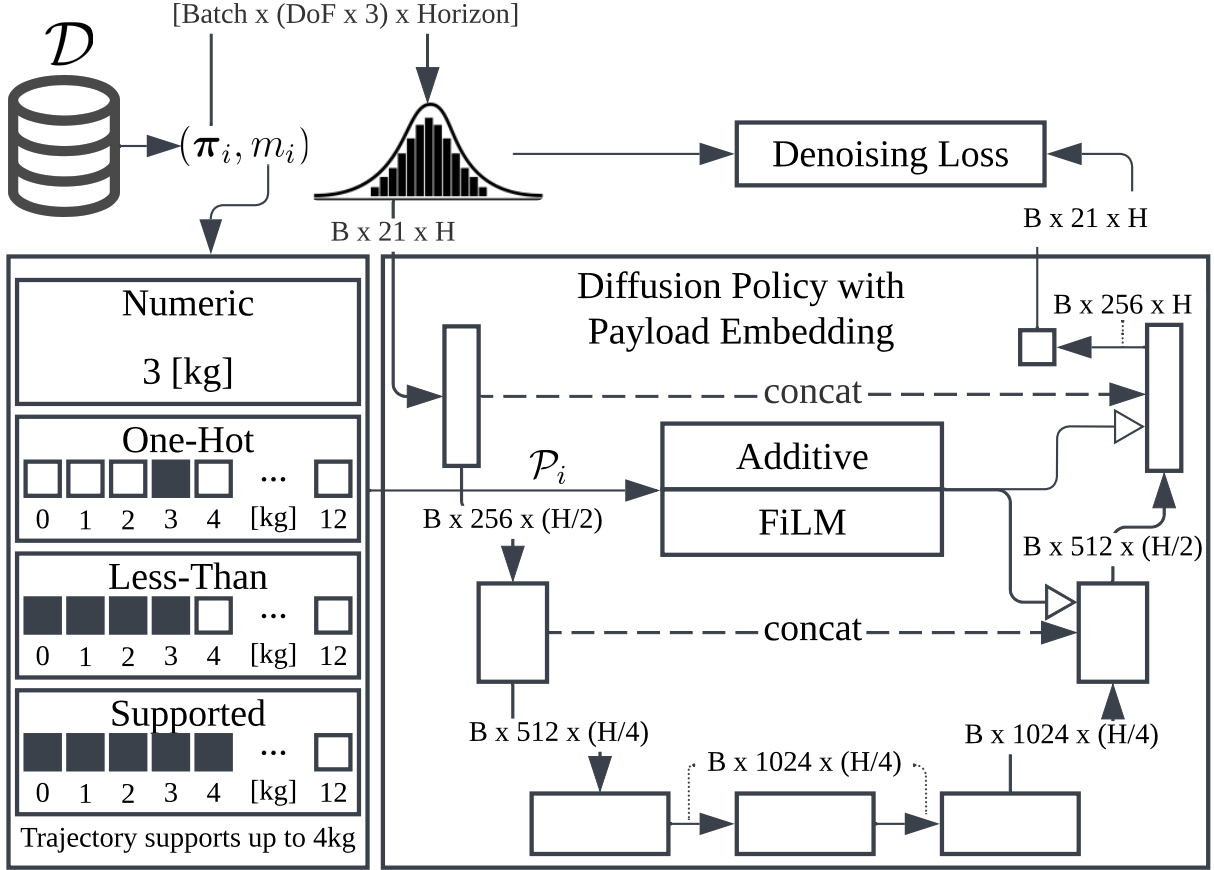


Figure 6.3: Our model conditions a 1D UNet denoising architecture [22] on various payload embeddings.

The payload encoding $\mathcal{P}$ is concatenated with a diffusion timestep embedding to form a global conditioning vector, which is then integrated into ϵ_θ through either additive or affine transform conditioning [111]. We consider the following approaches for designing $\mathcal{P}$.

Numeric Input. A random supported mass scalar $p_i \sim \mathcal{U}(0, m_i)$ is sampled and fed directly to the network. This approach naturally supports continuous payload values during inference, despite training on a discrete set of payload values.

One-Hot Encoding. For a system supporting payloads from 0 – 18kg in 1kg increments, a random supported mass $p_i \sim \mathcal{U}(0, m_i)$ is encoded as a 19-dimensional binary vector with a single non-zero entry at index $\lceil p_i \rceil$. While this categorical representation simplifies the learning task by discretizing the payload space, it constrains inference to the discrete payload values seen during training. During inference, continuous payload masses must be quantized to their ceiling value $\lceil p \rceil$ before input to the denoising network, ensuring a tight upper bound for conservative operation.

Less-Than Encoding. This encoding exploits the logical constraint that trajectories supporting a given payload must also support all lighter payloads. For a random supported mass $p_i \sim \mathcal{U}(0, m_i)$, we construct a 19-dimensional binary vector where entries are 1 for indices $j \leq \lceil p_i \rceil$ and 0 otherwise. This representation directly encodes the downward compatibility constraint into the input space, potentially improving the network’s ability to learn and respect payload-dependent constraints. During inference, continuous payload masses are similarly mapped to their ceiling value $\lceil p \rceil$ to construct the binary encoding.

Supported-Range Encoding. This approach encodes the complete range of supported payloads for a given trajectory. For training trajectory i with maximum supported mass m_i , we construct a 19-dimensional binary vector where entries are 1 for indices $j \leq \lceil m_i \rceil$ and 0 otherwise. During training, this encoding preserves the full payload compatibility information from $\mathcal{D}$. At inference time, for a target payload mass p , the encoding can be constructed either as a one-hot vector with a single non-zero entry at index $\lceil p \rceil$, or as a less-than encoding with ones for indices $j \leq \lceil p \rceil$. This dual interpretation enables both trajectory generation for specific payloads and exploration of the learned manifold of payload-trajectory relationships. Broadly, each encoding

method presents trade-offs between generalization capability and training simplicity.

6.5 Evaluation

To validate our method, we focus on pick-and-place motions—a canonical industrial robotics requirement where super-nominal payload manipulation can significantly extend the operational envelope of robot arms while obeying strict hardware limits. In our experiments, payloads were rigidly attached to the end-effector with a geometrically centered and uniform mass distribution. This was a deliberate experimental choice necessitated by the limited grasping force of our parallel gripper, ensuring repeatable execution across trials. Importantly, this choice is not a fundamental limitation of our method: as shown in the supplementary video, our approach is capable of handling non-rigidly attached objects and payloads with modest center-of-mass offsets, though some failure modes at higher payloads are also highlighted and explained there. During execution, invalid trajectories generated by the diffusion model are simply not executed. For valid trajectories that fail due to unmodeled dynamics (e.g., payload oscillations), standard robot safety stops are triggered automatically. Our approach relies on the manufacturer’s joint velocity controller for trajectory tracking, which provides sufficiently accurate state tracking for the tasks evaluated in this work.

6.5.1 Comparison of Payload Encodings

Our evaluation of different payload encodings, conducted with FiLM conditioning and payload encoding normalization, reveals several key insights in terms of planning success rate (Figure 6.4).

Numeric and one-hot approaches demonstrate comparable effectiveness, particularly for payloads within nominal capacity (0 – 3kg). However, all methods show a tendency to underestimate success rates for higher payloads, potentially due to data imbalance in the training set. When interpreted as a less-than encoding during inference, the supported-range encoding achieves competitive performance, but its one-hot interpretation leads to significantly lower success rates. While the encoding is designed to capture richer payload-trajectory relationships, the network struggles to fully utilize this information during generation. The inability of numeric, supported one-hot, and

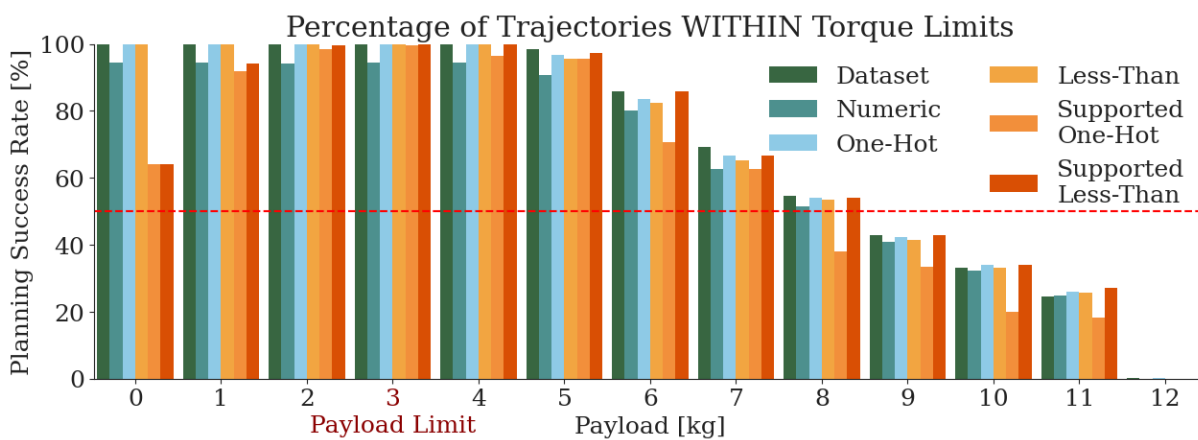


Figure 6.4: The aggregate success rate metric shows one-hot diffusion closely matching the underlying training data distribution.

supported less-than encodings to achieve 100% success rates in the most loosely-constrained and data-rich portions of the state space, i.e., the nominal operating regime ($0 - 3$ kg), suggest fundamental limitations in how these encoding schemes enable the diffusion model to obey kinematic and dynamic constraints. For subsequent experiments, we use with the one-hot encoding scheme, which demonstrates superior performance in terms of success rate.

6.5.2 Comparison across Planner Types

For the purposes of our discussion, we refer to DDIM (One-Hot) as a primary point of comparison. Denoising Diffusion Implicit Models (DDIM) offer accelerated inference due to fewer denoising steps (in our case, 5) [133]. We also present results for DDPM [39] with 25 denoising steps. Notably, our experiments demonstrate that DDIM achieves identical success rates to DDPM despite requiring significantly less planning time (≈ 0.01 s), validating our approach for realtime use (Figure 6.5c).

We evaluate our approach against a diverse set of trajectory planning methods using mean planning time to first solution and the corresponding success rate over 500 tabletop manipulation tasks as the primary metric (Figure 6.5c). This metric provides a fair basis for comparison across fundamentally different planning paradigms. Additional metrics include planning time factor (mean planning time of baseline / mean planning time of DDIM) (Figure 6.5a), relative success rate change over DDIM as a percentage (Figure 6.5b), and standard deviations of planning times (Figure 6.5d).

Plan-and-Filter Methods. Broadly, plan-and-filter methods employ a sequential process: first planning a geometric path via sampling-based motion planning, then time-parameterizing it to derive joint velocities and accelerations, and finally validating trajectory feasibility for the target payload using the robot dynamics model (Equation (6.1)). This iterative process continues until a feasible trajectory is found that satisfies all joint torque constraints induced by the payload [1]. We evaluate two CPU-accelerated geometric planning variants: VAMP (RRTConnect) with deterministic sampling and VAMP (RRTConnect) with random sampling [140]. We select Ruckig for time parameterization due to its ability to generate time-optimal trajectories in realtime [11].

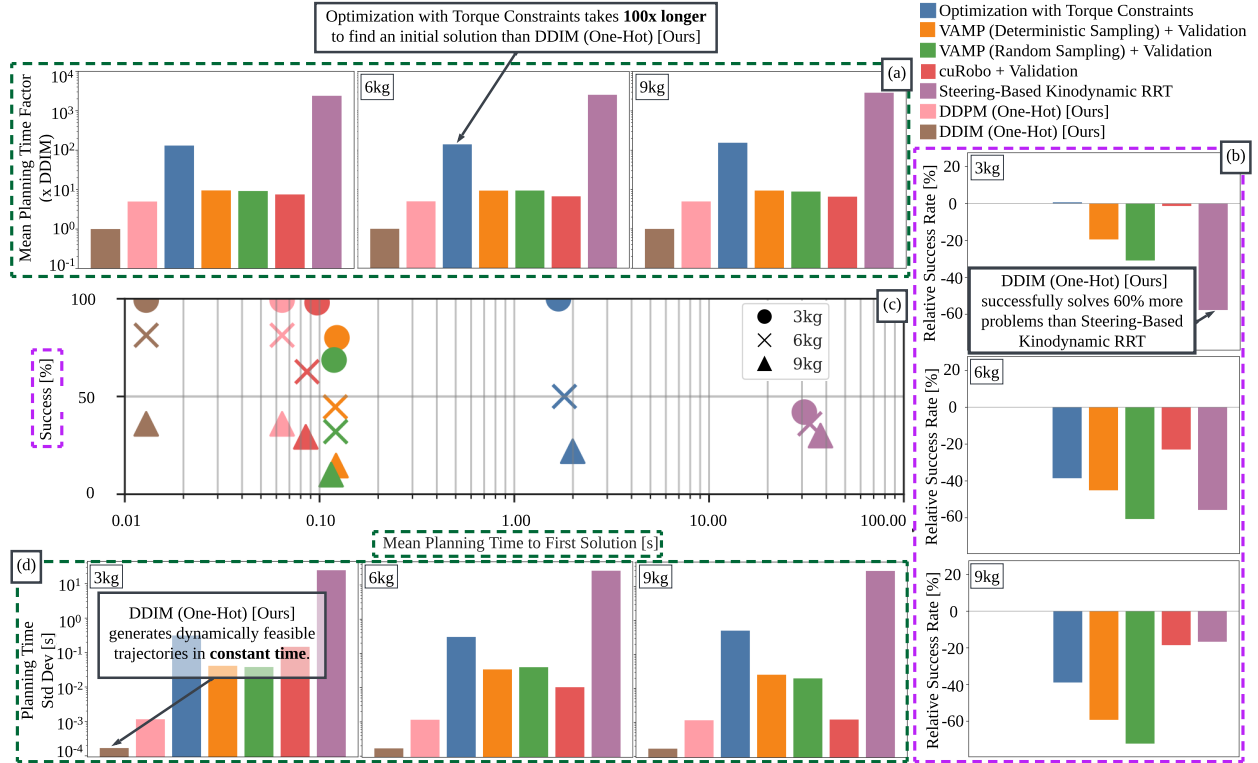


Figure 6.5: Comparative analysis of trajectory planning methods for payloads of 3kg, 6kg, and 9kg. DDIM (One-Hot) significantly outperforms a variety of baseline methods in both planning time to first solution and success rate, generating dynamically feasible trajectories in constant time across all payload conditions.

Geometric motion planning and time parameterization can alternatively be unified within a single optimization framework (here, cuRobo) which can be followed by the dynamics validity checking.

Since there is a decoupling between kinematic reasoning and dynamic reasoning, further exacerbated by the separate time parameterization step in the VAMP variants, these approaches demonstrate inferior performance compared to our one-hot DDIM model (Fig. 6.5c). VAMP with deterministic sampling achieves marginally higher success rates than VAMP with random sampling due to its low-dispersion sampling strategy, which provides more uniform exploration of the configuration space. Despite this sampling advantage, both variants exhibit higher standard deviations (Fig. 6.5d) and lower overall success rates, particularly with heavier payloads (Fig. 6.5b). These methods also exhibit significant planning time variability (Fig. 6.5a). The temporal predictability of our approach is particularly valuable for industrial pipelines, where high standard deviations in planning times can complicate integration with broader automation workflows. The success rate differential between cuRobo and DDIM particularly demonstrates that our model effectively learns from successful trajectories generated by imperfect planners. While cuRobo with validation often requires multiple attempts to find feasible solutions, our method leverages these curated trajectories during training to achieve higher success rates, especially under super-nominal payload conditions.

Kinodynamic Motion Planning. Plan-and-filter methods generate paths requiring post-processing to satisfy dynamic constraints, potentially invalidating the solution. By planning directly in the state space that includes joint angles, velocities and accelerations, kinodynamic planning ensures dynamically feasible solutions from the outset. Specifically, we employ steering-based kinodynamic RRT as our baseline, planning in the joint angle, velocity, and acceleration space [74]. Ruckig serves as our steering function owing to its ability to operate in realtime while respecting third-order constraints [11]. Each edge undergoes validity checking that includes dynamics (Equation (6.1)) and smoothness checks before addition to the planning graph. Results not only indicate high planning times to find the first solution ($> 1000\times$ compared to DDIM) due to “the curse of dimensionality” (Figure 6.5a), but also show high standard deviations due to random sampling in the high-dimensional state space (Figure 6.5d). The clustering of success rates observed across

all three payloads suggests an inherent planning bias in kinodynamic RRT, indicating insufficient exploration in this non-Euclidean state space (Figure 6.5a). Furthermore, kinodynamic approaches require extensive tuning of hyperparameters that interact with system dynamics. Suboptimal parameter selection frequently results in planning timeouts or physically implausible trajectories.

Trajectory Optimization with Torque Constraints. This baseline implements Sequential Least Squares Programming (SLSQP) optimization for trajectory generation, minimizing sum-squared jerk to ensure smoothness [43]. To facilitate direct comparison with our method, we constrain trajectory durations instead of performing time optimization [138, 43], isolating evaluation to payload manipulation capabilities. The formulation incorporates linearized representations of both dynamic payload constraints and collision avoidance parameters. The success rate disparity between optimization and DDIM becomes particularly pronounced with higher payloads as the optimization landscape becomes increasingly constrained. This significant performance gap warrants discussion in the context of our evaluation choices. We implemented an SLSQP optimization formulation as a baseline that directly handles dynamic constraints without sophisticated warm-starting or acceleration techniques. Recent work has shown that optimization-based planners can achieve substantially better performance both in planning time and success rate through strategies like learning-based seeding [44, 41]. However, our baseline still highlights the computational challenge of finding feasible solutions in high-dimensional spaces with nonlinear dynamic constraints.

6.6 Conclusion of this Work

This work demonstrates that manipulators can safely operate well beyond their nominal payload ratings through diffusion-based trajectory generation, challenging the traditional approach of hardware over-provisioning in industrial automation. By training on trajectories that span the full range of payload-dependent dynamic constraints and leveraging the successes of “imperfect” planners, our model learns to generate motions for complex tasks in constant time that respect system limitations without explicit constraint checking at runtime. This enables safe operation even at super-nominal payloads, maintaining 67.6% of the nominal workspace at $3\times$ rated capacity.

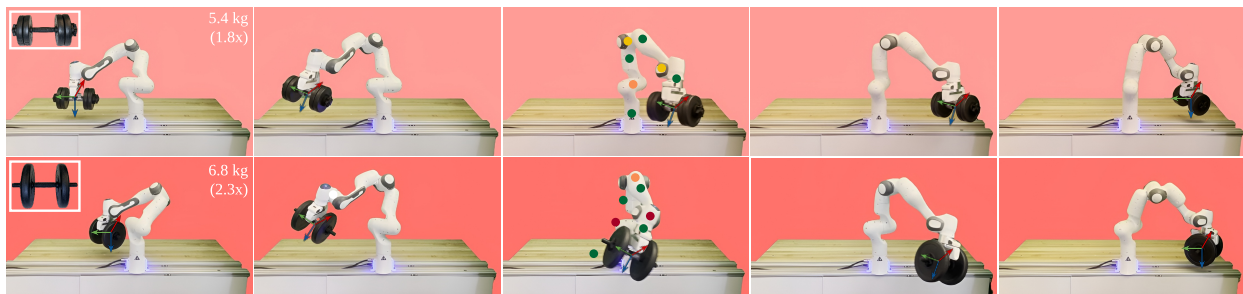


Figure 6.6: Two qualitative motions of the robot carrying super-nominal payloads of 5.4kg ($1.8\times$ nominal capacity) and 6.8kg ($2.3\times$ nominal capacity) are shown. Colored dots indicate joint torques as a percentage of joint limits: 0 – 20%, 20 – 40%, 40 – 60%, 60 – 80%, 80 – 100%. Joints 1, 3, and 5 exhibit higher torques, with joint 1 bearing the structural load of the arm and joints 3 and 5 positioning and orienting the payload.

Potential avenues for future work include studying how to determine the optimal composition of training data from different planners operating in varied environments. Moreover, the notion of conditioning on object-level characteristics can be generalized to handle objects with internal dynamics like liquids or articulated parts, moving beyond rigid-body assumptions. Such a framework could significantly reduce robotic system integration complexity and design burden by eliminating separate subsystems for different planning aspects, while maintaining the computational efficiency advantages and higher-order constraint satisfaction demonstrated in this work.

6.7 Limitations

While our results establish the viability of super-nominal payload manipulation on fixed/given hardware, several aspects warrant further discussion. **Rigid Attachment Choice.** Even though the rigid attachment experimental choice simplifies analysis by eliminating offset moments, it introduces two critical limitations in real-world execution (see accompanying video): dynamic effects from payload oscillation when objects are not perfectly rigid, and unmodeled torques when the object’s center of mass is significantly offset from the end-effector’s origin. These assumptions can lead to trajectory tracking errors or even task failures, particularly when handling flexible materials or asymmetric objects. Looking ahead, extending our approach to objects with internal dynamics will require explicit robot-object dynamics models to capture effects such as rotational inertia, slippage, and uneven mass distributions. Incorporating these nonlinearities, together with friction modeling for fingered grippers and suction force modeling for suction-based end-effectors, remains an important direction. Moreover, expanding the conditioning space to whole-body contact forces, fingertip wrench cones, and suction dynamics, as well as embedding more advanced controllers (e.g., impedance or admittance), will allow the same “constraint-aware by design” principle to scale to interaction-rich manipulation tasks. **Test-Time Robustness.** Assessing test-time robustness requires long-term deployment studies and, in many cases, proprietary Original Equipment Manufacturer (OEM) data on wear-and-tear. Whether robots should operate continuously at or near maximum capacity is also a design question for robot OEMs. While our method generates trajec-

tories that respect manufacturer-specified torque limits, joints do run hotter under high payloads, motivating further study of safety margins under sustained operation. **Embodiment Generalization.** We acknowledge that embodiment understanding must be more directly incorporated into the training process to enable generalization across different robotic platforms.

Our work also raises important theoretical questions about completeness and optimality guarantees for learning-based motion planning. Even though we can argue that diffusion models trained on optimal trajectories will generate similarly optimal solutions, establishing formal completeness guarantees remains an open challenge. Moreover, while our method demonstrates advantages in inference speed, traditional optimization-based approaches offer clearer constraint satisfaction guarantees and greater flexibility in adapting to new constraints without retraining. That said, training is not a bottleneck: we generate training trajectories across multiple payload conditions, with each trajectory verified for kinematic and dynamic feasibility throughout the robot’s operational workspace. The compact dimensionality of our denoised samples enables fast retraining of the UNet and efficient creation of a high-quality dataset with comprehensive operational space coverage, but without the need for real-world payload manipulation data (assuming the existence of a robot dynamics model, which also lends to system modularity).

More rigorous evaluations comparing CPU versus GPU performance and Python versus C++ implementations would enable direct baseline comparisons but standardizing these benchmarks remains challenging. Our evaluation of baseline methods maintains industry-standard implementations of baselines without additional optimizations for deployment infrastructure. Several other under-explored areas of research remain: optimal training data selection for maximizing planner generalization, disambiguating data imbalance from model generalization to explain the high variance in torques for higher payloads, leveraging diffusion models’ continual learning capabilities at industrial scale, and conditioning on real-world sensor modalities like point clouds for deployment.

6.8 Case Study: Diffusion Under Failure-Induced Embodiments

6.8.1 Motivation

The Mars rover Opportunity’s robotic arm first stalled on November 25, 2005; its shoulder joint experienced intermittent motor stalls for years, managed with voltage workarounds and revised stow procedures [98]. Similarly, Curiosity’s drill paused sampling for roughly eighteen months until a hand-designed drilling method restored coring [97]. Cases like these underscore that failures in continuously operating systems are inevitable [8]. Prevailing safety standards mandate *fail-freeze* behavior, requiring robots to halt when faults are detected [47, 48, 46]. The result is prolonged downtime and reduced autonomy. In this work, we contrast this overly conservative approach with *fail-active* behavior, a more ambitious (and harder to achieve) reliability goal in which robots maintain functionality, full or reduced, under fault conditions [87]. **Fail-active behavior is how humans operate:** we continue walking on a sprained ankle, writing with a non-dominant hand, going to work when we’ve had our hearts broken.

Faults reshape what the robot can physically do: actuation failures (e.g., joint range and velocity restrictions) shrink the feasible workspace and degrade manipulability, electrical faults (e.g., brownouts) reduce control authority, sensor drift can distort state estimates, and end-effector wear (e.g. lowered friction) degrades contact mechanics. Ultimately, these physical changes mean the robot can no longer move the way it originally planned.

In this case study, we focus on *actuation failures* as they directly affect how the robot moves. *Fail-active* operation requires adapting to these altered kinematics, which can cause end-effector motion to become non-holonomic; meaning the same control input may no longer yield the same end-effector motion [13]. It is also important to note that since degradation can evolve over time [7] and the failure state is not known a priori, **the space of failure modes is effectively unbounded**. Moreover, each joint can fail independently in multiple ways, and this complexity scales combinatorially with the robot’s degrees of freedom, **making reliable performance across failures hard to engineer**.

As failure-induced constraints redefine the robot’s workspace, manipulability, and feasible interactions with the world, we *posit that each failure mode can be interpreted as a new embodiment*. Through this lens, different embodiments are unable to perform the same tasks using the same behaviors. As no same set of behaviors can span all embodiments, multiple motion primitives are needed to expand the feasible action space and reachable task-space [17].

Synthesizing effective motion primitives for arbitrary failure conditions is challenging due to the complex relationship between robot embodiment constraints and feasible task-space behaviors. Existing approaches fall short in three key gaps: 1) they do not generalize to arbitrary failure configurations [17, 149]; 2) they are not capable of completing multiple manipulation primitives [115]; and 3) they do not address multi-joint failures [93, 58]. To the best of the authors’ knowledge, no existing work bridges all three of these gaps simultaneously. Diffusion models are uniquely suited to solve this. Because they excel at modeling multimodal data distributions, diffusion policies can generate across the range of actions required to span these infinite embodiments at inference.

6.8.2 Embodiment-Aware Conditioning

To this end, we present *DEFT*: a *Diffusion-based Embodiment-aware Fail-active Task-conditioned* trajectory generation framework, capable of maintaining robot manipulation functionality under failure conditions (Figure 6.7). Our contributions are: (i) an *embodiment vector* encoding per-joint actuation failures for online adaptation to previously unseen failure-induced embodiments and (ii) a *constraint encoding* selecting between constrained and unconstrained motions to enable the use of multiple motion primitives. Furthermore, we utilize start–goal inpainting and output clamping to enforce embodiment failure conditions [81].

Our goal is fail-active manipulation: generate functional, feasible trajectories under joint-level failures. We pose this as conditional trajectory generation given the failure-induced embodiment and the task constraint. Each failure specifies a new embodiment of the robot, and each constraint (e.g., **unconstrained** vs. **constrained** motion) induces a distinct trajectory distribution. Given the start and goal joint configurations the policy π infers a future sequence of joint states over

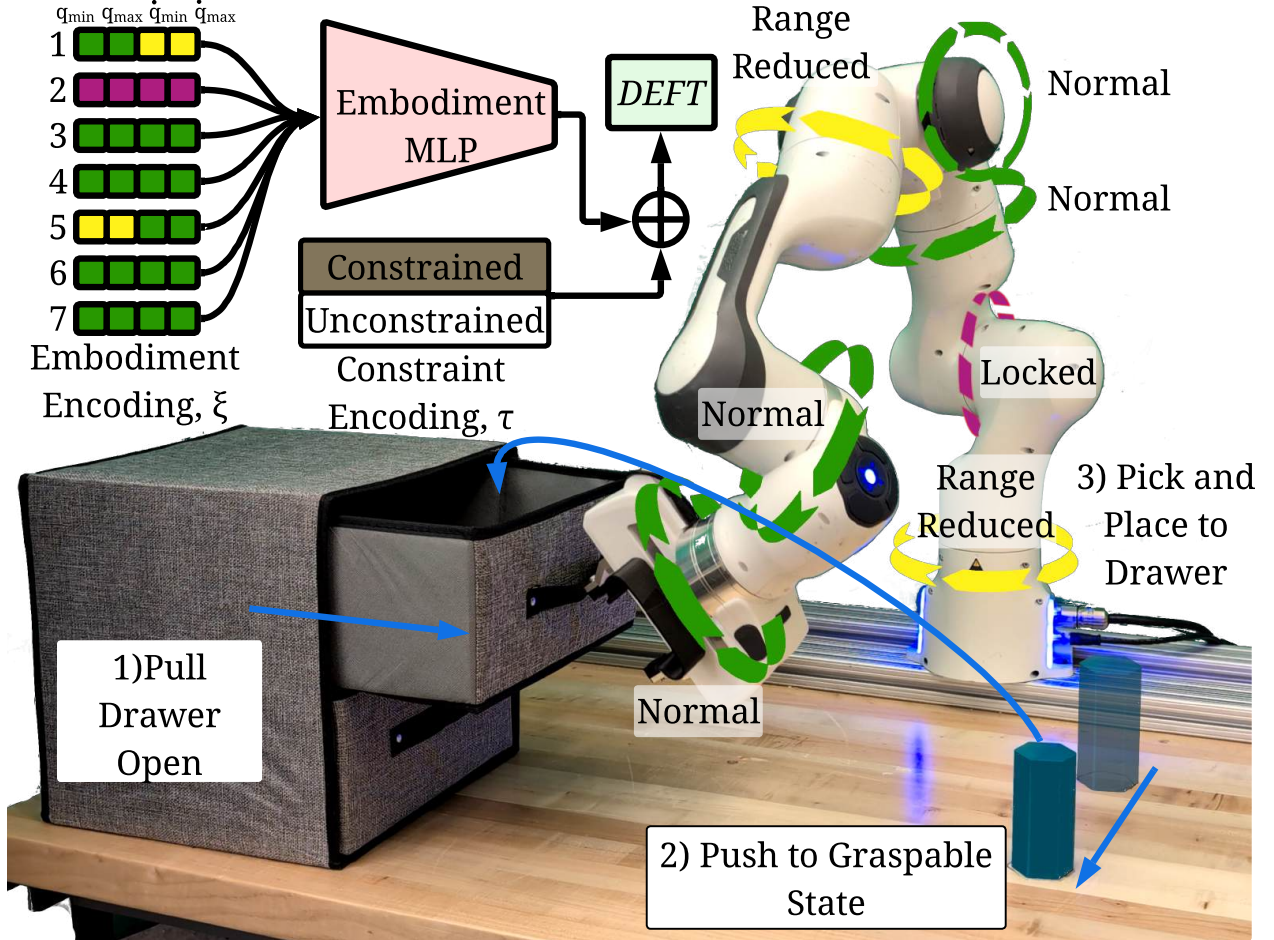


Figure 6.7: DEFT takes as input a structured embodiment encoding representing joint-level failures and a constraint encoding then synthesizes a feasible robot trajectory via a diffusion model. After experiencing a failure, a given task will likely need to be completed in a different manner. In this figure, the pick-and-place segment is no longer feasible, given the failure condition of a locked second joint and reduced velocity range on joint 1 and reduced angle range on joint 5, and the robot must now push the object to a graspable state to complete the task.

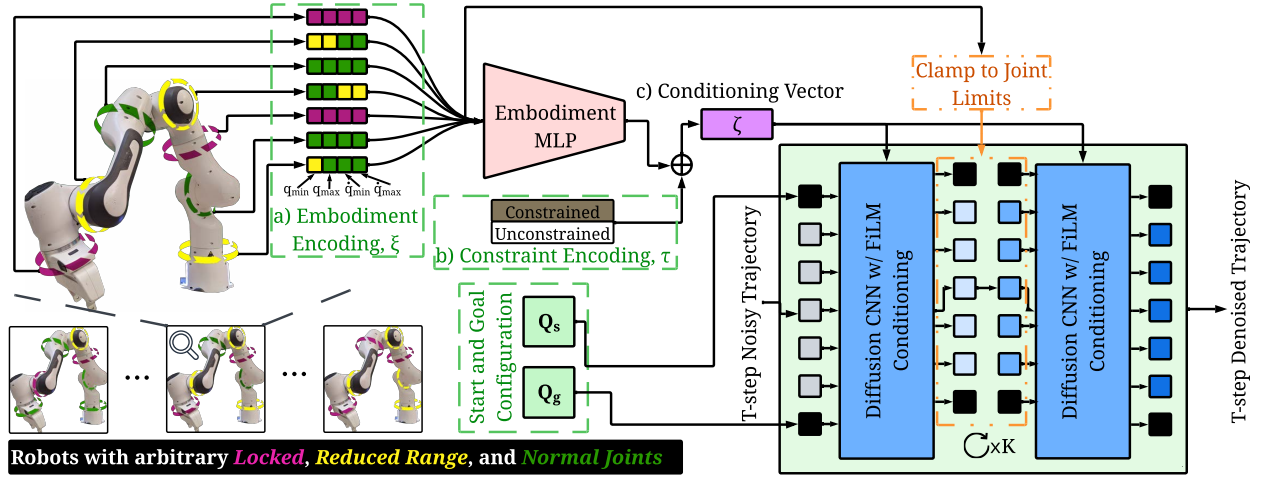


Figure 6.8: Overview of DEFT. (a) joint-level embodiment constraints are encoded into a structured representation capturing failure constrained joint position and velocity limits. This embodiment encoding is processed by a multilayer perceptron (MLP) and used to condition the diffusion model. (b) Task-specific constraints (e.g. unconstrained or constrained) are represented as one-hot vectors and concatenated with the embodiment encoding to constrain the generative process. Given these conditioning signals and start–goal joint configurations, the model generates feasible joint-space trajectories adapted explicitly to both the robot’s degraded embodiment and task requirements. At each step of the denoising process the predicted joint values are clamped to the failure-induced robot joint limits.

a fixed number of steps T . Unlike prior work that assumes a fixed set of failure configurations to control over, our method adapts online. It learns both the control behavior and the allowable motions from the current failure condition and the task constraint.

Embodiment Conditioning To inform the model of the robot’s failure constrained embodiment, we use an embodiment encoding vector ξ to condition the diffusion model. We model joint-level failures as per-joint constraints on the robot’s actuation capabilities. Each failure embodiment is represented as $\xi = [\mathbf{e}_q, \mathbf{e}_{\dot{q}}] \in \mathbb{R}^{4N}$, with $\mathbf{e}_{q,j} = [q_j^{\min}, q_j^{\max}]^\top$ and $\mathbf{e}_{\dot{q},j} = [\dot{q}_j^{\min}, \dot{q}_j^{\max}]^\top$ for all $j \in \{1, \dots, N\}$ such that $\mathbf{0} \in \mathbf{e}_{\dot{q},j}$. Failures reshape the robot’s feasible configuration space and task-space reachability and exist in a continuous space, allowing ξ to capture a wide spectrum of degradation severities, as shown in Figure 6.8. Let $\{\mathbf{q}_t\}_{t=1}^T$ denote a trajectory over T steps, where $\mathbf{q}_t \in \mathbb{R}^N$ is the joint configuration at time t , and $\dot{\mathbf{q}}_t$ its velocity. A configuration $\mathbf{q}_t$ satisfies the feasibility set $\mathcal{C}_t(\mathbf{q}_t, \xi)$ if:

$$\mathcal{C}_t(\mathbf{q}_t, \xi) = \left\{ \mathbf{q}_t \in \mathbb{R}^N \mid \begin{aligned} & q_{t,j} \in \mathbf{e}_{q,j}, \quad \dot{q}_{t,j} \in \mathbf{e}_{\dot{q},j} \\ & \forall j \in \{1, \dots, N\} \end{aligned} \right\}. \quad (6.2)$$

A trajectory is feasible if $\mathbf{q}_{1:T} \in \bigcap_{t=1}^T \mathcal{C}_t(\mathbf{q}_t, \xi)$. We condition on failures structurally: the model learns to generate motion plans that obey the constraints. To integrate embodiment information into the generative process, we pass ξ through an MLP to produce an embedding that modulates the diffusion model via FiLM. To ensure that the trajectory connects the desired start and goal states we apply start–goal inpainting, fixing $(\mathbf{q}_s, \mathbf{q}_g)$ in the input trajectory. In addition, we clamp the trajectory to the limits during both training and inference.

Constraint Conditioning We further condition the model on discrete task constraints using a one-hot encoding vector, τ . Let $\tau \in \{0, 1\}^K$ be a one-hot vector with exactly one nonzero entry, such that

$$\sum_{k=1}^K \tau_k = 1 \quad \text{and} \quad \tau_k \in \{0, 1\} \quad \forall k.$$

We concatenate τ with ξ to form the conditioning vector $\zeta = [\xi, \tau]^\top$, which is injected via FiLM. These constraints are not explicitly enforced during inference but must be learned implicitly through

the conditioning signal. The constraint encoding also informs the model of task-imposed end effector constraints, which it otherwise would not know while operating in joint space.

The formulation above enables zero-shot trajectory generation across diverse embodiments and task conditions. For example, when a shoulder joint becomes unusable, the robot can switch from lifting to pushing an object without task-specific retraining or policy switching.

We evaluate DEFT using two task constraints. *Unconstrained* motion corresponds to a feasible trajectory between two points and includes unconstrained manipulation where the object is rigidly secured to the end-effector. *Constrained* motion corresponds to planar, approximately straight-line motion with minimal end-effector orientation change—a spatial constraint for motion primitives such as pushing, pulling, or surface tracing. The valid trajectory sets are, respectively:

$$\mathcal{T}_{\text{Unconstrained}} = \left\{ \mathbf{q}_t \forall t \in \{1, \dots, T\} \mid \mathbf{q}_1 = \mathbf{q}_s, \mathbf{q}_T = \mathbf{q}_g, \right. \\ \left. \mathbf{q}_t \in \mathcal{C}_t(\mathbf{q}, \xi) \right\}. \quad (6.3)$$

$$\mathcal{T}_{\text{Constrained}} = \left\{ \mathbf{q}_t \forall t \in \{1, \dots, T\} \mid \mathbf{p}_t \in \mathcal{P} \wedge \right. \\ \left. \Delta p_t \leq \epsilon_p, \Delta R_t \leq \epsilon_R \wedge \mathbf{q}_t \in \mathcal{C}_t(\mathbf{q}, \xi) \right\}. \quad (6.4)$$

This allows DEFT to: 1) generalize to arbitrary failure configurations; 2) complete multiple manipulation primitives; and 3) handle multi-joint failures.

6.8.3 Results

We evaluate DEFT in two settings: (i) a **simulation analysis** and (ii) a **real-world comparison**. We ask if a single policy can remain **fail-active** under embodiment degradation.

Simulation Analysis We test three hypotheses to probe DEFT’s generalization to arbitrary joint failures (**H1** and **H2**) and to test whether DEFT produces both unconstrained and constrained motions (**H3**): **H.1** Does DEFT obey embodiment constraints of arbitrary actuation failures more often than classical planners? **H.2** Does DEFT **generalize** to out-of-distribution (OOD) failures? **H.3** Does DEFT adhere to constraint conditions more than approaches specialized to a single class of constraints?

We use the motion planning method from data generation as our baseline, with RRT for **unconstrained** motions and differential IK with optimization for **constrained** motions. These baselines do not handle both primitives. In contrast, DEFT generates both within a unified model. We evaluate on a fixed-base 7-DoF Franka Emika Panda arm. The study covers 4.7k failure conditions; of which 2.9k includes joint angle constraints (reduced and locked) and the rest are joint velocity constraints. For each condition, we randomly select 1–7 joints to be locked, range-limited, or velocity-restricted. Of these conditions, 22% are in-domain (ID) and 78% are out-of-domain (OOD). Each unique failure condition is tested 100 times, each with 100 start-goal pairs for a total of 4.7M trajectories evaluated.

Hypothesis Summary	Test Condition	Baseline	DEFT
H.1 Does DEFT handle arbitrary joint failures?	Angle Failure	48.2%	84.3%
	Velocity Failure	32.5%	70.8%
H.2 How well does DEFT handle unseen failures?	In Domain		78.33%
	Out of Domain		73.61%
H.3 How well does DEFT create multiple primitives?	Constrained	30.93%	46.42%
	Unconstrained	42.4%	99.58%

Table 6.1: Success rate of the Baseline methods and the DEFT method.

DEFT outperforms traditional planning baselines across various failure conditions (Table 6.1). DEFT achieves a success rate of 74.51%, while the baseline achieves only 36.85%—an absolute improvement of 37.66 percentage points. Near-parity between ID and OOD success indicates DEFT does not memorize seen cases. DEFT performs better for both primitives: 99.58% vs. 42.40% for unconstrained, and 46.42% vs. 30.93% for constrained motions.

Real-World Evaluation. We test DEFT on two long-horizon, multi-primitive tasks (Figure 6.9). The *Drawer Task* consists of five sequential phases: pulling the drawer open, pushing the object to a graspable position, grasping, placing the object inside, and pushing the drawer closed. The *Erasing Task* requires grasping an eraser from the table and sweeping it back and forth across a whiteboard to remove pre-written text. Together these tasks demand transitions between unconstrained free-space motion and constrained contact-rich interaction.

DEFT achieved perfect task completion on both real-world tasks (Table 6.2). The Optimiza-

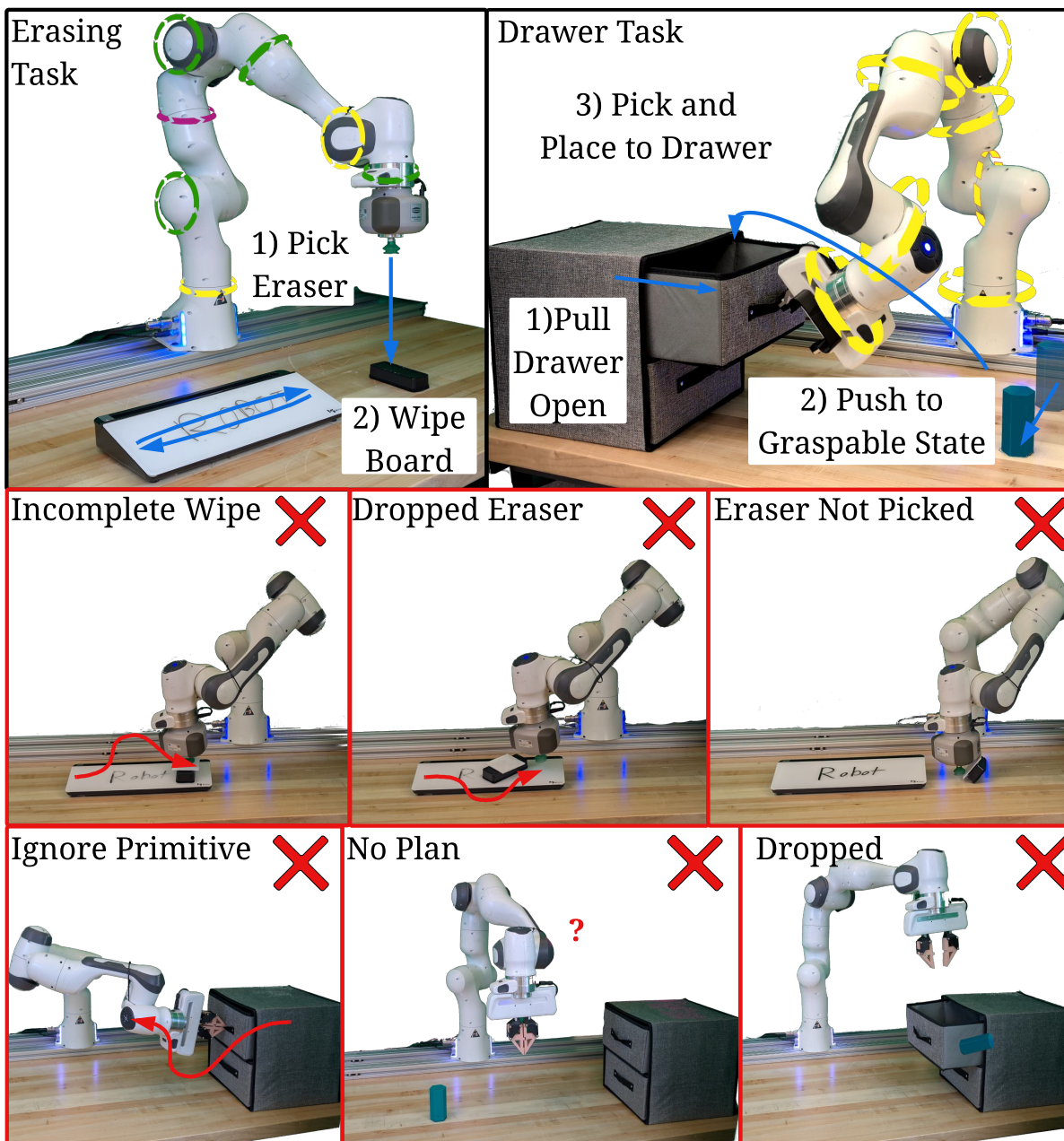


Figure 6.9: Left (Erasing): The robot grasps an eraser from the table and performs back-and-forth sweeps to remove text. Right (Drawer): The robot opens a drawer, pushes an object to a graspable pose, places it inside, and closes the drawer.

Table 6.2: Real-world task results over 10 runs.

Model	Task	Mean	Std.
DEFT	Drawer	1.00	0.00
DEFT	Erasing	1.00	0.00
Optimization	Drawer	0.00	0.00
Optimization	Erasing	0.35	0.32
DEFT-NoConditioning	Drawer	0.60	0.49
DEFT-NoConditioning	Erasing	0.93	0.12

tion baseline produced no executable plans on the drawer task. DEFT-NoConditioning succeeded in 6/10 drawer runs and 9.3/10 erasing runs, with failures attributable to missing embodiment conditioning and inpainting causing surface-constraint violations. These outcomes support our central hypothesis: embodiment and task conditioning, coupled with inpainting and clamping, yield a policy that remains fail-active under embodiment degradation.

Comparing against the sampling-based failure-centric approach from Chapter 4, DEFT removes the need for pre-computation of kinodynamic action maps and generalizes to previously unseen failure conditions at inference time—directly addressing the computational demand limitation identified in that work. The cost is the loss of formal completeness guarantees, which sampling-based kinodynamic methods provide under certain conditions.

6.9 Looking Forward

Together, the payload and embodiment results show that diffusion models can encode dynamic feasibility implicitly when the conditioning variable captures a physical factor that strongly shapes the feasible trajectory set. In the payload setting, a scalar mass input changes the distribution of torque-feasible motions; in the failure setting, an embodiment descriptor changes the distribution of executable motions under degraded actuation. This suggests a broader direction, but also exposes a limitation: the conditioning signal must be chosen with knowledge of the constraint variation being represented. Chapter 7 therefore asks whether constraint-aware generation can move beyond hand-selected physical parameters by conditioning on behavioral responses to diagnostic motion probes.

Chapter 7

Behavioral Anchor Conditioning for Constrained Trajectory Diffusion

Chapter 6 showed that conditional trajectory diffusion can internalize feasibility when the conditioning variable is a known physical parameter, such as payload mass or embodiment state, that biases the feasible motion distribution. This chapter removes the assumption that such a parameter is available. Instead, it represents an environment by the feasibility responses it induces on fixed diagnostic trajectory snippets, producing a compact behavioral signature that conditions diffusion toward near-feasible trajectory seeds for downstream optimization.

7.1 Introduction

Diffusion-based policies can learn expressive motion priors from demonstrations and can enable generalization across tasks [49, 22, 154]. In planning settings, generation must be environment-conditioned: the same start and goal can require very different trajectories depending on the constraints imposed by the scene or task [21, 128]. This makes the conditioning representation central to the success of the model, since the diffusion model can only generate appropriate trajectories if its condition captures the environment differences that matter for motion.

Constraint information, including kinematic and dynamic constraints derived from scene, robot embodiment, and task, enter the trajectory denoising model through a learned observation embedding via FiLM [111], concatenation [22], or cross-attention [57]. A common approach is to condition on high-dimensional perceptual inputs, such as RGB images [22, 134], depth maps [41], point clouds [154, 118], occupancy grids, or learned visual embeddings [135]. These representations

are intuitively appealing because they preserve scene information, moving from appearance-level information in RGB toward more explicit geometric information in depth or point clouds.

However, perceptual richness does not guarantee behavioral usefulness: a representation may preserve visual or geometric variation that is irrelevant to planning, while discarding subtle constraint-relevant variation. For trajectory diffusion, two environments should be considered equivalent if they induce the same or similar feasible trajectory distribution, even if they look different perceptually. Conversely, two perceptually similar environments should be represented differently if they force different trajectory behavior, for example because a small geometric or dynamic change makes one class of motions infeasible. This motivates behavioral equivalence (not perceptual similarity) as the right level of abstraction [29, 77]: the conditioning variable should collapse perceptual variance that does not affect motion, while preserving constraint-induced differences that do. This notion is constraint-agnostic: the relevant constraints may be kinematic, geometric, dynamic, torque-based, contact-based, safety-related, or task-specific; the representation only needs to encode how those constraints affect feasible motion.

This chapter introduces behavioral anchor conditioning, a method for constructing compact environment codes for trajectory diffusion from binary feasibility responses to diagnostic trajectory snippets. Figure 7.1 illustrates the anchor-selection process and the construction of binary environment signatures from diagnostic feasibility probes. Each snippet acts as a fixed motion probe, and each environment is represented by the vector of feasibility outcomes produced by evaluating those probes under the environment’s constraint model. The resulting signature is not intended to reconstruct the scene—it is designed to preserve distinctions that affect feasible motion. This chapter studies the collision-avoidance setting, where the feasibility oracle is a geometric collision checker. The broader formulation is compatible with other binary constraint oracles, but the quantitative results in this chapter focus on collision-aware trajectory generation.

7.2 Related Work

Diffusion models for robot trajectories. Diffusion models have recently become a

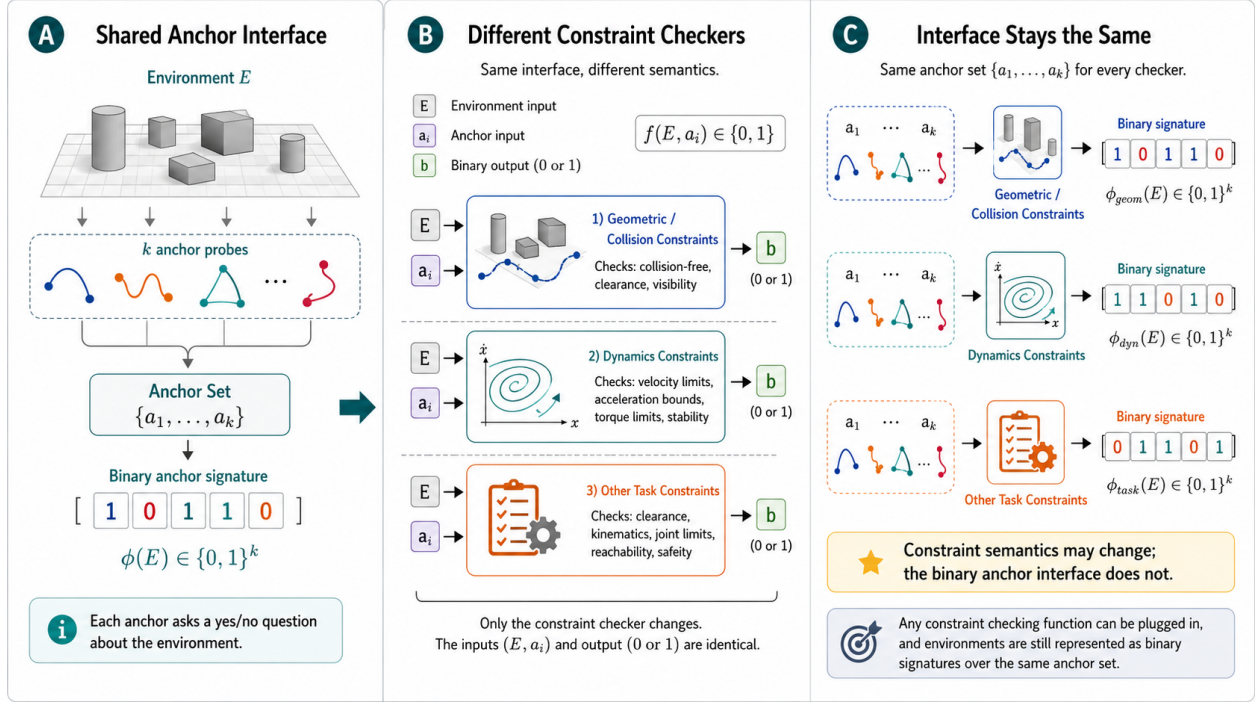


Figure 7.1: Diagnostic anchor selection. Candidate trajectory snippets are evaluated across environments using a binary feasibility oracle. Snippets that produce the same response in every environment provide little information, while snippets that split currently indistinguishable environments refine the behavioral signature. The selected anchors define a compact binary code for each environment, where environments with identical signatures are treated as equivalent by the downstream trajectory diffusion model.

powerful class of generative priors for robot decision making and trajectory generation [131, 39]. Diffuser formulates planning as iterative denoising over trajectories, showing that a diffusion model can represent flexible, multimodal behavior in sequential decision problems [49]. Diffusion Policy extends this idea to visuomotor control by learning a conditional denoising process over action sequences and executing the generated actions in a receding-horizon manner [22]. Subsequent work has improved the structure of the generated trajectories and the observation conditions used to guide denoising. ChainedDiffuser combines keypose prediction with local trajectory diffusion for manipulation [148], while 3D Diffuser Actor denoises end-effector pose trajectories conditioned on tokenized 3D scene features, language, and proprioception [57]. DP3 similarly argues that compact point-cloud observations provide a strong interface between 3D perception and diffusion-policy learning [154]. In motion planning, Motion Planning Diffusion learns trajectory priors that can seed optimization-based planners [21], with follow-up work guiding denoising through ensembles of collision costs [123] and conditioning on depth to seed GPU-based trajectory optimization [41]. These works demonstrate that diffusion models can represent expressive trajectory distributions, but they generally assume that the conditioning representation already exposes the environment differences that matter for feasible motion.

Environment representations for learning-based planning. A central question in learning-based planning is how to encode the scene so that a policy or planner can generalize across environments. Early neural motion-planning work encoded workspaces from point clouds or learned obstacle features, showing that learned representations can accelerate collision-free planning in unseen scenes [118, 135, 30]. More recent diffusion-based planners and manipulation policies have moved toward structured 3D, object-centric, and affordance-based conditions. PRESTO represents the environment using sparse key configurations that approximate configuration-space obstacles for diffusion-based trajectory seeding [128]. Lan-o3dp conditions a diffusion policy on language-selected, task-relevant object point clouds and uses denoising-time cost guidance for collision avoidance [75]. SPOT represents manipulation tasks through object-centric $SE(3)$ pose trajectories and uses these trajectories to condition a diffusion policy [40]. AffordDP further conditions diffusion

policies on transferable affordances, represented through 3D contact points and post-contact trajectories, to improve generalization to novel object categories [147]. These approaches reduce dependence on raw appearance by introducing geometric, object-level, or affordance-level structure. However, their abstractions are still primarily organized around what is perceived in the scene, rather than around which motions the scene permits or forbids.

Behaviorally aligned abstraction. Our work is closest in spirit to representation-learning approaches that define abstraction by behavioral relevance rather than perceptual similarity [29, 77]. DeepMDP learns latent models whose representations preserve reward and transition structure [33], and deep bisimulation methods learn visual-control representations that discard task-irrelevant variation while preserving control-relevant dynamics [156]. Language-guided state abstraction similarly seeks task-aligned state descriptions that suppress irrelevant observation dimensions for imitation learning [109]. We adopt the same high-level principle, but instantiate it for environment-conditioned trajectory diffusion. Instead of asking a learned visual embedding to infer which perceptual differences affect planning, we define an environment through its binary responses to a fixed set of diagnostic trajectory snippets under the active constraint model. The resulting feasibility signature acts as a compact behavioral code: environments with similar feasible trajectory distributions can map to similar signatures even when they differ visually, while small perceptual changes remain separated whenever they change which trajectories are feasible. This positions our method between structured conditioning for diffusion policies and control-relevant abstraction, with the key distinction that the conditioning variable is built from explicit constraint responses rather than from perception alone.

7.3 Methods

This section introduces behavioral anchor conditioning for trajectory diffusion. The method constructs a compact environment representation by evaluating selected diagnostic trajectory snippets under an environment-specific feasibility oracle. The resulting binary signature is then used as the environment-conditioning input to the trajectory denoising model.

7.3.1 Problem Setup

Let $\mathcal{E} = \{E_1, \dots, E_N\}$ denote a set of training environments. Each environment defines a constraint model under which robot trajectories can be evaluated. In this chapter’s experiments, these constraints are geometric collision constraints, but the formulation allows any binary feasibility criterion. We assume access to a constraint oracle

$$f(E, \tau) \in \{0, 1\}, \quad (7.1)$$

where $f(E, \tau) = 1$ indicates that trajectory τ satisfies the constraints of environment E , and $f(E, \tau) = 0$ indicates that at least one constraint is violated.

A trajectory is represented in robot configuration space as

$$\tau = (q_0, q_1, \dots, q_T), \quad q_t \in \mathbb{R}^d, \quad (7.2)$$

where d is the number of robot degrees of freedom. The goal is to learn a conditional generative model that samples trajectories from

$$p(\tau \mid q_{\text{start}}, q_{\text{goal}}, \phi(E)), \quad (7.3)$$

where q_{start} and q_{goal} define the planning query and $\phi(E)$ is the environment representation. The central question is how to construct $\phi(E)$ so that it preserves the environment differences that matter for feasible motion.

7.3.2 Conditional Trajectory Diffusion

We use a denoising diffusion model over trajectories. Given a clean trajectory τ_0 , the forward process corrupts it with Gaussian noise:

$$q(\tau_t \mid \tau_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t} \tau_0, (1 - \bar{\alpha}_t)I), \quad (7.4)$$

where $t \in \{1, \dots, T_d\}$ is the diffusion timestep and $\bar{\alpha}_t$ is determined by the noise schedule. A neural denoiser ϵ_θ is trained to predict the added noise conditioned on the noisy trajectory, diffusion

timestep, planning query, and environment code:

$$\epsilon_\theta = \epsilon_\theta(\tau_t, t, q_{\text{start}}, q_{\text{goal}}, \phi(E)). \quad (7.5)$$

The training objective is the standard noise-prediction loss

$$\mathcal{L}(\theta) = \mathbb{E}_{\tau_0, t, \epsilon} [\|\epsilon - \epsilon_\theta(\tau_t, t, q_{\text{start}}, q_{\text{goal}}, \phi(E))\|_2^2], \quad (7.6)$$

where $\epsilon \sim \mathcal{N}(0, I)$. During inference, trajectories are generated by iterative denoising from random noise while conditioning on the start state, goal state, and environment representation.

This formulation makes the choice of $\phi(E)$ critical. If two environments require different motion behavior but receive similar conditioning input, the denoising model has insufficient information to generate environment-appropriate trajectories. Conversely, if the representation preserves irrelevant perceptual variation, the model must learn to ignore it. Behavioral anchor conditioning addresses this by constructing $\phi(E)$ from feasibility responses to diagnostic motion probes.

7.3.3 Diagnostic Anchor Signatures

We approximate behavioral equivalence using a finite set of diagnostic trajectory snippets. A snippet is a short contiguous segment extracted from a demonstrated or planned trajectory. Given a trajectory $\tau = (q_0, \dots, q_T)$, a snippet of horizon H starting at index r is

$$s = (q_r, q_{r+1}, \dots, q_{r+H-1}). \quad (7.7)$$

Let $\mathcal{S} = \{s_1, \dots, s_M\}$ denote the set of candidate snippets extracted from the training trajectories. Each candidate snippet acts as a constraint probe. Evaluating snippet s_i in environment E_j gives a binary response

$$B_{ij} = f(E_j, s_i), \quad B \in \{0, 1\}^{M \times N}. \quad (7.8)$$

Each row of B describes the feasibility pattern of one snippet across environments, and each column describes how one environment responds to all candidate snippets.

Given an anchor set $A = \{a_1, \dots, a_k\} \subseteq \mathcal{S}$ of selected snippets, environment E_j is encoded by its feasibility responses to those anchors (i.e., the corresponding entries of B):

$$\phi_A(E_j) = [f(E_j, a_1), f(E_j, a_2), \dots, f(E_j, a_k)] \in \{0, 1\}^k. \quad (7.9)$$

The signature $\phi_A(E_j)$ is a behavioral code rather than a reconstruction of the scene. It records how the environment responds to selected motion probes, preserving distinctions that affect feasibility while discarding variation that is irrelevant to the chosen constraint model.

7.3.4 Entropy-Based Anchor Selection

The goal of anchor selection is to choose snippets whose feasibility responses distinguish environments according to their constraint-induced behavior. A snippet that is feasible in every environment, or infeasible in every environment, provides no information because it does not separate environments. Informative snippets are those that split groups of environments that are currently indistinguishable under the anchors selected so far.

Let A_m denote the anchors selected after m greedy iterations, and let

$$C_m(E_j) = \phi_{A_m}(E_j) \quad (7.10)$$

be the current signature assigned to environment E_j . These signatures induce a partition of the training environments into groups

$$\mathcal{G}_m = \{G_c : c \in \text{range}(C_m)\}, \quad G_c = \{E_j \in \mathcal{E} : C_m(E_j) = c\}. \quad (7.11)$$

Within each group G_c , all environments are indistinguishable under the selected anchors.

For an unselected candidate snippet $s_i \notin A_m$, define the empirical probability that the snippet is feasible within group G_c :

$$p_{i,c} = \frac{1}{|G_c|} \sum_{E_j \in G_c} B_{ij}. \quad (7.12)$$

The informativeness of the candidate within that group is measured by binary entropy,

$$h(p_{i,c}) = -p_{i,c} \log_2 p_{i,c} - (1 - p_{i,c}) \log_2 (1 - p_{i,c}), \quad (7.13)$$

with $h(0) = h(1) = 0$. The conditional entropy gain of candidate snippet s_i is

$$\Delta H(s_i \mid A_m) = \sum_{G_c \in \mathcal{G}_m} \frac{|G_c|}{N} h(p_{i,c}). \quad (7.14)$$

This score is high when a snippet produces balanced feasibility outcomes inside large groups of environments that are not yet distinguished. At each iteration, the next anchor is selected as

$$a_{m+1} = \arg \max_{s_i \notin A_m} \Delta H(s_i \mid A_m), \quad (7.15)$$

and the anchor set is updated:

$$A_{m+1} = A_m \cup \{a_{m+1}\}. \quad (7.16)$$

The procedure terminates when k anchors have been selected, when the maximum entropy gain falls below a threshold, or when the resulting signatures are sufficiently discriminative for the downstream task.

This greedy objective can be interpreted as selecting the next feasibility query that provides the most information about the environment identity conditioned on the queries already selected. More importantly for planning, it selects probes that expose constraint-relevant variation not captured by previous anchors.

7.3.5 Behavioral Equivalence and Encoding New Environments

The selected anchor set defines a finite approximation to behavioral equivalence. Two environments are equivalent under anchor set A if they produce the same constraint-response signature:

$$E_i \sim_A E_j \iff \phi_A(E_i) = \phi_A(E_j). \quad (7.17)$$

This equivalence relation does not require the environments to be visually or geometrically identical. It only requires them to respond identically to the selected diagnostic probes. Conversely, visually similar environments can receive different signatures if they permit different anchor snippets.

After the anchors have been selected, a new environment E_{new} is encoded by evaluating the same anchors under its constraint oracle:

$$\phi_A(E_{\text{new}}) = [f(E_{\text{new}}, a_1), \dots, f(E_{\text{new}}, a_k)]. \quad (7.18)$$

The resulting binary vector has the same coordinate semantics as the training signatures and can be used directly as the environment-conditioning input for the diffusion model. For the collision-avoidance experiments in this chapter, each bit records whether the corresponding anchor snippet is collision-free in the environment. More generally, the same interface can be used with any constraint oracle that maps trajectory snippets to binary feasibility outcomes.

7.4 Evaluation

This section evaluates whether diagnostic anchor conditioning provides a more task-aligned environment representation for collision-aware trajectory diffusion. The empirical study focuses on geometric collision constraints, where the feasibility oracle evaluates whether trajectory snippets and generated trajectories remain collision-free in a given environment. Although the anchor construction is formulated for arbitrary binary constraint oracles, the quantitative results in this chapter are limited to the collision-avoidance setting. The broader constraint-agnostic interpretation is discussed separately as an extension of the interface.

The main question is whether conditioning on binary anchor signatures improves the quality of generated trajectory initializations compared with perceptual, hand-designed, environment-agnostic, and random conditioning baselines. We evaluate this question using collision-free rate, collision fraction, collision steps, and worst-case clearance metrics across three training seeds.

7.4.1 Experimental Setup

We compare trajectory diffusion models that share the same denoising architecture and training procedure, but differ in the representation used for environment conditioning. The proposed method, denoted *Anchor Traj*, conditions the model on binary constraint-response signatures induced by the selected diagnostic anchor snippets. These anchors are chosen using the entropy-based procedure described in Section 7.3, which greedily selects snippets whose feasibility outcomes maximally refine the current partition of training environments.

We compare against four conditioning baselines. *Point Cloud* conditions the diffusion model

on a point-cloud representation of the environment using a standard PointNet++ encoder [117]. *Key Config* conditions on key robot configurations produced by PRESTO [128]. *Start-Goal* provides only the planning query and does not condition on environment information. *Random* assigns random vectors as environment codes, testing whether performance gains arise merely from providing distinct identifiers rather than behaviorally meaningful structure. We also report *VAMP* [140] as an oracle-like upper bound. VAMP is not intended as a directly comparable learned conditioning method; rather, it indicates the approximate performance ceiling achievable by the underlying planner or trajectory source.

All learned models are trained using three random training seeds. Error bars indicate variation across seeds. In the plots, the notation “ $n/3$ ” indicates that a baseline outperformed Anchor Traj on n out of three seeds for the corresponding metric.

7.4.2 Metrics

We evaluate generated trajectories using four collision-related metrics. The *collision-free rate* measures the fraction of generated trajectories that satisfy collision constraints over the full trajectory. The *collision steps* metric counts the number of states along a trajectory that are in collision. The *collision fraction* normalizes this quantity by trajectory length, measuring the proportion of the trajectory that violates collision constraints. Finally, we compute worst-case clearance metrics over trajectories. The *mean worst-case clearance* measures the average minimum clearance along generated trajectories, while the *tail worst-case clearance* reports the fifth percentile of worst-case clearance, capturing rare but severe constraint violations.

7.4.3 Collision-Avoidance Results

Figure 7.2 summarizes the collision-avoidance results across the learned conditioning methods and the VAMP reference ceiling. The main result is that anchor conditioning improves the quality of generated trajectory initializations. While the collision-free rate provides a useful binary success measure, the clearest advantage of Anchor Traj appears in the violation-severity metrics: collision

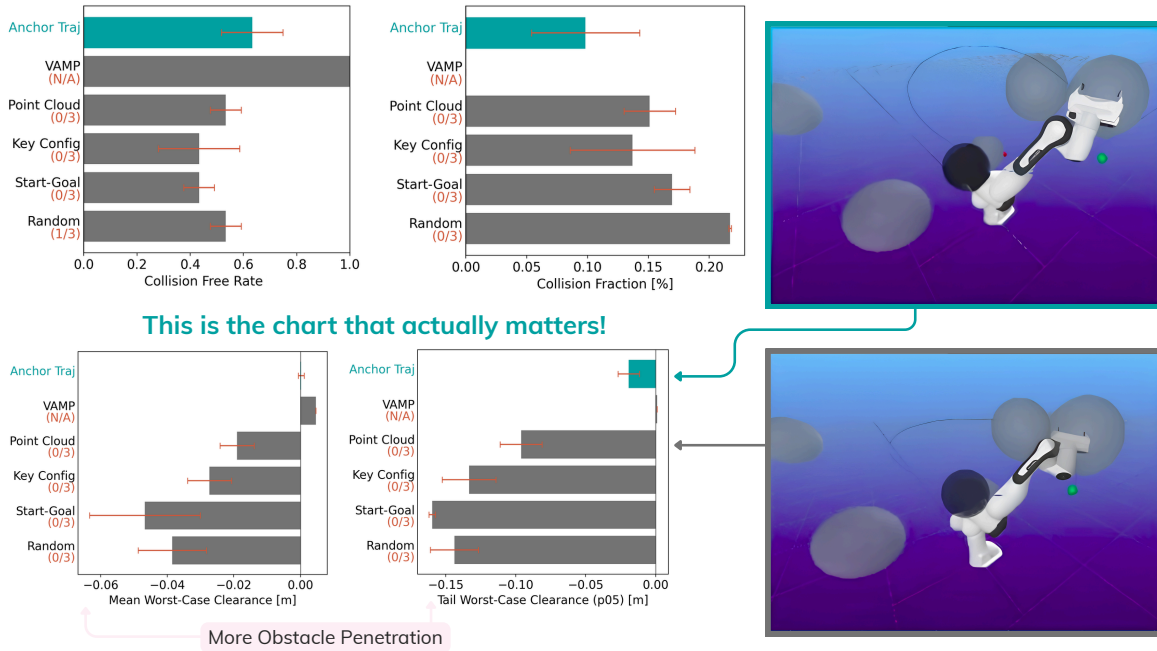


Figure 7.2: Collision-avoidance evaluation across three training seeds. Anchor Traj denotes the proposed diagnostic anchor conditioning method. VAMP is shown as an oracle-like upper bound rather than a learned conditioning baseline. Point Cloud uses a PointNet++ environment encoder, Key Config uses PRESTO key configurations, Start-Goal does not condition on environment information, and Random uses random environment codes. Anchor Traj provides the strongest overall learned-model performance, with the clearest improvements appearing in collision-severity and worst-case-clearance metrics.

fraction, collision steps, and worst-case clearance. These metrics are especially important in this setting because the diffusion model is used to generate trajectory candidates that may be passed to a downstream planner or optimizer. A trajectory that is not fully feasible but has fewer and less severe collisions is a better initialization than one that penetrates obstacles deeply or remains in collision for a large portion of the rollout.

7.4.3.1 Collision-Free Rate

Anchor Traj achieves the strongest collision-free rate among the learned diffusion-based conditioning methods. VAMP remains the reference ceiling, as expected, because it represents the performance of the underlying trajectory source rather than a learned conditioning baseline. Among the learned methods, Anchor Traj outperforms Point Cloud, Key Config, Start-Goal, and Random conditioning on average. The comparison with Start-Goal shows that environment information is necessary: conditioning only on the planning query is insufficient when the same start and goal can require different motions in different environments. The comparison with Point Cloud shows that providing geometric information alone does not guarantee the best planning behavior, since the diffusion model must still learn which geometric distinctions are relevant for feasibility. The comparison with Random is also informative. Random environment codes can sometimes help because they give the model a way to distinguish environments, but they do not encode how each environment constrains motion. Anchor signatures provide a more structured conditioning signal because each bit corresponds to the feasibility response of a diagnostic trajectory probe.

At the same time, the collision-free-rate improvement should not be overstated. The margin over some baselines is moderate, and Random conditioning performs competitively in some seeds. This suggests that binary success rate alone does not fully capture the benefit of the proposed representation. For that reason, the remaining metrics are important for understanding how anchor conditioning changes the quality of the generated trajectories.

7.4.3.2 Collision Fraction and Collision Steps

Collision fraction and collision steps measure the severity of constraint violation rather than only whether a trajectory is completely collision-free. These metrics show a clearer advantage for Anchor Traj. Compared with the other learned conditioning methods, Anchor Traj produces trajectories with a smaller fraction of waypoints in collision and fewer collision steps along the trajectory. This result is important because diffusion-generated trajectories are often used as initial guesses. A failed trajectory with a small number of colliding waypoints is much closer to feasibility than a failed trajectory that remains in collision for a large part of its duration. In this sense, Anchor Traj improves not only the probability of producing a fully feasible sample, but also the quality of infeasible samples. The generated trajectories are shifted closer to the feasible set, which should reduce the amount of correction required from a downstream optimizer.

This supports the central motivation for behavioral anchor conditioning. The anchor signature does not attempt to reconstruct the full scene geometry. Instead, it exposes how the environment responds to representative motion probes. The resulting code appears to provide the denoising model with information that is directly relevant to avoiding constraint violations.

7.4.3.3 Worst-Case Clearance

Worst-case clearance provides another measure of trajectory quality. It evaluates the minimum clearance along a trajectory, making it sensitive to rare but severe obstacle penetrations. Negative clearance values indicate penetration into obstacles, so values closer to zero from below indicate less severe collision when a trajectory is not fully collision-free.

Anchor Traj achieves the best worst-case clearance behavior among the learned conditioning methods. Both the mean worst-case clearance and the lower-tail clearance improve relative to Point Cloud, Key Config, Start-Goal, and Random conditioning. The tail metric is particularly important because it captures severe failure cases. A method that occasionally produces deeply penetrating trajectories may be difficult for a downstream optimizer to repair, even if its average

performance appears reasonable.

The clearance results therefore strengthen the conclusion from the collision-fraction metrics. Anchor conditioning does not merely improve binary feasibility. It also reduces the severity of the remaining failures. This is the strongest empirical evidence that the proposed representation provides better trajectory initializations for constrained generation.

7.4.4 Discussion of Results

Overall, the collision-avoidance results support the hypothesis that behaviorally defined environment codes can be more useful for trajectory diffusion than purely perceptual or hand-designed conditioning inputs. Point Cloud conditioning provides direct geometric information, but the model must learn from data which geometric features matter for feasible motion. Key Config conditioning provides a more structured representation, but it may not preserve all distinctions that affect the generated trajectory distribution. Start-Goal conditioning confirms that the planning query alone is insufficient, and Random conditioning shows that simply assigning different identifiers to environments does not consistently capture the structure needed for feasibility. The main advantage of Anchor Traj is that its conditioning signal is constructed from constraint responses. Each bit in the signature records whether a selected diagnostic trajectory snippet is feasible in the environment. As a result, environments are distinguished when they impose different feasibility outcomes on representative motions. This gives the diffusion model a low-dimensional input that is aligned with the downstream planning objective.

The most important interpretation of the results is that Anchor Traj produces better initial guesses. Collision-free rate is useful, but it is a strict binary metric. Collision fraction, collision steps, and worst-case clearance show how close generated trajectories are to feasibility when they fail. On these measures, anchor conditioning provides a clearer improvement. This matters because trajectory diffusion is often used not as a final planner by itself, but as a generator of candidate trajectories that can be refined, optimized, or checked by a downstream planning system. These results should be interpreted within the scope of the experiment. The empirical evaluation in

this chapter validates anchor conditioning for geometric collision avoidance. The formulation is compatible with other binary constraint oracles, but the experiments here do not yet demonstrate improved performance for dynamic, torque, contact, or task-specific constraints.

7.5 Discussion

7.5.1 Extension to Other Constraint Oracles

The anchor-conditioning interface is not specific to collision checking. The only requirement is access to a binary feasibility oracle that evaluates whether a trajectory snippet satisfies the constraints associated with an environment. In the collision-avoidance experiments above, this oracle checks geometric validity. However, the same construction could be applied with a different oracle that incorporates dynamic feasibility, torque limits, contact constraints, or task-specific rules.

Under this broader view, the semantics of the constraint checker may change, but the representation interface remains the same. A fixed set of anchors is evaluated in each environment, and the resulting binary responses form a signature used to condition the trajectory diffusion model. For example, a geometric oracle would produce a signature indicating which anchor snippets are collision-free, while a dynamics-aware oracle could produce a signature indicating which snippets satisfy both collision and torque constraints.

The experiments in this chapter validate the approach only for geometric collision constraints. Extending the empirical evaluation to richer constraint models remains future work. Nevertheless, the formulation suggests that diagnostic anchor conditioning can be understood as a general method for converting arbitrary feasibility queries into compact behavioral environment codes.

7.5.2 Limitations

First, the method assumes access to a constraint oracle that can evaluate anchor snippets in each environment. In the collision-avoidance setting, this oracle is a collision checker. In other settings, the oracle may require dynamics simulation, torque computation, contact reasoning, or

task-level feasibility checks. The usefulness of the method therefore depends on whether these feasibility queries are cheaper or easier than solving the full planning problem.

Second, the binary signature discards information about the reason and severity of failure. A snippet that barely violates a clearance threshold and a snippet that deeply penetrates an obstacle both receive the same infeasible label. This binary abstraction is useful because it provides a simple and constraint-agnostic interface, but it may lose information that could help the diffusion model distinguish mild violations from severe ones. Future versions of the method could use multi-valued or continuous anchor responses, such as clearance, penetration depth, or torque margin.

Third, the representation is limited by the selected anchor set. Two environments with identical anchor signatures are treated as equivalent by the downstream diffusion model, even if they differ in ways that are not tested by the selected probes. Increasing the number of anchors can reduce this ambiguity, but it also increases the dimensionality of the conditioning vector and the cost of encoding new environments. The quality of the representation therefore depends on whether the candidate snippets contain probes that expose the constraint variations relevant to the test environments.

Finally, the chapter argues that better collision fraction and clearance should make generated trajectories better initializations for downstream optimization. This interpretation is supported by the fact that Anchor Traj produces trajectories closer to feasibility, but a complete evaluation should directly measure downstream planner performance, including optimization iterations, repair success rate, and total planning time.

Behavioral anchor conditioning completes the thesis progression from online feasibility evaluation, to reusable kinodynamic motion structure, to learned conditional representations. Unlike payload or embodiment conditioning, the conditioning signal here is not a hand-selected physical parameter but a probe-induced signature of how constraints affect motion. The experiments validate this idea for collision-aware trajectory generation, where anchor-conditioned samples are closer to feasibility than the baselines; extending the same representation to dynamic, torque, contact, and task-specific oracles remains the next step beyond this thesis.

Chapter 8

Discussion and Future Directions

This thesis expands robotic manipulation beyond geometric reasoning by developing methods that model and plan over dynamic and non-prehensile interactions, enabling new capabilities governed by object inertial properties, impulsive contact, and motion-induced forces. Across three parts—online kinodynamic search, amortized reusable motion structure, and learned conditional generative models—the work studies where knowledge of dynamic feasibility should reside in a manipulation system, what form it should take, and how that choice shapes robot motions. Each part develops a different mechanism for placing constraint knowledge: an online planner that evaluates dynamics through simulation, a precomputed graph that amortizes dynamic structure across queries, and a learned distribution that absorbs feasibility into the generation process itself.

This chapter first revisits the main contributions and then distills the broader lessons that cut across them. These lessons fall into three classes—constraints and system design, multimodal capabilities beyond geometric skills, and the scope and limits of statistical learning—each paired with the open questions it raises for the next generation of dynamics-aware manipulation systems.

8.1 Summary of Contributions and Key Takeaways

8.1.1 Poking as a Complementary Dynamic Manipulation Primitive

Chapter 3 introduced poking as a dynamic, non-prehensile manipulation primitive for object rearrangement in cluttered environments. Where pushing relies on sustained contact and therefore couples object displacement to end-effector motion, poking uses brief impulsive contacts to transfer

momentum to the object. This distinction changes the geometry of the planning problem: a short robot motion can induce a comparatively large object motion, allowing the planner to reach object configurations that are inaccessible to pushing or grasping alone.

PokeRRT operationalizes this idea through a closed-loop sampling-based kinodynamic planner that uses physics simulation as a forward model. Rather than requiring an analytical model of impulsive contact, the planner samples candidate poking actions, propagates their effects through simulation, and replans as the observed object state evolves. Across six experimental scenarios, this approach achieved higher success rates and shorter task completion times than pushing and grasping baselines, particularly in settings where grasping was geometrically infeasible or sustained-contact pushing was constrained by clutter and reachability.

The central contribution of this work is not only the poking primitive itself, but the demonstration that simulation-in-the-loop kinodynamic planning can expose useful manipulation behaviors that are difficult to capture with purely geometric abstractions. Poking illustrates the broader premise of the thesis: dynamic interaction expands the set of feasible manipulation strategies when the planner is allowed to reason about state transitions, rather than only static configurations.

8.1.2 Multi-Query Planning via Lazy Edge Bundles

Chapter 4 addressed a practical limitation of simulation-based kinodynamic planning: dynamic feasibility is expensive to evaluate repeatedly. In single-query planners, each new problem typically requires propagating candidate controls, checking collisions, and constructing feasible motion structure from scratch. This is acceptable for isolated demonstrations, but limits deployment in settings where a robot must solve many related planning problems in the same workspace.

LazyBoE shifts part of this computation offline by precomputing control-parameterized edge bundles over discretized state and control spaces. These bundles encode reusable local motion structure, while lazy propagation and collision checking defer expensive evaluations until they are needed for a candidate solution. The resulting planner preserves the flexibility of kinodynamic search while amortizing repeated dynamics computation across queries.

The broader contribution is a change in how dynamic feasibility is represented. Rather than treating every planning query as an independent search through continuous dynamics, *LazyBoE* stores reusable structure that can be selectively activated at planning time. This makes multi-query kinodynamic planning more practical and establishes an intermediate point in the thesis arc: dynamics are no longer only evaluated online through search, but are partially represented, reused, and amortized across related problems.

8.1.3 Conditional Trajectory Diffusion for Dynamic Constraint Satisfaction

Chapters 5 and 6 shifted from explicit kinodynamic search to learned generative representations of dynamic feasibility. Rather than constructing a trajectory online through repeated propagation, collision checking, or constraint evaluation, the approach reframes motion generation as conditional sampling: a model learns the distribution of trajectories that satisfy dynamic constraints under specified task conditions, then samples from that distribution at inference time.

The primary instantiation of this idea is payload-conditioned trajectory diffusion. As payload mass increases, the feasible motion set changes because joint torque, acceleration, and inertial constraints become more restrictive. Instead of enforcing these constraints through runtime checking, the diffusion model conditions directly on payload mass and learns to generate joint-space trajectories whose positions, velocities, and accelerations are consistent with the corresponding dynamic limits. The result is near-constant-time trajectory generation for super-nominal payload manipulation, maintaining workspace accessibility even at three times the robot’s payload capacity.

The case study on failure-induced embodiments demonstrates that the same conditioning principle extends beyond payload variation. In *DEFT*, actuation failures are represented as changes in embodiment, and the diffusion model conditions on the resulting failure configuration when generating trajectories. This allows a single model to adapt across multiple joint-failure conditions and motion primitives, treating failure not as an exception requiring a separate controller, but as another structured input to the generative process.

Together, these results show that dynamic constraints can be encoded implicitly through

conditioning when the conditioning signal captures the physical factors that shape feasibility. This marks a conceptual transition in the thesis: dynamic feasibility is no longer evaluated only by a planner or simulator, but represented as a learned conditional distribution over trajectories.

8.1.4 Behavioral Anchor Conditioning for Task-Agnostic Trajectory Seeding

Chapter 7 extended the generative view of dynamic feasibility to constrained trajectory optimization. While trajectory optimization provides a powerful framework for enforcing complex kinematic and dynamic constraints, its performance depends heavily on initialization. Poor seeds can cause solvers to converge slowly, become trapped in infeasible regions, or fail entirely, especially when the feasible set is narrow or shaped by higher-order dynamic constraints.

The proposed approach uses diffusion not as a final trajectory generator, but as an initialization mechanism. Instead of conditioning on a task-specific scalar such as payload mass, an embodiment descriptor such as a failure configuration, or a perceptual encoding of the scene, the model conditions on a behavioral signature of the environment: the binary feasibility responses of a selected set of diagnostic trajectory snippets evaluated under the environment’s constraint oracle. These signatures provide a constraint-agnostic representation of how the environment shapes feasible motion, collapsing perceptual variation that does not affect planning while preserving the distinctions that do, and allowing the model to generate trajectory seeds that begin near feasible regions of the search space.

By translating arbitrary constraints into anchor-signature conditioning, this general interface between learned generation and constrained optimization reduces the amount of task-specific engineering required to initialize optimization-based planners. In collision-aware trajectory generation, anchor conditioning produces seeds closer to feasibility than perceptual, hand-designed, environment-agnostic, and random conditioning baselines, while preserving the ability of the downstream optimizer to enforce constraints explicitly.

In the broader arc of the thesis, behavioral anchor conditioning represents the most abstract representation of dynamic feasibility developed here. Where earlier chapters evaluate dynamics

online, reuse precomputed motion structure, or learn trajectory distributions conditioned on known physical parameters, this approach defines the conditioning signal itself by probing the feasible set. It suggests a general strategy for combining learned priors with optimization: use learning to move close to the constraint manifold, then use optimization to refine and certify the final motion.

8.2 Lessons and Open Questions

The contributions summarized above use progressively more compute-efficient mechanisms for motion generation—online kinodynamic search, reusable precomputed motion structure, and learned conditional generation. Across the thesis, the main lessons fall into three classes. The first concerns how constraints shape system design. The second concerns the capabilities that dynamic, non-prehensile interaction unlocks beyond geometric skills. The third concerns the scope and limits of statistical learning as a representation of dynamic feasibility. Each class is paired below with the open questions it raises.

8.2.1 Constraints and System Design

A recurring result across the thesis is that kinematic and dynamic feasibility are coupled. A robot motion may be collision-free, reachable, and valid in configuration space, yet still be infeasible once timing, actuation, payload, contact, or object dynamics are considered. The same nominal path can become invalid when the robot carries a heavy object, operates near torque limits, transfers momentum through impulsive contact, or induces motion in a coupled system such as a liquid-filled container. Conversely, behaviors that appear unavailable under a purely geometric abstraction may become feasible when the planner reasons over dynamic state transitions. The results in Chapters 3, 4 and 6 show both sides of this coupling: ignoring dynamics can cause plans to fail, but exploiting dynamics can expand the set of useful manipulation strategies.

This coupling weakens the standard separation between geometric planning, time parameterization, control, and feasibility checking. That decomposition is effective when dynamic constraints are mild corrections to an otherwise geometric problem. It is less effective when the dynamic con-

straint determines the task outcome itself. In poking, the relevant transition is produced by impulse and object inertia. In liquid transport, the admissibility of a motion depends on motion-induced fluid response. In super-nominal payload manipulation, feasible trajectories are shaped by torque, acceleration, and traversal-time limits. In these settings, dynamic feasibility must appear inside motion generation, either through simulation-in-the-loop search, reusable kinodynamic structure, learned feasibility models, or conditional generative priors.

This also makes constraint representation a system-level design decision. A binary feasibility checker, a differentiable constraint, a physics simulator, a precomputed graph, and a dataset of feasible trajectories all expose different algorithmic interfaces. The choice determines whether the downstream method can search, optimize, sample, certify, or only reject candidate motions. It also affects modularity, inference time, retraining requirements, transfer across domains, and the degree to which feasibility is explicitly enforced or only statistically predicted. Thus, the central design question is not only which constraints a manipulation system should respect, but where those constraints should reside and what computational form they should take.

This perspective raises two open questions for future systems. The first is whether tightly coupled kinodynamic planning can become practical at the scale and reliability now associated with geometric motion planning. Accelerated collision checking and parallel geometric planning have changed the cost structure of configuration-space search, but dynamic feasibility has a different computational profile: it may require simulation, contact propagation, control rollout, and history-dependent constraint evaluation inside the planning loop. The open question is therefore not only whether simulation can be made faster, but what planning algorithms are best suited to exploit parallel dynamic evaluation.

The second open question concerns the role of learning in system design. The results of this thesis suggest that learning is most useful when it reshapes an interface: selecting constraints, predicting feasible modes, initializing optimization, estimating dynamic admissibility, or generating candidate motions that a structured planner can refine. The open question is where learned adaptation provides the largest gains, and how structured planners should be designed so that they can

consume learned information without losing reliability, interpretability, or constraint awareness.

8.2.2 Multimodal Capabilities Beyond Geometric Skills

The contributions in this thesis suggest two motivations for moving beyond purely geometric manipulation skills. The first is efficiency. Dynamic interactions can sometimes replace slower geometric solutions even when both are feasible. Poking, throwing, sliding, scooping, and related behaviors can increase throughput when precise grasping is unnecessary, because they allow the robot to exploit contact, momentum, and object motion rather than rigidly controlling the object throughout the entire trajectory. In Chapter 3, for example, poking enables object rearrangement in clutter by using brief impulsive contacts to produce object motions that would require longer end-effector motions under sustained pushing or grasping.

The second reason is coverage. Dynamic skills can make tasks feasible that are difficult or impossible under a purely prehensile or geometric abstraction. Manipulation under high payloads, joint failures, limited grasp access, cluttered reachability, sustained contact, and coupled robot-object dynamics all require reasoning about more than collision-free motion. The results in Chapters 3, 4 and 6 show that when the planner or generator accounts for these constraints, the robot can demonstrate capabilities that would otherwise be excluded from the feasible motion space. Dynamics are therefore not only a source of modeling difficulty; they are a source of manipulation capability.

Non-prehensile interaction also changes the precision assumptions built into the task. Prehensile manipulation often aims to reduce uncertainty by fixing the object relative to the robot hand. Non-prehensile manipulation instead allows the robot to act through brief contacts, sliding surfaces, cages, impacts, or whole-body interactions. This broadens the design space from precise object transport to task-level influence over object state. Non-prehensile motion primitives are useful precisely because many tasks do not require exact control over the full object trajectory. They require the object to reach an acceptable region, move away from clutter, enter a container, cross a boundary, or become accessible for a later action. In such cases, imprecision is not necessarily an

error, but structure that the planner can exploit.

This view raises a basic representational question: what should count as a manipulation skill? Named skills such as grasping, pushing, poking, sliding, caging, throwing, and scooping are useful because they organize the design of planners and datasets. However, physical interaction does not always respect these boundaries. A single manipulation episode may transition continuously between sliding, pulling, caging, reorienting, and grasping as contact modes change. Human manipulation makes this especially clear: people do not appear to select from a rigid library of atomic skills, but instead move fluidly across interaction modes as the task demands. The open question is whether robotic systems should maintain explicit skill libraries, learn broader generative policies that absorb transitions between skills, or use hybrid representations in which named skills provide structure while learned models handle the continuous transitions between them.

Dynamic manipulation also makes embodiment harder to ignore. Many data collection and augmentation methods are effective because they treat the end-effector trajectory as the central object of learning. This is a reasonable abstraction for many quasi-static prehensile tasks, where the gripper motion largely determines object motion. It is less sufficient when success depends on robot inertia, joint limits, actuator bandwidth, compliance, contact timing, whole-body motion, or momentum transfer. A demonstration that captures only the end-effector path may omit the physical quantities that made the behavior feasible for the demonstrator. This creates an open question for robot learning: how should demonstrations represent embodiment when dynamics matter, and what minimal information about the robot body, actuation limits, and contact conditions must be preserved for a behavior to transfer across platforms?

8.2.3 Scope and Limits of Statistical Learning

The learning-based chapters of this thesis use diffusion models to represent dynamically feasible motion, but they do not imply that diffusion replaces planning. A more precise interpretation is that diffusion provides fast candidate motions by learning from prior planning examples. The model observes feasible trajectories during training and learns recurring patterns in how constraints

shape motion, allowing it to generate useful trajectory proposals at inference time. These proposals can then be checked, refined, or optimized by a downstream planning system. This is useful because repeated feasibility reasoning can be expensive, especially when constraints depend on payloads, failures, higher-order robot limits, or task-specific dynamic effects. However, the generated motion is still meaningful only because it is tied to explicit notions of feasibility: the data must contain feasible behavior, the conditioning variables must describe the relevant physical regime, and downstream checking or optimization may still be required. Diffusion is therefore most valuable as a complement to planning, not as a substitute for it.

This interpretation also clarifies why data design is often the dominant lever. Across the experiments in this thesis, performance depends strongly on the structure of the training distribution: which payloads are represented, which failure cases are included, how collision avoidance appears in the demonstrations, whether the trajectories are dynamically feasible, and how augmentation expands the feasible set. These choices often matter as much as the architecture or the particular constraint-injection mechanism. For dynamics-aware robot learning, the dataset is not a neutral input to the model—it is a computational representation of the feasible motion space. A poorly designed dataset can make a powerful model appear weak, while a carefully structured dataset can allow a relatively simple model to generate useful constrained motions.

This raises an important question about composition. A strong claim would be that diffusion models can combine training examples to generate genuinely new constraint-satisfying behaviors. A more conservative interpretation is that they interpolate effectively within the structure of their training data, but struggle when asked to invent new strategies outside that support. The distinction matters because robot manipulation often requires behavior under changed objects, constraints, payloads, embodiments, and environments. The open question is therefore not whether diffusion can produce smooth and plausible trajectories, but under what conditions it can compose physical constraints and motion patterns beyond the dataset that produced it. Answering this question will require training distributions and evaluation protocols designed specifically to separate interpolation from generalization [38].

The choice of input representation is part of the same issue. In vision-language-action systems, much of the computational cost is often concentrated in the visual encoder rather than the action head. For dynamics-constrained manipulation, this raises a more basic question about what the policy should consume. RGB images provide broad perceptual generality, but may hide the geometric, contact, and embodiment information that determines dynamic feasibility. Point clouds, signed distance fields, object-centric states, constraint descriptors, learned embeddings, and hybrid geometric-visual representations expose different forms of structure at different computational costs. The open question is which representation offers the right balance between perceptual flexibility, efficiency, and awareness of the constraints that actually govern motion.

A related caution is that diffusion models may behave, in some regimes, like powerful conditional lookup tables over the training distribution [38]. If so, then improvements attributed to increasingly structured representations—from RGB images to point clouds to signed distance fields, for example—may sometimes reflect better dataset alignment rather than deeper physical reasoning. The important question is not whether a representation appears more geometric or more structured, but whether it measurably improves constraint satisfaction, behavior composition, and generalization under changed physical conditions. Therefore, learned planners should be tested not only on in-distribution success, but also on their ability to maintain feasibility when the task, embodiment, or constraint regime changes.

These limitations suggest a productive role for diffusion inside larger planning systems. Rather than treating a learned generative model as a complete planner, it may be more useful to treat it as a local motion generator or proposal mechanism. Some behaviors, such as poking, can be well captured through simulation-in-the-loop search or explicit kinodynamic reasoning. Others may be easier to learn from demonstrations. A global planner could sequence across heterogeneous skills, while diffusion proposes short-horizon motions that satisfy local task, embodiment, and constraint conditions. The open question is how to design the interface between these components: what information should the global planner pass to the learned local model, what guarantees or feasibility checks should be returned, and how the system should preserve the strengths of both

structured planning and learned generation.

8.3 Final Synthesis

The broader conclusion of this thesis is that robotic manipulation cannot be fully understood through a purely geometric view of action. Geometric reasoning remains essential: robots must still satisfy reachability, collision avoidance, grasp validity, and configuration-space constraints. However, many of the behaviors that expand manipulation capability are governed by quantities that geometry alone does not capture. Contact, inertia, timing, actuation limits, payload variation, fluid motion, and momentum transfer can determine not only whether a motion succeeds, but what kinds of motions are available in the first place. Dynamic and non-prehensile interactions are therefore not edge cases at the boundary of manipulation. They expose fundamental questions about how robotic systems represent feasibility, how skills are defined, how planners and learned models should interact, and how physical generalization should be evaluated.

The systems developed in this thesis address these questions from several directions. Simulation-in-the-loop planning shows that dynamic interactions such as poking and whole-body contact can expose useful behaviors that are difficult to specify geometrically. Reusable kinodynamic structure shows that dynamic feasibility can be amortized across related planning queries rather than recomputed from scratch. Conditional generative models show that learned trajectory distributions can absorb information about payloads, failures, and constraint manifolds, enabling fast motion generation when the relevant physical parameters are made explicit. Together, these results suggest that the next generation of manipulation systems should not treat geometry, timing, contact, dynamics, and learning as isolated components in a sequential pipeline. They should instead expose these factors through interfaces that allow planners, optimizers, simulators, and learned models to share responsibility for kinodynamic feasibility, enabling robots to move more fluidly across the diverse interaction modes that manipulation in the physical world requires.

Bibliography

- [1] Ava Abderezaei, Anuj Pasricha, Alex Klausenstock, and Alessandro Roncone. Clutter-aware spill-free liquid transport via learned dynamics. In 2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), pages 9908–9915. IEEE, 2024.
- [2] Anurag Ajay, Yilun Du, Abhi Gupta, Joshua B. Tenenbaum, Tommi S. Jaakkola, and Pulkit Agrawal. Is conditional generative modeling all you need for decision making? In The Eleventh International Conference on Learning Representations, 2023.
- [3] Marcin Andrychowicz, Bowen Baker, Maciek Chociej, Rafal Jozefowicz, Bob McGrew, Jakub Pachocki, Arthur Petron, Matthias Plappert, Glenn Powell, Alex Ray, et al. Learning dexterous in-hand manipulation. arXiv preprint arXiv:1808.00177, 2018.
- [4] Jon Arrizabalaga, Lukas Pries, Riddhiman Laha, Runkang Li, Sami Haddadin, and Markus Ryll. Geometric slosh-free tracking for robotic manipulators. In 2024 IEEE International Conference on Robotics and Automation (ICRA), pages 1226–1232, 2024.
- [5] Alex Austin, Brent Sherwood, John Elliott, Anthony Colaprete, Kris Zacny, Philip Metzger, Michael Sims, Harrison Schmitt, Sandra Magnus, Terry Fong, et al. Robotic lunar surface operations 2. Acta Astronautica, 176:424–437, 2020.
- [6] Yahav Avigal, Jeffrey Ichnowski, Max Yiye Cao, and Ken Goldberg. Gomp-st: Grasp optimized motion planning for suction transport. In International Workshop on the Algorithmic Foundations of Robotics, pages 488–505. Springer, 2022.
- [7] Federico Barbieri, J Wesley Hines, Michael Sharp, and Mauro Venturini. Sensor-based degradation prediction and prognostics for remaining useful life estimation: Validation on experimental data of electric motors. International Journal of Prognostics and Health Management, 6(3), 2015.
- [8] Piotr Barosz, Grzegorz Gołda, and Adrian Kampa. Efficiency analysis of manufacturing line with industrial robots and human operators. Applied Sciences, 10(8):2862, 2020.
- [9] Maria Bauza, Ferran Alet, Yen-Chen Lin, Tomás Lozano-Pérez, Leslie P Kaelbling, Phillip Isola, and Alberto Rodriguez. Omnipush: accurate, diverse, real-world dataset of pushing dynamics with RGB-D video. In 2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), pages 4265–4272. IEEE, 2019.
- [10] P. Beeson and B. Ames. TRAC-IK: An open-source library for improved solving of generic inverse kinematics. In 2015 IEEE-RAS 15th International Conference on Humanoid Robots (Humanoids), pages 928–935, 2015.

- [11] Lars Berscheid and Torsten Kröger. Jerk-limited real-time trajectory generation with arbitrary target states. Robotics: Science and Systems XVII, 2021.
- [12] Aude Billard and Danica Kragic. Trends and challenges in robot manipulation. Science, 364(6446), 2019.
- [13] A. M. Bloch, P. S. Krishnaprasad, and R. M. Murray. Nonholonomic Mechanics and Control. Springer, 2016.
- [14] Jeannette Bohg, Antonio Morales, Tamim Asfour, and Danica Kragic. Data-driven grasp synthesis—a survey. IEEE Transactions on Robotics, 30(2):289–309, 2013.
- [15] Robert Bohlin and Lydia E Kavraki. Path planning using lazy prm. In Proceedings of the IEEE International Conference on Robotics and Automation (ICRA), pages 521–528, 2000.
- [16] Ricard Bordalba, Lluís Ros, and Josep M Porta. Randomized kinodynamic planning for constrained systems. In 2018 IEEE international conference on robotics and automation (ICRA), pages 7079–7086. IEEE, 2018.
- [17] Gilberto Briscoe-Martinez, Anuj Pasricha, Ava Abderezaei, Santosh Chaganti, Sarath Chandra Vajrала, Sri Kanth Popuri, and Alessandro Roncone. Exploring how non-prehensile manipulation expands capability in robots experiencing multi-joint failure. In IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), 2024.
- [18] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Joseph Dabis, Chelsea Finn, Keerthana Gopalakrishnan, Karol Hausman, Alexander Herzog, Jasmine Hsu, et al. Rt-1: Robotics transformer for real-world control at scale. In Robotics: Science and Systems, 2023.
- [19] Ian M Bullock and Aaron M Dollar. Classifying human manipulation behavior. In 2011 IEEE International Conference on Rehabilitation Robotics, pages 1–6. IEEE, 2011.
- [20] Timothy M Caldwell and Nikolaus Correll. Fast sample-based planning for dynamic systems by zero-control linearization-based steering. Robotics Research: Volume 2, pages 453–469, 2018.
- [21] Joao Carvalho, An T Le, Mark Baierl, Dorothea Koert, and Jan Peters. Motion planning diffusion: Learning and planning of robot motions with diffusion models. In 2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), pages 1916–1923. IEEE, 2023.
- [22] Cheng Chi, Siyuan Feng, Yilun Du, Zhenjia Xu, Eric Cousineau, Benjamin Burchfiel, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. In Proceedings of Robotics: Science and Systems (RSS), 2023.
- [23] David Coleman, Ioan Sucan, Sachin Chitta, and Nikolaus Correll. Reducing the barrier to entry of complex robotic software: a moveit! case study. arXiv preprint arXiv:1404.3785, 2014.
- [24] Jack Collins, David Howard, and Jurgen Leitner. Quantifying the reality gap in robotic manipulation tasks. In 2019 International Conference on Robotics and Automation (ICRA), pages 6706–6712. IEEE, 2019.

- [25] Erwin Coumans and Yunfei Bai. Pybullet, a python module for physics simulation for games, robotics and machine learning. <http://pybullet.org>, 2016–2021.
- [26] Roberto Di Leva, Marco Carricato, Hubert Gatttringer, and Andreas Mueller. Sloshing dynamics estimation for liquid-filled containers performing 3-dimensional motions: modeling and experimental validation. *Multibody System Dynamics*, 56, 08 2022.
- [27] Roberto Di Leva et al. Time-optimal trajectory planning for anti-sloshing 2-dimensional motions of an industrial robot. In *2021 20th International Conference on Advanced Robotics (ICAR)*, pages 32–37, 2021.
- [28] Mehmet Remzi Dogar and Siddhartha S Srinivasa. A framework for push-grasping in clutter. In *Robotics: Science and systems*, volume 2, 2011.
- [29] Norm Ferns, Prakash Panangaden, and Doina Precup. Metrics for finite Markov decision processes. In *Proceedings of the 20th Conference on Uncertainty in Artificial Intelligence (UAI)*, pages 162–169, 2004.
- [30] Adam Fishman, Adithyavairavan Murali, Clemens Eppner, Bryan Peele, Byron Boots, and Dieter Fox. Motion policy networks. In *Conference on Robot Learning*, pages 967–977. PMLR, 2023.
- [31] Claudio Gaz, Marco Cognetti, Alexander Oliva, Paolo Robuffo Giordano, and Alessandro De Luca. Dynamic identification of the franka emika panda robot with retrieval of feasible parameters using penalty-based optimization. *IEEE Robotics and Automation Letters*, 4(4):4147–4154, 2019.
- [32] Zhaoqiang Ge and Xiaochi Ge. Controllability of singular distributed parameter systems in the sense of mild solution. *Journal of Systems Science and Complexity*, 33(5):1485–1496, Oct 2020.
- [33] Carles Gelada, Saurabh Kumar, Jacob Buckman, Ofir Nachum, and Marc G. Belle-mare. Deepmdp: Learning continuous latent space models for representation learning. In *Proceedings of the 36th International Conference on Machine Learning*, pages 2170–2179, 2019.
- [34] Huy Ha, Pete Florence, and Shuran Song. Scaling up and distilling down: Language-guided robot skill acquisition. In *Conference on Robot Learning*, pages 3766–3777. PMLR, 2023.
- [35] Jiwon Han. A study on the coffee spilling phenomena in the low impulse regime. *Achievements in the Life Sciences*, 10(1):87–101, 2016.
- [36] Kris Hauser. Lazy collision checking in asymptotically-optimal motion planning. In *2015 IEEE international conference on robotics and automation (ICRA)*, pages 2951–2957. IEEE, 2015.
- [37] Joshua A Haustein, Jennifer King, Siddhartha S Srinivasa, and Tamim Asfour. Kinodynamic randomized rearrangement planning via dynamic transitions between statically stable states. In *2015 IEEE International Conference on Robotics and Automation (ICRA)*, pages 3075–3082. IEEE, 2015.

- [38] Chengyang He, Xu Liu, Gadiel Mark Sznaiers Camps, Joseph Bruno, Guillaume Adrien Sartoretti, and Mac Schwager. Demystifying robot diffusion policies: Action memorization and a simple lookup table alternative. In The Fourteenth International Conference on Learning Representations, 2026.
- [39] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. Advances in neural information processing systems, 33:6840–6851, 2020.
- [40] Cheng-Chun Hsu, Bowen Wen, Jie Xu, Yashraj Narang, Xiaolong Wang, Yuke Zhu, Joydeep Biswas, and Stan Birchfield. Spot: Se(3) pose trajectory diffusion for object-centric manipulation. In 2025 IEEE International Conference on Robotics and Automation (ICRA), 2025.
- [41] Huang Huang, Balakumar Sundaralingam, Arsalan Mousavian, Adithyavairavan Murali, Ken Goldberg, and Dieter Fox. Diffusionseeder: Seeding motion optimization with diffusion for rapid motion planning. arXiv preprint arXiv:2410.16727, 2024.
- [42] Wesley Huang. Impulsive Manipulation. PhD thesis, Carnegie Mellon University, Pittsburgh, PA, August 1997.
- [43] Jeffrey Ichnowski, Yahav Avigal, Yi Liu, and Ken Goldberg. Gomp-fit: Grasp-optimized motion planning for fast inertial transport. In 2022 International Conference on Robotics and Automation (ICRA), pages 5255–5261. IEEE, 2022.
- [44] Jeffrey Ichnowski, Yahav Avigal, Vishal Satish, and Ken Goldberg. Deep learning can accelerate grasp-optimized motion planning. Science Robotics, 5(48):eabd7710, 2020.
- [45] Jeffrey Ichnowski, Michael Danielczuk, Jingyi Xu, Vishal Satish, and Ken Goldberg. Gomp: Grasp-optimized motion planning for bin picking. In 2020 IEEE International Conference on Robotics and Automation (ICRA), pages 5270–5277. IEEE, 2020.
- [46] International Electrotechnical Commission. Safety of machinery — electrical equipment of machines — part 1: General requirements, 2016. IEC 60204-1.
- [47] International Organization for Standardization. Robots and robotic devices, safety requirements for industrial robots, part 1: Robots, 2011. ISO 10218-1.
- [48] International Organization for Standardization. Safety of machinery, safety-related parts of control systems, part 1: General principles for design, 2015. ISO 13849-1.
- [49] Michael Janner, Yilun Du, Joshua Tenenbaum, and Sergey Levine. Planning with diffusion for flexible behavior synthesis. In Proceedings of the 39th International Conference on Machine Learning, volume 162 of Proceedings of Machine Learning Research, pages 9902–9915. PMLR, 2022.
- [50] Yan-Bin Jia, Matthew Gardner, and Xiaoqian Mu. Batting an in-flight object to the target. The International Journal of Robotics Research, 38(4):451–485, 2019.
- [51] Jeff Johnson and Kris Hauser. Optimal acceleration-bounded trajectory planning in dynamic environments along a specified path. In 2012 IEEE International Conference on Robotics and Automation, pages 2035–2041. IEEE, 2012.

- [52] Mrinal Kalakrishnan, Sachin Chitta, Evangelos Theodorou, Peter Pastor, and Stefan Schaal. Stomp: Stochastic trajectory optimization for motion planning. In 2011 IEEE international conference on robotics and automation, pages 4569–4574. IEEE, 2011.
- [53] Jay Kamat, Joaquim Ortiz-Haro, Marc Toussaint, Florian T Pokorny, and Andreas Orthey. Bitkomo: Combining sampling and optimization for fast convergence in optimal motion planning. In 2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), pages 4492–4497. IEEE, 2022.
- [54] Sertac Karaman and Emilio Frazzoli. Sampling-based algorithms for optimal motion planning. The International Journal of Robotics Research, 30(7):846–894, 2011.
- [55] Sertac Karaman, Matthew R Walter, Alejandro Perez, Emilio Frazzoli, and Seth Teller. Anytime motion planning using the rrt. In 2011 IEEE International Conference on Robotics and Automation, pages 1478–1483. IEEE, 2011.
- [56] Lydia E Kavraki, Petr Svestka, J-C Latombe, and Mark H Overmars. Probabilistic roadmaps for path planning in high-dimensional configuration spaces. IEEE transactions on Robotics and Automation, 12(4):566–580, 1996.
- [57] Tsung-Wei Ke, Nikolaos Gkanatsios, and Katerina Fragkiadaki. 3d diffuser actor: Policy diffusion with 3d scene representations. In Proceedings of The 8th Conference on Robot Learning, volume 270 of Proceedings of Machine Learning Research, pages 1949–1974. PMLR, 2025.
- [58] Mincheol Kim, Ukcheol Shin, and Jung-Yup Kim. Learning quadrupedal locomotion with impaired joints using random joint masking. In IEEE International Conference on Robotics and Automation (ICRA), 2024.
- [59] Raymond Kim, Stephen Balakirsky, Konrad Ahlin, Matthew Marcum, and Anirban Mazumdar. Enhancing payload capacity with dual-arm manipulation and adaptable mechanical intelligence. Journal of Mechanisms and Robotics, 13(2):021012, 2021.
- [60] Seungsu Kim, Ashwini Shukla, and Aude Billard. Catching objects in flight. IEEE Transactions on Robotics, 30(5):1049–1065, 2014.
- [61] Jennifer E King, Joshua A Haustein, Siddhartha S Srinivasa, and Tamim Asfour. Nonprehensile whole arm rearrangement planning on physics manifolds. In 2015 IEEE International Conference on Robotics and Automation (ICRA), pages 2508–2515. IEEE, 2015.
- [62] Zachary Kingston, Mark Moll, and Lydia E Kavraki. Sampling-based methods for motion planning with constraints. Annual review of control, robotics, and autonomous systems, 1:159–185, 2018.
- [63] Michal Kleinbort, Kiril Solovey, Zakary Littlefield, Kostas E Bekris, and Dan Halperin. Probabilistic completeness of rrt for geometric and kinodynamic planning with forward propagation. IEEE Robotics and Automation Letters, 4(2):x–xvi, 2018.
- [64] Alina Kloss, Maria Bauza, Jiajun Wu, Joshua B Tenenbaum, Alberto Rodriguez, and Jeanette Bohg. Accurate vision-based manipulation through contact reasoning. In 2020 IEEE International Conference on Robotics and Automation (ICRA), pages 6738–6744. IEEE, 2020.

- [65] MH Korayem and A Nikoobin. Maximum payload for flexible joint manipulators in point-to-point task using optimal control approach. The International Journal of Advanced Manufacturing Technology, 38:1045–1060, 2008.
- [66] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. Imagenet classification with deep convolutional neural networks. In Advances in Neural Information Processing Systems, volume 25, 2012.
- [67] Alan Kuntz, Chris Bowen, and Ron Alterovitz. Fast anytime motion planning in point clouds by interleaving sampling and interior point optimization. In Robotics Research: The 18th International Symposium ISRR, pages 929–945. Springer, 2019.
- [68] Tobias Kunz and Mike Stilman. Probabilistically complete kinodynamic planning for robot manipulators with acceleration limits. In 2014 IEEE/RSJ International Conference on Intelligent Robots and Systems, pages 3713–3719. IEEE, 2014.
- [69] Tobias Kunz and Mike Stilman. Kinodynamic rrtts with fixed time step and best-input extension are not probabilistically complete. In Algorithmic foundations of robotics XI, pages 233–244. Springer, 2015.
- [70] Maria Kyrarini, Fotios Lygerakis, Akilesh Rajavenkatanarayanan, Christos Sevastopoulos, Harish Ram Nambiappan, Kodur Krishna Chaitanya, Ashwin Ramesh Babu, Joanne Mathew, and Fillia Makedon. A survey of robots in healthcare. Technologies, 9(1), 2021.
- [71] Yong-Hong Lan, Bin Wu, Yue-Xiang Shi, and Yi-Ping Luo. Iterative learning based consensus control for distributed parameter multi-agent systems with time-delay. Neurocomputing, 357:77–85, 2019.
- [72] Steven M LaValle. Rapidly-exploring random trees: A new tool for path planning. Technical Report TR 98-11, Computer Science Department, Iowa State University, 1998.
- [73] Steven M LaValle. Planning algorithms. Cambridge university press, 2006.
- [74] Steven M LaValle and James J Kuffner Jr. Randomized kinodynamic planning. The international journal of robotics research, 20(5):378–400, 2001.
- [75] Hang Li, Qian Feng, Zhi Zheng, Jianxiang Feng, and Alois Knoll. Language-guided object-centric diffusion policy for generalizable and collision-aware robotic manipulation. arXiv preprint arXiv:2407.00451, 2024.
- [76] Jue Kun Li, Wee Sun Lee, and David Hsu. Push-Net: Deep planar pushing for objects with unknown physical properties. In Robotics: Science and Systems, 2018.
- [77] Lihong Li, Thomas J Walsh, and Michael L Littman. Towards a unified theory of state abstraction for MDPs. In Proceedings of the Ninth International Symposium on Artificial Intelligence and Mathematics (ISAIM), 2006.
- [78] Yanbo Li, Zakary Littlefield, and Kostas E Bekris. Sparse methods for efficient asymptotically optimal kinodynamic planning. In Algorithmic foundations of robotics XI, pages 263–282. Springer, 2015.

- [79] Yanbo Li, Zakary Littlefield, and Kostas E Bekris. Asymptotically optimal sampling-based kinodynamic planning. The International Journal of Robotics Research, 35(5):528–564, 2016.
- [80] Zakary Littlefield. Efficient and Asymptotically Optimal Kinodynamic Motion Planning. PhD thesis, Rutgers The State University of New Jersey, School of Graduate Studies, 2020.
- [81] Andreas Lugmayr, Martin Danelljan, Andres Romero, Fisher Yu, Radu Timofte, and Luc Van Gool. Repaint: Inpainting using denoising diffusion probabilistic models. In 2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pages 11451–11461, 2022.
- [82] Kevin M Lynch and Matthew T Mason. Dynamic nonprehensile manipulation: Controllability, planning, and experiments. The International Journal of Robotics Research, 18(1):64–92, 1999.
- [83] Sonja Macfarlane and Elizabeth A. Croft. Jerk-bounded manipulator trajectory planning: Design for real-time applications. IEEE Transactions on Robotics and Automation, 19(1):42–52, February 2003.
- [84] Riccardo Maderna, Andrea Casalino, Andrea Maria Zanchettin, and Paolo Rocco. Robotic handling of liquids with spilling avoidance: A constraint-based control approach. In 2018 IEEE International Conference on Robotics and Automation (ICRA), pages 7414–7420, 2018.
- [85] Jeffrey Mahler, Jacky Liang, Sherdil Niyaz, Michael Laskey, Richard Doan, Xinyu Liu, Juan Aparicio Ojea, and Ken Goldberg. Dex-net 2.0: Deep learning to plan robust grasps with synthetic point clouds and analytic grasp metrics. arXiv preprint arXiv:1703.09312, 2017.
- [86] Aditya Mandalika, Sanjiban Choudhury, Oren Salzman, and Siddhartha Srinivasa. Generalized lazy search for robot motion planning: Interleaving search and edge evaluation via event-based toggles. In Proceedings of the International Conference on Automated Planning and Scheduling, volume 29, pages 745–753, 2019.
- [87] Jean-Charles Maré. Reliability. In Aerospace Actuators 1, chapter 2, pages 33–61. John Wiley & Sons, Ltd, 2016.
- [88] Matthew T Mason. Mechanics and planning of manipulator pushing operations. The International Journal of Robotics Research, 5(3):53–71, 1986.
- [89] Matthew T Mason. Toward robotic manipulation. Annual Review of Control, Robotics, and Autonomous Systems, 1:1–28, 2018.
- [90] Troy McMahon, Aravind Sivaramakrishnan, Edgar Granados, Kostas E Bekris, et al. A survey on the integration of machine learning with sampling-based motion planning. Foundations and Trends® in Robotics, 9(4):266–327, 2022.
- [91] Naim Montazeri, Alexandra CM Oliveira, Brian H Himelbloom, Mary Beth Leigh, and Charles A Crapo. Chemical characterization of commercial liquid smoke products. Food science & nutrition, 1(1):102–115, 2013.
- [92] Lorenzo Moriello, Luigi Biagiotti, Claudio Melchiorri, and Andrea Paoli. Manipulating liquids with robots: A sloshing-free solution. Control Engineering Practice, 78:129–141, 2018.

- [93] Zonggao Mu, Liang Han, Wenfu Xu, Bing Li, and Bin Liang. Kinematic analysis and fault-tolerant trajectory planning of space manipulator under a single joint failure. Robotics and biomimetics, 3:1–10, 2016.
- [94] Rafael I. Cabral Muchacho, Riddhiman Laha, Luis F.C. Figueredo, and Sami Haddadin. A solution to slosh-free robot trajectory optimization. In 2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), pages 223–230, 2022.
- [95] Muhayyuddin, Mark Moll, Lydia Kavraki, and Jan Rosell. Randomized physics-based motion planning for grasping in cluttered and uncertain environments. IEEE Robotics and Automation Letters, 3(2):712–719, 2018.
- [96] Brian K Muirhead and Lorraine Fesq. Coalescing nasa’s views of fault and health management. Spacecraft Fault Management Workshop, Jet Propulsion Laboratory, Pasadena, CA, 2012.
- [97] NASA Science. First drilled sample on mars since 2016. NASA Science: Mars, 2018.
- [98] NASA/JPL. Shoulder motor balks on opportunity rover’s robotic arm. Jet Propulsion Laboratory News, 2008.
- [99] Sharan Nayak and Michael W Otte. Bidirectional sampling-based motion planning without two-point boundary value solution. IEEE Transactions on Robotics, 38(6):3636–3654, 2022.
- [100] Hyuk C Nho and Peter Meckl. Intelligent feedforward control and payload estimation for a two-link robotic manipulator. IEEE/ASME transactions on mechatronics, 8(2):277–282, 2003.
- [101] Alex Nichol, Prafulla Dhariwal, Aditya Ramesh, Pranav Shyam, Pamela Mishkin, Bob McGrew, Ilya Sutskever, and Mark Chen. Glide: Towards photorealistic image generation and editing with text-guided diffusion models. arXiv preprint arXiv:2112.10741, 2021.
- [102] Abby O’Neill, Abdul Rehman, Abhiram Maddukuri, Abhishek Gupta, Abhishek Padalkar, Abraham Lee, Acorn Pooley, Agrim Gupta, Ajay Mandlekar, Ajinkya Jain, et al. Open x-embodiment: Robotic learning datasets and rt-x models: Open x-embodiment collaboration 0. In 2024 IEEE International Conference on Robotics and Automation (ICRA), pages 6892–6903. IEEE, 2024.
- [103] Andreas Orthey, Constantinos Chamzas, and Lydia E Kavraki. Sampling-based motion planning: A comparative review. Annual Review of Control, Robotics, and Autonomous Systems, 7, 2023.
- [104] Evangelos Papadopoulos, Farhad Aghili, Ou Ma, and Roberto Lampariello. Robotic manipulation and capture in space: A survey. Frontiers in Robotics and AI, page 228, 2021.
- [105] Chonhyon Park, Jia Pan, and Dinesh Manocha. Itomp: Incremental trajectory optimization for real-time replanning in dynamic environments. In Proceedings of the international conference on automated planning and scheduling, volume 22, pages 207–215, 2012.
- [106] Anuj Pasricha and Alessandro Roncone. The virtues of laziness: Multi-query kinodynamic motion planning with lazy methods. In 2024 IEEE International Conference on Robotics and Automation (ICRA), pages 14286–14292. IEEE, 2024.

- [107] Anuj Pasricha, Yi Shiuan Tung, Bradley Hayes, and Alessandro Roncone. PokeRRT: A Kinodynamic Planning Approach for Poking Manipulation. In IROS 2021 Workshop on Impact-Aware Robotics. IEEE, 2021.
- [108] Anuj Pasricha, Yi-Shiuan Tung, Bradley Hayes, and Alessandro Roncone. Pokerrt: Poking as a skill and failure recovery tactic for planar non-prehensile manipulation. IEEE Robotics and Automation Letters, 7(2):4480–4487, 2022.
- [109] Andi Peng, Ilia Sucholutsky, Belinda Z. Li, Theodore R. Sumers, Thomas L. Griffiths, Jacob Andreas, and Julie A. Shah. Learning with language-guided state abstractions. arXiv preprint arXiv:2402.18759, 2024.
- [110] Alejandro Perez, Robert Platt, George Konidaris, Leslie Kaelbling, and Tomas Lozano-Perez. Lqr-rrt*: Optimal sampling-based motion planning with automatically derived extension heuristics. In 2012 IEEE International Conference on Robotics and Automation, pages 2537–2542. IEEE, 2012.
- [111] Ethan Perez, Florian Strub, Harm De Vries, Vincent Dumoulin, and Aaron Courville. Film: Visual reasoning with a general conditioning layer. In Proceedings of the AAAI conference on artificial intelligence, volume 32, 2018.
- [112] Hung Pham and Quang-Cuong Pham. A new approach to time-optimal path parameterization based on reachability analysis. IEEE Transactions on Robotics, 34(3):645–659, 2018.
- [113] Quang-Cuong Pham. A general, fast, and robust implementation of the time-optimal path parameterization algorithm. IEEE Transactions on Robotics, 30(6):1533–1540, 2014.
- [114] Quang-Cuong Pham, Stéphane Caron, and Yoshihiko Nakamura. Kinodynamic planning in the configuration space via admissible velocity propagation. In Robotics: Science and Systems, volume 32, 2013.
- [115] Tan-Hanh Pham, Godwyll Aikins, Tri Truong, and Kim-Doang Nguyen. Adaptive compensation for robotic joint failures using partially observable reinforcement learning. Algorithms, 17(436):1–16, 2024.
- [116] Erion Plaku, Lydia E Kavraki, and Moshe Y Vardi. Motion planning with dynamics by a synergistic combination of layers of planning. IEEE Transactions on Robotics, 26(3):469–482, 2010.
- [117] Charles Ruizhongtai Qi, Li Yi, Hao Su, and Leonidas J Guibas. PointNet++: Deep hierarchical feature learning on point sets in a metric space. In Advances in Neural Information Processing Systems, volume 30, 2017.
- [118] Ahmed Hussain Qureshi, Yinglong Miao, Anthony Simeonov, and Michael C Yip. Motion planning networks: Bridging the gap between learning-based and classical motion planners. IEEE Transactions on Robotics, 37(1):48–66, 2020.
- [119] Nathan Ratliff, Matt Zucker, J Andrew Bagnell, and Siddhartha Srinivasa. Chomp: Gradient optimization techniques for efficient motion planning. In 2009 IEEE international conference on robotics and automation, pages 489–494. IEEE, 2009.

- [120] Jan Reinhold, Manuel Amersdorfer, and Thomas Meurer. A dynamic optimization approach for sloshing free transport of liquid filled containers using an industrial robot. In 2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), pages 2336–2341, 2019.
- [121] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, pages 10684–10695, 2022.
- [122] Fabio Ruggiero, Vincenzo Lippiello, and Bruno Siciliano. Nonprehensile dynamic manipulation: A survey. IEEE Robotics and Automation Letters, 3(3):1711–1718, 2018.
- [123] Kallol Saha, Vishal Mandadi, Jayaram Reddy, Ajit Srikanth, Aditya Agarwal, Bipasha Sen, Arun Singh, and Madhava Krishna. Edmp: Ensemble-of-costs-guided diffusion for motion planning. In 2024 IEEE International Conference on Robotics and Automation (ICRA), pages 10351–10358. IEEE, 2024.
- [124] Gautam Salhotra, I-Chun Arthur Liu, Marcus Dominguez-Kuhne, and Gaurav S Sukhatme. Learning deformable object manipulation from expert demonstrations. IEEE Robotics and Automation Letters, 7(4):8775–8782, 2022.
- [125] Sezimaria FP Saramago and Marco Ceccarelli. An optimum robot path planning with payload constraints. Robotica, 20(4):395–404, 2002.
- [126] Edward Schmerling and Marco Pavone. Kinodynamic planning. Encyclopedia of Robotics. Springer,, 2019.
- [127] John Schulman, Jonathan Ho, Alex X Lee, Ibrahim Awwal, Henry Bradlow, and Pieter Abbeel. Finding locally optimal, collision-free trajectories with sequential convex optimization. In Robotics: Science and Systems, volume 9, pages 1–10, 2013.
- [128] Mingyo Seo, Yoonyoung Cho, Yoonchang Sung, Peter Stone, Yuke Zhu, and Beomjoon Kim. Presto: Fast motion planning using diffusion models based on key-configuration environment representation. arXiv preprint arXiv:2409.16012, 2024.
- [129] Yu She, Wenfu Xu, Haijun Su, Bin Liang, and Hongliang Shi. Fault-tolerant analysis and control of ssrms-type manipulators with single-joint failure. Acta Astronautica, 120:270–286, 2016.
- [130] Rahul Shome and Lydia E Kavraki. Asymptotically optimal kinodynamic planning using bundles of edges. In Proceedings of the 2021 International Conference on Robotics and Automation (ICRA), Xian, China, volume 30, 2021.
- [131] Jascha Sohl-Dickstein, Eric Weiss, Niru Maheswaranathan, and Surya Ganguli. Deep unsupervised learning using nonequilibrium thermodynamics. In International conference on machine learning, pages 2256–2265. pmlr, 2015.
- [132] Changkyu Song and Abdeslam Boularias. Learning to slide unknown objects with differentiable physics simulations. In Proceedings of Robotics: Science and Systems (RSS), Corvallis, Oregon, 2020.

- [133] Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising diffusion implicit models. In International Conference on Learning Representations, 2021.
- [134] Ajay Sridhar, Dhruv Shah, Catherine Glossop, and Sergey Levine. Nomad: Goal masked diffusion policies for navigation and exploration. In 2024 IEEE International Conference on Robotics and Automation (ICRA), pages 63–70. IEEE, 2024.
- [135] Robin Strudel, Ricardo Garcia Pinel, Justin Carpentier, Jean-Paul Laumond, Ivan Laptev, and Cordelia Schmid. Learning obstacle representations for neural motion planning. In Proceedings of the 2020 Conference on Robot Learning, volume 155 of Proceedings of Machine Learning Research, pages 355–364. PMLR, 2021.
- [136] Jochen Stüber, Claudio Zito, and Rustam Stolkin. Let’s push things forward: A survey on robot pushing. Frontiers in Robotics and AI, 7:8, 2020.
- [137] Ioan A Sutan and Lydia E Kavraki. A sampling-based tree planner for systems with complex dynamics. IEEE Transactions on Robotics, 28(1):116–131, 2011.
- [138] Balakumar Sundaralingam, Siva Kumar Sastry Hari, Adam Fishman, Caelan Garrett, Karl Van Wyk, Valts Blukis, Alexander Millane, Helen Oleynikova, Ankur Handa, Fabio Ramos, et al. Curobo: Parallelized collision-free robot motion generation. In 2023 IEEE International Conference on Robotics and Automation (ICRA), pages 8112–8119. IEEE, 2023.
- [139] Octo Model Team, Dibya Ghosh, Homer Walke, Karl Pertsch, Kevin Black, Oier Mees, Sudeep Dasari, Joey Hejna, Tobias Kreiman, Charles Xu, et al. Octo: An open-source generalist robot policy. arXiv preprint arXiv:2405.12213, 2024.
- [140] Wil Thomason, Zachary Kingston, and Lydia E Kavraki. Motions in microseconds via vectorized sampling-based planning. In 2024 IEEE International Conference on Robotics and Automation (ICRA), pages 8749–8756. IEEE, 2024.
- [141] Marc Toussaint. Newton methods for k-order markov constrained motion problems. arXiv preprint arXiv:1407.0414, 2014.
- [142] Marc A Toussaint, Kelsey Rebecca Allen, Kevin A Smith, and Joshua B Tenenbaum. Differentiable physics and stable modes for tool-use and manipulation planning. In Robotics: Science and Systems, 2018.
- [143] US Census Bureau. Annual survey of manufactures industrial robotic equipment: 2018 and 2019, 2022.
- [144] C-YE Wang, Wojciech K Timoszyk, and James E Bobrow. Payload maximization for open chained manipulators: finding weightlifting motions for a puma 762 robot. IEEE Transactions on Robotics and Automation, 17(2):218–224, 2001.
- [145] Thomas Wolf et al. Huggingface’s transformers: State-of-the-art natural language processing. ArXiv, abs/1910.03771, 2019.
- [146] J Zachary Woodruff and Kevin M Lynch. Planning and control for dynamic, nonprehensile, and hybrid manipulation tasks. In 2017 IEEE International Conference on Robotics and Automation (ICRA), pages 4066–4073. IEEE, 2017.

- [147] Shijie Wu, Yihang Zhu, Yunao Huang, Kaizhen Zhu, Jiayuan Gu, Jingyi Yu, Ye Shi, and Jingya Wang. Afforddp: Generalizable diffusion policy with transferable affordance. In Proceedings of the Computer Vision and Pattern Recognition Conference (CVPR), pages 6971–6980, 2025.
- [148] Zhou Xian, Nikolaos Gkanatsios, Theophile Gervet, Tsung-Wei Ke, and Katerina Fragkiadaki. Chaineddiffuser: Unifying trajectory diffusion and keypose prediction for robotic manipulation. In Proceedings of The 7th Conference on Robot Learning, volume 229 of Proceedings of Machine Learning Research, pages 2323–2339. PMLR, 2023.
- [149] Biyun Xie and Anthony A Maciejewski. Maximizing the probability of task completion for redundant robots experiencing locked joint failures. IEEE Transactions on Robotics, 38(1):616–625, 2021.
- [150] Lei Yan, Zonggao Mu, Wenfu Xu, and Bingsong Yang. Coordinated compliance control of dual-arm robot for payload manipulation: Master-slave and shared force control. In 2016 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), pages 2697–2702. IEEE, 2016.
- [151] Lei Yan, Theodoros Stouraitis, João Moura, Wenfu Xu, Michael Gienger, and Sethu Vijayakumar. Impact-aware bimanual catching of large-momentum objects. IEEE Transactions on Robotics, 2024.
- [152] Chenning Yu and Sicun Gao. Reducing collision checking for sampling-based motion planning using graph neural networks. Advances in Neural Information Processing Systems, 34:4274–4289, 2021.
- [153] Kuan-Ting Yu, Maria Bauza, Nima Fazeli, and Alberto Rodriguez. More than a million ways to be pushed. a high-fidelity experimental dataset of planar pushing. In 2016 IEEE/RSJ international conference on intelligent robots and systems (IROS), pages 30–37. IEEE, 2016.
- [154] Yanjie Ze, Gu Zhang, Kangning Zhang, Chenyuan Hu, Muhan Wang, and Huazhe Xu. 3d diffusion policy: Generalizable visuomotor policy learning via simple 3d representations. arXiv preprint arXiv:2403.03954, 2024.
- [155] Andy Zeng, Shuran Song, Johnny Lee, Alberto Rodriguez, and Thomas Funkhouser. Tossingbot: Learning to throw arbitrary objects with residual physics. IEEE Transactions on Robotics, 36(4):1307–1319, 2020.
- [156] Amy Zhang, Rowan McAllister, Roberto Calandra, Yarin Gal, and Sergey Levine. Learning invariant representations for reinforcement learning without reconstruction. In International Conference on Learning Representations, 2021.
- [157] Tony Z Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware. In Robotics: Science and Systems, 2023.
- [158] Jiaji Zhou, Yifan Hou, and Matthew T Mason. Pushing revisited: Differential flatness, trajectory planning, and stabilization. The International Journal of Robotics Research, 38(12-13):1477–1489, 2019.
- [159] Chi Zhu, Yasumichi Aiyama, Tamio Arai, and Atsuo Kawamura. Frictional sliding motion in releasing manipulation. Advanced Robotics, 19(2):141–168, 2005.

- [160] Brianna Zitkovich, Tianhe Yu, Sichun Xu, Peng Xu, Ted Xiao, Fei Xia, Jialin Wu, Paul Wohlhart, Stefan Welker, Ayzaan Wahid, et al. Rt-2: Vision-language-action models transfer web knowledge to robotic control. In Conference on Robot Learning, pages 2165–2183. PMLR, 2023.
- [161] Claudio Zito, Rustam Stolkin, Marek Kopicki, and Jeremy L Wyatt. Two-level RRT planning for robotic push manipulation. In 2012 IEEE/RSJ International Conference on Intelligent Robots and Systems, pages 678–685. IEEE, 2012.